Unity Pro
Concept Application Converter
User Manual

09/2017

33002515.19

www.schneider-electric.com

Schneider Electric

The information provided in this documentation contains general descriptions and/or technical characteristics of the performance of the products contained herein. This documentation is not intended as a substitute for and is not to be used for determining suitability or reliability of these products for specific user applications. It is the duty of any such user or integrator to perform the appropriate and complete risk analysis, evaluation and testing of the products with respect to the relevant specific application or use thereof. Neither Schneider Electric nor any of its affiliates or subsidiaries shall be responsible or liable for misuse of the information contained herein. If you have any suggestions for improvements or amendments or have found errors in this publication, please notify us.

You agree not to reproduce, other than for your own personal, noncommercial use, all or part of this document on any medium whatsoever without permission of Schneider Electric, given in writing. You also agree not to establish any hypertext links to this document or its content. Schneider Electric does not grant any right or license for the personal and noncommercial use of the document or its content, except for a non-exclusive license to consult it on an "as is" basis, at your own risk. All other rights are reserved.

All pertinent state, regional, and local safety regulations must be observed when installing and using this product. For reasons of safety and to help ensure compliance with documented system data, only the manufacturer should perform repairs to components.

When devices are used for applications with technical safety requirements, the relevant instructions must be followed.

Failure to use Schneider Electric software or approved software with our hardware products may result in injury, harm, or improper operating results.

Failure to observe this information can result in injury or equipment damage.

© 2017 Schneider Electric. All Rights Reserved.

Table of Contents

Safety Information. 9
About the Book 13
Part I Requirements and conversion 15
Chapter 1 General Description of the Unity Pro Concept Converter 17
General Description 18
Conversion with the Conversion Wizard 19
Chapter 2 Requirements 21
Concept Version 22
Supported Hardware Platforms 23
Configuration 24
System. 25
EFBs 33
Programming Language SFC 36
Programming Language LD 37
Programming Language ST/IL 49
Programming Language LL984 51
Programming Language FBD 52
Chapter 3 Language Differences. 53
Functions Not Present in Unity 55
EFB Replaced by Function 56
FFBs Not Available For All Platforms 57
INOUT Parameters 62
Parameter Type Changed 63
ANY_ARRAY_WORD Parameters 64
Unique Naming required 65
Incomplete LD Generation. 66
LD Execution Order Changed 67
Edge Detection Difference. 71
Constants 72
Indices in ST and IL. 73
Calculate with TIME and REAL 74
WORD Assignments to BOOL Arrays 75
Topological Address Overlapping 76
Substitute %QD by %MF. 77
Structure Alignment Changed 78

Undefined Output on Disabled EFs 79
Variables at Empty Pins 81
The set action remains active, even when the associated step becomes inactive 82
SFC Section Retains its State When Performing an Online Modification 83
SFCCNTRL Function Block in Unity Behaves Different to Concept . . . 84
Weekday Numbering 85
System Timer 86
Initial Values 87
Macros 89
Chapter 4 Possible application behavior change 91
General 92
Concept Behavior 93
IEC Demands 94
Unity Behavior 97
Consequences 99
Chapter 5 The Conversion Process 105
Conversion Process 105
Chapter 6 Conversion Procedure 107
Exporting a Project from Concept 108
Importing a Project into Unity Pro 109
Missing Datatypes at the Beginning of the Import 110
Converting Only Parts of a Concept Application 111
Removing Accidentally Included Concept Macros 112
Initialization Values 113
If the convertedMomentum application contain more than one XMIT block 114
Part II Blocks from Concept to Unity Pro 115
Chapter 7 BYTE_TO_BIT_DFB: Type conversion 117
Description 117
Chapter 8 CREADREG: Continuous register reading 121
Description 122
Mode of Functioning 125
Parameter description 126
Modbus Plus Error Codes 127
Chapter 9 CWRITREG: Continuous register writing 129
Description 130
Mode of Functioning 133
Parameter description 134

Chapter 10 DINT_AS_WORD_DFB: Type conversion 135
Description 135
Chapter 11 DIOSTAT: Module function status (DIO) 137
Description 137
Chapter 12 GET_TOD: Reading the hardware clock (Time Of Day) . 139
Description 139
Chapter 13 LIMIT_IND_DFB: Limit with indicator 143
Description 143
Chapter 14 LOOKUP_TABLE1_DFB: Traverse progression with 1st degree interpolation 147
Description 148
Detailed description 150
Chapter 15 PLCSTAT: M580 CPU function status DFB 153
PLC Status DFB 153
Chapter 16 PLCSTAT: Quantum PLC function status 159
Description 160
Derived Data Types 162
PLC status (PLC_STAT) 164
RIO status (RIO_STAT) for Quantum 166
DIO status (DIO_STAT) 168
Chapter 17 READREG: Read register 175
Description 176
Mode of Functioning 179
Parameter description 180
Chapter 18 RIOSTAT: Module function status (RIO) 183
Description 183
Chapter 19 SET_TOD: Setting the hardware clock (Time Of Day) . . 187
Description 187
Chapter 20 WORD_AS_BYTE_DFB: Type conversion 191
Description 191
Chapter 21 WORD_TO_BIT_DFB: Type conversion 193
Description 193
Chapter 22 WRITEREG: Write register 197
Description 198
Mode of Functioning 201
Parameter description 202

Part III Import / Export of LL984 Applications 205
Chapter 23 Import 207
Importing an LL984 Application 208
Importing PLCs 216
Importing Specific CPU Configurations 219
Importing Hot Standby Configurations 226
Importing I/O Modules 228
Importing Communication Settings 234
Importing Descriptors and Comments 237
Import Restrictions 239
Importing Application Parts with the Conversion Wizard 243
Chapter 24 Export 245
Exporting/Importing LL984 Segments/Networks 245
Part IV I/O Modules Conversion Table 247
Chapter 25 Supported / Not Supported I/O Modules 249
Quantum Modules 250
Series 800 Modules 256
Series 200/500 Modules 262
SY/MAX Modules 265
Compact Modules 268
Chapter 26 Special Conversion for Compact I/O Modules 275
26.1 Digital Input Modules Conversion 276
Digital Input Modules Conversion 276
26.2 Digital Output Modules Conversion 277
Digital Output Modules Conversion 277
26.3 Digital Mixed Modules Conversion 278
Digital Mixed Modules Conversion 278
26.4 Analog Input Modules Conversion 279
Analog Input Modules Conversion 280
ADU 206/256 281
ADU 205 Voltage and Current 286
ADU 210 Voltage and Current 290
ADU 204/ADU 254 Thermocouple 291
ADU 214 Mixed Voltage, Current, Thermocouple and Resistors 293
ADU 257 One and Two Channel Thermocouple with 32 bit 297
ADU 211/212, ADU 216 Hardware Switch Configured Modules 298
FRQ xxx, ZAE xxx, MOT_xxx, VIC_xxx, Counting and Positioning Modules 300

26.5 Analog Output Modules Conversion 301
Analog Output Modules Conversion 302
DAU 204 303
DAU 208 306
DAU 2x2 307
Appendices 309
Appendix A FAQ Build Errors. 311
General 312
Object Link Creation Error 313
Object Must be Connected to a Successor 314
Link Together with Variable isn't Allowed 315
Data Type 'xxxx' Expected 316
Empty DFB to Replace Obsolete EFB. 322
Undefined Symbol 'xxxx' 323
Call of Non-Function Block 325
Parameter 'xxxx' Has to Be Assigned 328
' xxxx' Is Not a Parameter of 'yyyy' 329
DDT Component Is Missing 330
EHC Parameters Out of Range 331
Not a Valid Address 332
140 NOG 111 00 Configuration Not Converted. 333
E1163 Use of Unconfigured Direct Address 334
The Instance Is Located on an Address That Is Not Configured. 335
Appendix B FAQ Conversion Errors 337
FAQ Conversion Errors 337
Index 343

Safety Information

Important Information

NOTICE

Read these instructions carefully, and look at the equipment to become familiar with the device before trying to install, operate, service, or maintain it. The following special messages may appear throughout this documentation or on the equipment to warn of potential hazards or to call attention to information that clarifies or simplifies a procedure.



The addition of this symbol to a “Danger” or “Warning” safety label indicates that an electrical hazard exists which will result in personal injury if the instructions are not followed.



This is the safety alert symbol. It is used to alert you to potential personal injury hazards. Obey all safety messages that follow this symbol to avoid possible injury or death.

⚠ DANGER

DANGER indicates a hazardous situation which, if not avoided, **will result in** death or serious injury.

⚠ WARNING

WARNING indicates a hazardous situation which, if not avoided, **could result in** death or serious injury.

⚠ CAUTION

CAUTION indicates a hazardous situation which, if not avoided, **could result** in minor or moderate injury.

NOTICE

NOTICE is used to address practices not related to physical injury.

PLEASE NOTE

Electrical equipment should be installed, operated, serviced, and maintained only by qualified personnel. No responsibility is assumed by Schneider Electric for any consequences arising out of the use of this material.

A qualified person is one who has skills and knowledge related to the construction and operation of electrical equipment and its installation, and has received safety training to recognize and avoid the hazards involved.

BEFORE YOU BEGIN

Do not use this product on machinery lacking effective point-of-operation guarding. Lack of effective point-of-operation guarding on a machine can result in serious injury to the operator of that machine.

⚠ WARNING

UNGUARDED EQUIPMENT

- Do not use this software and related automation equipment on equipment which does not have point-of-operation protection.
- Do not reach into machinery during operation.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

This automation equipment and related software is used to control a variety of industrial processes. The type or model of automation equipment suitable for each application will vary depending on factors such as the control function required, degree of protection required, production methods, unusual conditions, government regulations, etc. In some applications, more than one processor may be required, as when backup redundancy is needed.

Only you, the user, machine builder or system integrator can be aware of all the conditions and factors present during setup, operation, and maintenance of the machine and, therefore, can determine the automation equipment and the related safeties and interlocks which can be properly used. When selecting automation and control equipment and related software for a particular application, you should refer to the applicable local and national standards and regulations. The National Safety Council's Accident Prevention Manual (nationally recognized in the United States of America) also provides much useful information.

In some applications, such as packaging machinery, additional operator protection such as point-of-operation guarding must be provided. This is necessary if the operator's hands and other parts of the body are free to enter the pinch points or other hazardous areas and serious injury can occur. Software products alone cannot protect an operator from injury. For this reason the software cannot be substituted for or take the place of point-of-operation protection.

Ensure that appropriate safeties and mechanical/electrical interlocks related to point-of-operation protection have been installed and are operational before placing the equipment into service. All interlocks and safeties related to point-of-operation protection must be coordinated with the related automation equipment and software programming.

NOTE: Coordination of safeties and mechanical/electrical interlocks for point-of-operation protection is outside the scope of the Function Block Library, System User Guide, or other implementation referenced in this documentation.

START-UP AND TEST

Before using electrical control and automation equipment for regular operation after installation, the system should be given a start-up test by qualified personnel to verify correct operation of the equipment. It is important that arrangements for such a check be made and that enough time is allowed to perform complete and satisfactory testing.

⚠ WARNING

EQUIPMENT OPERATION HAZARD

- Verify that all installation and set up procedures have been completed.
- Before operational tests are performed, remove all blocks or other temporary holding means used for shipment from all component devices.
- Remove tools, meters, and debris from equipment.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

Follow all start-up tests recommended in the equipment documentation. Store all equipment documentation for future references.

Software testing must be done in both simulated and real environments.

Verify that the completed system is free from all short circuits and temporary grounds that are not installed according to local regulations (according to the National Electrical Code in the U.S.A, for instance). If high-potential voltage testing is necessary, follow recommendations in equipment documentation to prevent accidental equipment damage.

Before energizing equipment:

- Remove tools, meters, and debris from equipment.
- Close the equipment enclosure door.
- Remove all temporary grounds from incoming power lines.
- Perform all start-up tests recommended by the manufacturer.

OPERATION AND ADJUSTMENTS

The following precautions are from the NEMA Standards Publication ICS 7.1-1995 (English version prevails):

- Regardless of the care exercised in the design and manufacture of equipment or in the selection and ratings of components, there are hazards that can be encountered if such equipment is improperly operated.
- It is sometimes possible to misadjust the equipment and thus produce unsatisfactory or unsafe operation. Always use the manufacturer's instructions as a guide for functional adjustments. Personnel who have access to these adjustments should be familiar with the equipment manufacturer's instructions and the machinery used with the electrical equipment.
- Only those operational adjustments actually required by the operator should be accessible to the operator. Access to other controls should be restricted to prevent unauthorized changes in operating characteristics.

About the Book

At a Glance

Document Scope

This document describes the functionality and performance scope of the Concept Application Converter for Unity Pro.

Validity Note

This document is valid for Unity Pro 13.0 or later.

Related Documents

Title of Documentation	Reference Number
Unity Pro, Program Languages and Structure, Reference Manual	35006144 (English), 35006145 (French), 35006146 (German), 35013361 (Italian), 35006147 (Spanish), 35013362 (Chinese)
Unity Pro, Operating Modes	33003101 (English), 33003102 (French), 33003103 (German), 33003104 (Spanish), 33003696 (Italian), 33003697 (Chinese)
LL984 Editor, Reference Manual, LL984 Specifics	EIO0000000549 (English), EIO0000000802 (French), EIO0000000803 (German), EIO0000000804 (Italian), EIO0000000805 (Spanish), EIO0000000806 (Chinese)
Modicon Modbus Plus Network Planning and Installation Guide	31003525

You can download these technical publications and other technical information from our website at http://www.schneider-electric.com/en/download

Part I
Requirements and conversion

Overview

This section contains requirements and information about the conversion.

What Is in This Part?

This part contains the following chapters:

Chapter	Chapter Name	Page
1	General Description of the Unity Pro Concept Converter	17
2	Requirements	21
3	Language Differences	53
4	Possible application behavior change	91
5	The Conversion Process	105
6	Conversion Procedure	107

Chapter 1
General Description of the Unity Pro Concept Converter

Overview

This chapter contains a general description of the Unity Pro Concept Converter.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
General Description	18
Conversion with the Conversion Wizard	19

General Description

Brief description

The Concept Converter is an integrated function in Unity Pro, which is used to convert Concept applications into Unity Pro. This means that Concept programs can also be operated in Unity Pro.

Substitute objects are used in place of objects that cannot be converted. The Unity Pro project can be analyzed using the main menu **Create → Analyze Project**. Subsequently messages are displayed in the output window to find the substitute objects.

Elements on the Concept application that can not be converted are logged in the conversion report.

Descriptions of the respective procedures are provided in chapter Conversion procedure *(see page 107)*.

NOTE: Back conversion from Unity Pro to Concept is not possible.

⚠ WARNING
UNEXPECTED APPLICATION BEHAVIOR
The Concept Converter translates the application but does not ensure its correct operation. Test the application after the conversion.
Failure to follow these instructions can result in death, serious injury, or equipment damage.

Conversion

The conversion is carried out in 4 steps:

1. **In Concept:** Export the Concept application using the Concept converter which creates an ASCII file (*.ASC).
   NOTE: Do not use the **project with used DFBs (Re-Connect to Equal)** option when creating the *.ASC file. Unity Pro cannot import the application if this option is used.
2. **In Unity Pro:** Open the exported ASCII file (*.ASC) in Unity Pro.
3. **In Unity Pro:** Automatic conversion of the ASCII file into Unity Pro source file format.
4. **In Unity Pro:** Automatic import of the Unity Pro source file.

Conversion options for Concept projects

You can enter conversion options *(see Unity Pro, Operating Modes)* in Unity Pro before the conversion that have effects on the conversion result.

Atrium cannot be converted

Atrium configurations cannot be converted into Unity Pro.

Conversion Wizard

Please refer to Conversion with the Conversion Wizard *(see page 19)*.

Conversion with the Conversion Wizard

Application Conversion as a Whole

To convert an application as a whole, while keeping the same PLC family, and selection of application parts or remapping of I/O objects is not needed, use the Concept Application Converter directly via the Unity Pro menu **File → Open**.

Partial Application Conversion

To convert an application partially and/or the PLC family must be changed or remapping of I/O objects is needed, use the conversion wizard via the Unity Pro menu **Tools → Convert Partially**.

For detailed information, please refer to the Introduction *(see Unity Pro, Operating Modes)* to the Conversion Wizard.

Conversion Wizard

The conversion wizard is an integrated part of Unity Pro.

You can use it to

- convert applications, exported out of legacy applications (Concept and PL7) to Unity Pro
- convert legacy applications partially or as a whole
- remap I/O objects (channels, variables etc.) during conversion by means of the wizard
- adapt concurrently the hardware configuration of the new application in Unity Pro
- modify the amount of used memory in the CPU

The conversion wizard is available if you have chosen to install a converter (e.g. Concept Application Converter) during the setup of Unity Pro.

General Procedure

General procedure to convert a legacy application to Unit Pro

Step	Action
1	Export your application out of your legacy programming system (e.g. as an ASC file out of Concept).
2	Create a new application in Unity Pro selecting a CPU with enough memory and the I/O access capabilities needed. Optionally you can configure the I/O modules expected to be needed but you can modify the hardware configuration even later (see step 6).
3	Launch the conversion wizard in Unity Pro via **Tools → Convert Partially**. **Result:** The conversion wizard asks you to select the exported legacy source file.
4	Select the exported legacy source file. **Result:** The converter analyzes the source file and displays the result in the 3 tabs of the conversion wizard.
5	Select the parts of the application (or the complete application) to be converted in the **Structure** tab.

Step	Action
6	Remap the I/O objects for getting them compliant with the new hardware configuration. Concurrently you can modify the hardware configuration of the new application in Unity Pro. **Note:** To save a backup file of your intermediate I/O mapping you can use the **Save** button. With **Load** you can reload your latest saved intermediate I/O mapping.
7	After finishing all your selections and manual modifications click **OK**. **Result:** The converter applies the defined remapping to the selected parts of the source file and imports the results into the opened Unity Pro Application.
8	Continue working on the opened application, save it or export as an XEF file.

Conversion Wizard Documentation

For detailed information on the conversion wizard, please refer to the *Operating Modes Manual -> Conversion Wizard*.

Chapter 2
Requirements

Overview

This chapter contains the requirements for converting a Concept project into a Unity Pro project.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Concept Version	22
Supported Hardware Platforms	23
Configuration	24
System	25
EFBs	33
Programming Language SFC	36
Programming Language LD	37
Programming Language ST/IL	49
Programming Language LL984	51
Programming Language FBD	52

Concept Version

General

Projects from Concept versions 2.11 and 2.5 and 2.6 can be converted to Unity Pro projects.

Preconversion

If an older version of a Concept project should be converted to Unity Pro, the project must be first converted within Concept to bring it to version 2.6 status for security reasons.

Supported Hardware Platforms

General

The Concept Converter accepts applications using the following hardware platforms:

- Quantum
- Compact
- Momentum

Manual Corrections

NOTE: The Concept Converter converts as far as possible the modules when equivalencies are existing. It is mandatory to check the result according to the process needs. The settings of the hardware modules (parameters) are not converted but set to default values and must be entered for each module in Unity Pro. Channel objects are converted as far as possible. Nevertheless, the program may have to be adapted according to the different behavior with the original module.

Quantum Applications

Concept Quantum applications are converted to Unity Pro Quantum applications.

NOTE: Concept Quantum applications cannot be converted to Unity Pro Momentum applications.

Compact Applications

With global conversion, Concept Compact applications are converted to Quantum applications with a default hardware configuration containing a CPU (140 CPU 534 14A/U) and a power supply (140 CPS 424 00).

With partial conversion (conversion wizard) it is recommended to prepare a Modicon M340 hardware configuration.

Momentum Applications

With global conversion, Concept Momentum applications are converted to Quantum applications with a default hardware configuration containing a CPU (140 CPU 534 14A/U) and a power supply (140 CPS 424 00).

With partial conversion (conversion wizard) it is recommended to prepare a Modicon M340 hardware configuration.

Safety PLC

NOTE: It is not possible to recover an application from Concept to Unity Pro Safety PLC. To build a safety system, refer to the *Safety Manual*.

Configuration

General

Concept sections created using the LL984 programming language appear as well as non-IEC LL984 sections in Unity.

Restrictions for old LL984 configurations

The following points from LL984 configurations are no longer supported by Unity Pro:

Not supported by Unity Pro	Supported by Unity Pro
Loadables	Necessary functionality of system loadables has been integrated into Unity Pro. Unity Pro does NOT provide equivalents for all other loadables.
ASCII messages	Will not be converted.
6x range (register in expanded memory)	Will not be converted.
Data protection configuration extension	Will not be supported.

Hot Standby (HSBY)

There are the following differences for converting the Concept Hot Standby to Unity Pro:

Concept	Unity Pro
The Hot Standby system in Concept is based on the 140 CHS 111 00 module.	This module is **no** longer supported by Unity Pro.
The 140 CHS 111 00 module is purely a Hot Standby Module for **a single** slot. The power is supplied via the rack.	The CPU 671 60 module is a CPU module for **two slots** with a fixed assigned connection for data exchange. The Hot Standby system is integrated into the CPU 671 60 module.

The Concept converter replaces the CPU from Concept with the new Hot Standby CPU 671 60 and the Concept Hot Standby Module 140 CHS 111 00 is removed. All Hot Standby parameters will be transferred to the Unity application.

NOTE: It is not possible to recover an application from Concept to Unity Pro safety PLC. To build a safety system, refer to the *safety manual*.

NOTE: As the CPU in Concept only requires **one** slot, but the new Unity CPU requires **two**, overlaps in the rack my arise. These must be resolved manually by the user.

System

Security

The access authorizations defined in Concept are **not** converted to Unity Pro.

Security under Unity Pro does **not** - refer to the corresponding installation as it does under Concept.

Program Execution

Program execution using Concept and Unity Pro are different. It can lead to different behavior during the first program run after a restart.

Program execution for Concept:

1. Write the outputs (program run n-1)
2. Read the inputs (program run n)
3. Program processing

Program execution for Unity Pro:

1. Read the inputs
2. Program processing
3. Write the outputs

Example:

In Concept, you have assigned a 4x register to a digital output and stopped the PLC when the value is "true". After a restart, the value remains "True" during the first program run even if you have modified the process conditions.

Specified Execution Order

The execution order in the function block language in Concept is determined first of all by how the FFBs are positioned. If the FFBs are then linked graphically, the execution order is determined by the data flow. After this the execution order can be changed based on the intention.

In Unity Pro after conversion it is not possible to see in what order the FFBs were positioned. Therefore, whenever the order cannot be determined unambiguously from the data flow rule alone, the order is defined by the Concept project.

The defined execution sequence is shown by means of a rectangle with the step number in the upper right-hand corner of the FFB.

Single Sweep Function

The single sweep function is no longer supported by Unity Pro.

The corresponding functionality can be realized in Unity Pro using the Debug function "Breakpoints".

EFB Download

Using Concept, all platform dependent EFBs can be placed at any time and loaded in all PLC platforms. Any errors detected during the program execution are written to the message memory.

In Unity Pro, only valid EFBs can be placed. Download to the PLC is only possible if the EFBs used are consistent with the PLC platform.

Reference Data Editor (RDE)

RDE tables created in Concept are converted to Unity Pro when they are placed in the same directory as the Concept ASCII file.

Global Variable Values

Because of different restart behaviors after a power outage, it is possible that the global variable states of two PLCs that restart differently are not the same after the first program run.

There are two different types of restart behavior:

1. All 16 bit PLCs (all Momentum, Quantum 113, 213, 424) continue executing the program at the point at which it was interrupted.
2. All 32 bit PLCs (Quantum 434, 534) start the program run at the beginning.

Unity Pro supports the 1st type of restart behavior described above.

State RAM

The Concept State RAM register addresses are assigned to IEC conforming addresses in Unity Pro.

I/O module addresses are converted either to "flat" addresses or to topological addresses.

State RAM Register Without I/O Module

State RAM register addresses **without** assigned I/O modules are represented with "flat" addresses:

Concept	Unity Pro
4x	%MWx
3x	%IWx (1)
0x	%Mx
1x	%Ix
(1) = If Modicon M340 is the target platform, there is no equivalent for input State RAM registers (%IWx). The addresses are converted formally to flat addresses and must be corrected by the user.	

For this, the register number is added to the end of the introduction.

The address reads as follows:

```
%[IM][W]Register number
```

State RAM Register With I/O Module

State RAM register addresses **with** assigned I/O modules can either be represented on Quantum with "flat" addressing as described above or with topological addressing.

To define that State RAM register addresses will be converted to topological addressing, open the **Conversion Settings** tab via **Tools → Options** in Unity Pro and activate the **Generate Topological Addresses for Quantum** check box before converting.

If the check box is not activated, the State RAM register addresses are converted to "flat" addresses (for Quantum only).

If Compact or Momentum applications are converted with the conversion wizard, topological addressing is used by default, regardless whether the check box is activated or not.

State RAM register addresses with assigned I/O modules (topological)

Concept	Unity Pro
4x	%QWt
3x	%IWt
0x	%Qt
1x	%It
t = topological description	

The following information is read from the configuration to provide a sufficient topological description of State RAM register addresses with assigned I/O modules:

- Bus number (corresponds to drop head in Concept)
- Drop
- Rack
- Module
- Channel

The complete address reads as follows:

```
%[IQ][W]<\Busnumber.Drop\>Rack.Module.Channel
```

State RAM Assignment Using Derived Data Types

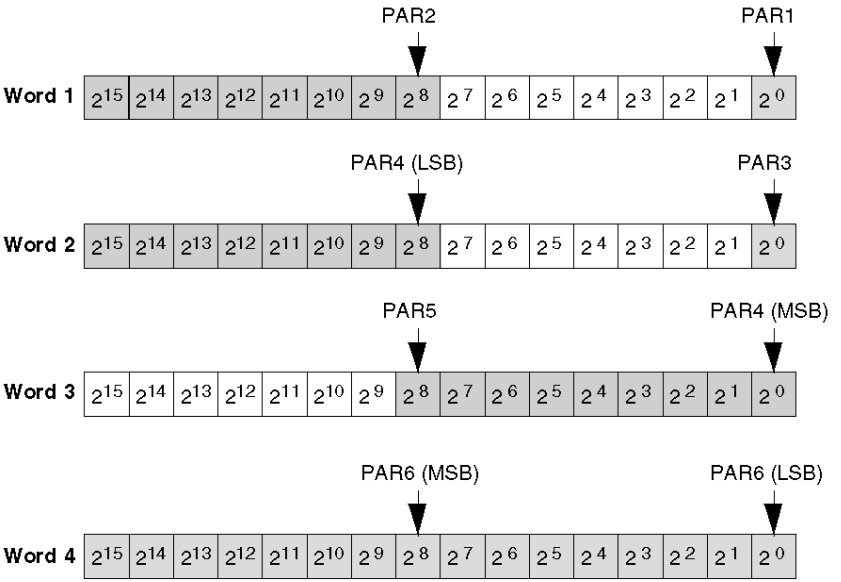
In Concept, data structure elements begin at BYTE limits.

In Unity Pro, data structure elements begin at WORD limits.

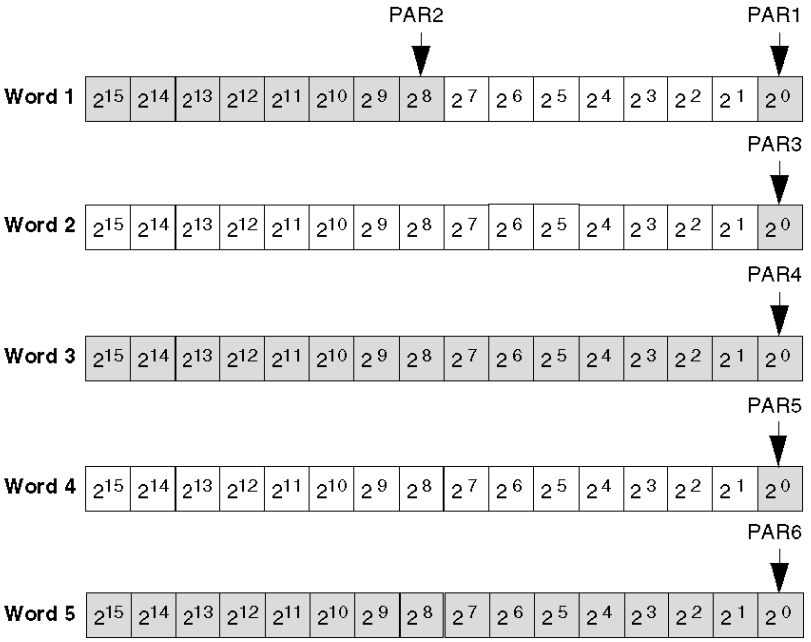
Example of a derived data type:

```
TYPE
  SKOE:
   STRUCT
   PAR1: BOOL;
   PAR2: BYTE;
   PAR3: BOOL;
   PAR4: WORD;
   PAR5: BOOL;
   PAR6: WORD;
   END_STRUCT;
END_TYPE
```

The derived data types are stored in the state RAM when using **Concept**:



The same derived data types are stored in the state RAM when using **Unity Pro**:



Timer, Date, Battery Monitoring

Timer address, date/time of day and the battery monitoring can no longer be assigned to the State RAM with Unity Pro. All required information can be accessed via the control panel.

When Concept is converted to Unity Pro, DFBs are created which can be simulated in Unity Pro without further manual modifications of these functionalities.

NOTE: The Concept Timer Register is 16 bits long and has an accuracy of 10 ms. The equivalent system word %SW18 in Unity Pro is 32 bits long and has an accuracy of 100 ms. If this accuracy is not sufficient, the FREERUN function from the System library can be used, which delivers accuracy of up to 1 ms.

NOTE: When dealing with days of the week, the value 1 corresponds to **Sunday** in Concept and **Monday** in Unity Pro.

Quantum Diagnostics Words

In Unity, the diagnostics words are specified to be a certain number:

- Local I/O: 16 Words
- RIO I/O: 16 Words
- DIO I/O: 16 Words

In Concept it was also possible to specify a smaller number of diagnostics words for the individual I/Os.

Keep this difference in mind, since it can cause problems.

Topological Addresses

The topological addresses are assigned so that if the hardware configuration remains the same, they occupy the same I/O connections as they were assigned in Concept.

The user sees the hardware addresses in Unity Pro that they are using, without having to carry out the intermediate step via the State RAM.

Located Variable

Located BOOL variables in Concept are converted to EBOOL variables in Unity Pro.

Unity Pro provides this new EBOOL variable for the detection of transitions (edges). This "Elementary BOOL type" is used for %Ix, %Mx and unlocated variables.

EBOOL variables can be forced.

The EBOOL variable provides three informational items:

- Current value
- Historical value
- Force information.

Only the current value can be accessed, the other values can only be accessed via product specific functions.

Longer Cycle Time via EBOOL

In Unity, as opposed to Concept, the edge and force information is updated from EBOOL variables during program runtime.

For this reason on the Quantum CPU 434, CPU 534 and CPU 311 platforms the assignment of EBOOL variables is only **half as fast** as the assignment of BOOL variables.

NOTE: If you need variables in the signal memory, use BOOL variables and assign them to the memory area %MW (e.g. BoolVar : BOOL AT %MW10). Otherwise use unlocated BOOL variables.

Constants

Constants in Concept are converted to write-protected variables in Unity Pro.

Unity Pro does not provide constants. Comparable functionality is achieved using write-protected variables.

%Mx Register

In Concept, the 0x registers are **not buffered**. They are reset to zero with every warm restart.

In Unity Pro, the %Mx registers are **buffered** ("RETENTIVE", "VAR_RETAIN"), i.e. Conform to IEC.

Do not use the possibility to set the 0x register to zero on every warm restart if you use a project in Concept that you want to convert to Unity Pro.

NOTE: If you require non-buffered behavior, define the warm restart event with the SYSSTATE function block and explicitly copy the value 0 (zero) to the %Mx register.

Forced Outputs (%M)

⚠ WARNING

UNEXPECTED SYSTEM BEHAVIOR

Do not relay on the Memory Protect switch.

The behavior of forced outputs (%M) between Modsoft/Proworx/Concept and Unity Pro has changed.

- With Modsoft/ProWORX/Concept you **cannot** force outputs when the Memory Protect switch of the Quantum CPU is set to the "On" position.
- With Unity Pro you **can** force outputs even when the Memory Protect switch of the Quantum CPU is set to the "On" position.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

⚠ WARNING

UNEXPECTED SYSTEM BEHAVIOR

Reforce the outputs following a cold start.

The behavior of forced outputs (%M) between Modsoft/Proworx/Concept and Unity Pro has changed.

- With Modsoft/ProWORX/Concept, forced outputs **maintain** their values following a cold start.
- With Unity Pro, forced outputs **lose** their values following a cold start.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

Quantum Remote I/O Control

In Concept, only LL984 sections can be assigned I/O stations (Drops). This is not possible in Concept projects with IEC conforming sections (FBD, LD, SFC, IL, ST).

Unity Pro offers this option, in which a logic is recreated in accordance with LL984. This logic must be entered manually, however.

Example of a section processing order in Unity Pro:

```
Section n-2
Section n-1
RIO call (u,v,w)
Section n
Section n+1
RIO call (u+1,w,x)
Section n+2
RIO call (u+2,x,y)
```

RIO (x,y,z) is the explicit I/O call here:

- Write the outputs to the I/O station x.
- Wait at the inputs of the I/O station y.
- Prepare the inputs of the I/O station z.

NOTE: Take these new settings into consideration when structuring your project.

Setting Variables Cyclically

Unlocated variables **cannot** be set cyclically in Unity Pro. (It is possible in Concept).

If you need to set variables cyclically in your project, you should use located variables.

%Mx/%1x registers (EBOOL) can be forced.

%MWx/%IWx registers can be set cyclically (only numerical values).

EFBs

General

The following options are available for converting Concept EFBs to Unity Pro:

- The EFBs are also supported in Unity Pro; they are mapped on a one to one basis.
- The EFBs are **no longer** supported in Unity Pro.
  Instead of EFBs appropriate DFBs are placed in the application. The functionality remains unaffected by this.
- The EFBs are **no longer** supported by Unity Pro.
  Instead of EFBs, DFBs with no programmatic content are placed in the application. These DFBs contain all the Concept parameters.
  An error message is displayed that says that the programmatic content for these DFBs must still be created.

Generic EFs

There are only a few generic elementary functions in Concept (EFs) e.g. MOVE, SEL, MUX. With many other functions, the elementary data type is added to the name of the function.

In Unity Pro, many of these functions are used without the elementary data type added to the name (as defined in IEC 61131). Therefore, the converter removes the added data type from the name of the function.

In some cases, the use of generic functions in Unity Pro will lead to analytic errors. In these cases, disable the **Generate Generic EFs** check box.

Open the **Conversion Settings** tab via **Tools → Options** in Unity Pro to enable/disable the **Generate Generic EFs** check box before converting.

- When this checkbox is enabled, the converter removes the added data type from the name of the function.
- When this checkbox is not enabled, the converter will leave the added data type in the name of the function.

DIAGNO Library

When converting Concept to Unity Pro for all DIAGNO blocks the station parameter is omitted.

The following EFBs from the DIAGNO library in Concept are converted to empty DFBs in Unity Pro.

- ACT_DIA
- XACT_DIA
- ERR2HMI
- ERRMSG

NOTE: These DFBs, created in Unity Pro have all the Concept parameters but no programmatic content. An error message is displayed that says that the programmatic content for these DFBs must still be created.

During the program creation in Unity Pro replace the DFBs ACT_DIA, and XACT_DIA with the DFB XACT.

For all DIAGNO blocks which can be extended in Concept (D_PRE, D_GRP ...), the extensible inputs (IN1 ... INx) are **gathered together in one** input. This is implemented using a nested logic AND link. In the FBD language the AND block is positioned at the same location as the DIAGNO block by the converter. This overlap must be resolved manually by the user.

SYSTEM Library

The SKP_RST_SCT_FALSE and LOOPBACK EFBs cannot be used in Unity Pro.

FUZZY Library

The FUZZY library is not supported with the normal Unity Pro range but can be installed as an optional library.

HANDTABL Library

The HANDTABL library is no longer supported by Unity Pro.

EXPERTS Library

The following Concept EFBs are converted to DFBs in Unity Pro:

- ERT_TIME
- SIMTSX22
- EFBs from the EX family
- EFBs from the MVB family
- EFBs from the ULEX family

NOTE: These DFBs, created in Unity Pro have all the Concept parameters but no programmatic content. An error message is displayed that says that the programmatic content for these DFBs must still be created.

The data structures DPM_TIME and ERT_10_TTAG from the time stamp module 140 ERT 854 10 have been changed. The MS element was broken up into MS_LSB and MS_MSB. For more information about this, see *State RAM Assignment Using Derived Data Types, page 28*.

Outputs which describe data structures must be assigned event variables using the (=>) assignment operator within the parameter brackets in the ST and IL languages. This happens automatically during conversion (from Unity 2.0 onwards). The functionality remains the same but the section of the program looks a little different.

EFBs that Use Time Functions

In Unity Pro, function components using Time functions (Timer, Diagnostic, Control Components) remain in RUN mode, even if the SPS is set to STOP mode.

⚠ CAUTION
UNEXPECTED BEHAVIOR OF THE CONTROL Function components using Time functions behave differently in Unity Pro and Concept. You must take these different behaviors into consideration during the conversion of Concept applications. **Failure to follow these instructions can result in injury or equipment damage.**

Converted EFBs

During conversion, Unity Pro standardizes the EFB offer by grouping redundant EFBs. The respective EFBs are automatically converted and the project adjusted accordingly.

Renamed EFBs

The following diagnostics EFBs are renamed when converting Concept to Unity Pro:

Concept	Unity Pro
XACT	D_ACT
XREA_DIA	D_REA
XLOCK	D_LOCK
XGRP_DIA	D_GRP
XDYN_DIA	D_DYN
XPRE_DIA	D_PRE

The Quantum configuration EFB for the Backplane Expander 140 XBE 100 00 is renamed when converting Concept to Unity Pro:

Concept	Unity Pro
XBP	XBE

Programming Language SFC

General

For some programming languages there are restrictions to observe when converting a project from Concept to Unity Pro.

Parallel/Alternative Sequence

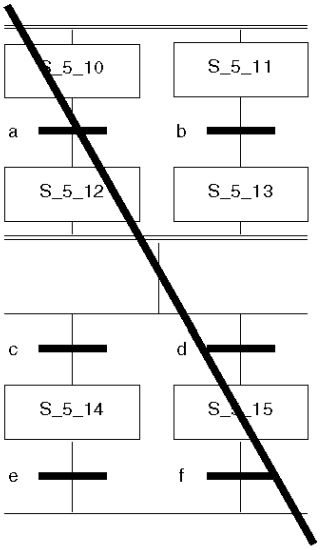
A parallel branch may not be directly followed by an alternative branch.

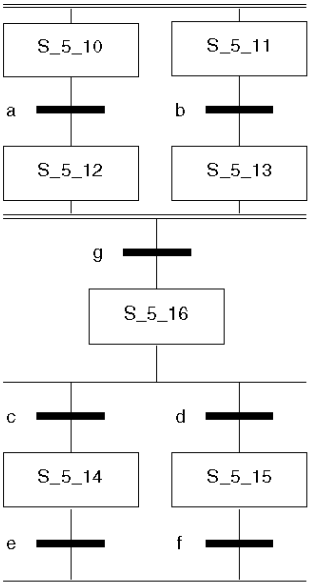
This type of sequence is not permitted according to IEC 1131.

Unity Pro does **not** support this type of sequence, although it is possible in Concept.

The converter transfers this type of project to Unity Pro, but manual modifications are subsequently required.

This problem can be solved by inserting an dummy step between the branches.





Programming Language LD

General

For some programming languages there are restrictions to observe when converting a project from Concept to Unity Pro.

Conversion of the picture

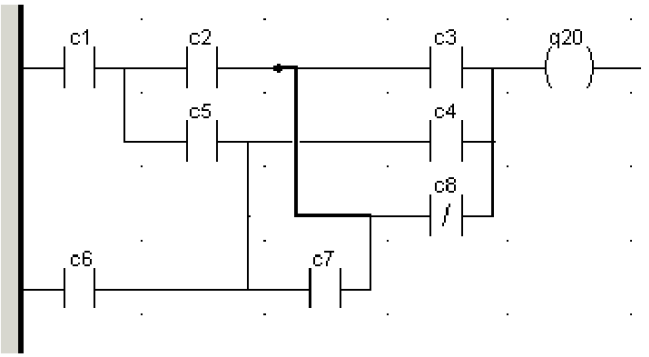
When converting a Concept project to Unity Pro, the ladder diagram LD Picture is also converted, which can lead to a restructuring of the picture.

Crossovers with connections between Boolean objects

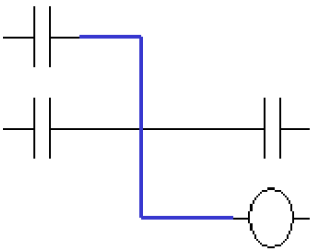
In Concept, FFB connections between Boolean objects may be edited.

This may result in crossovers.

Example of an FFB connection between Boolean objects (coils, contacts, horizontal, and vertical connections) in Concept:



Following the conversion from Concept to Unity, an FFB connection between Boolean objects may look like this:

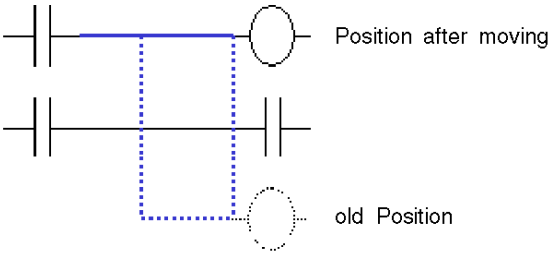


In the Unity LD Editor, such FFB connection may be:

- deleted,
- moved,
- copied and pasted.

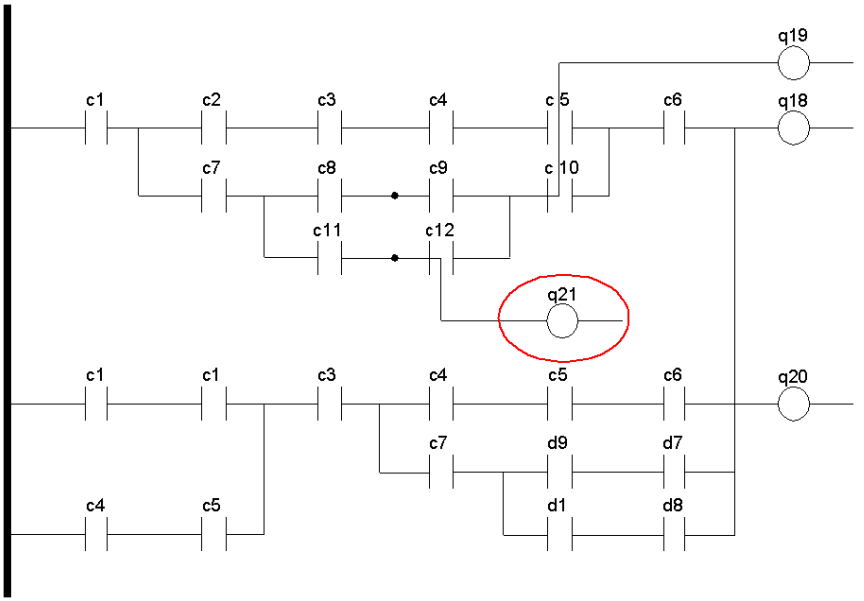
However, such FFB connection **cannot** be created in the Unity LD Editor.

The FFB connection will remain after moving in Unity.



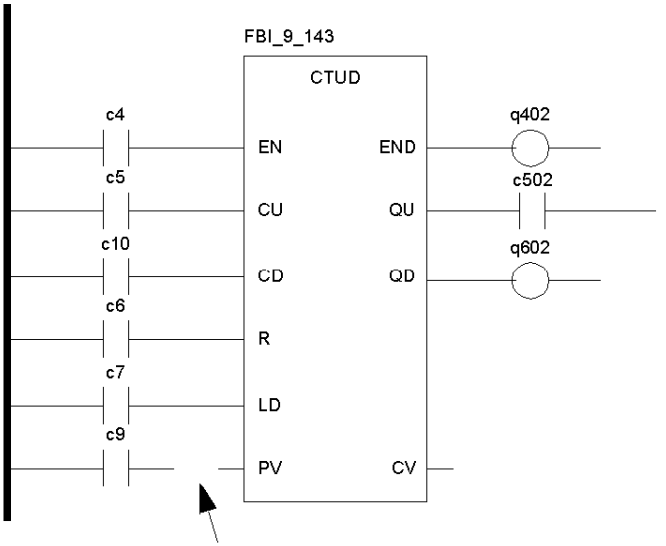
Connection to the right bus bar

A connection to the right bus bar is no longer required.



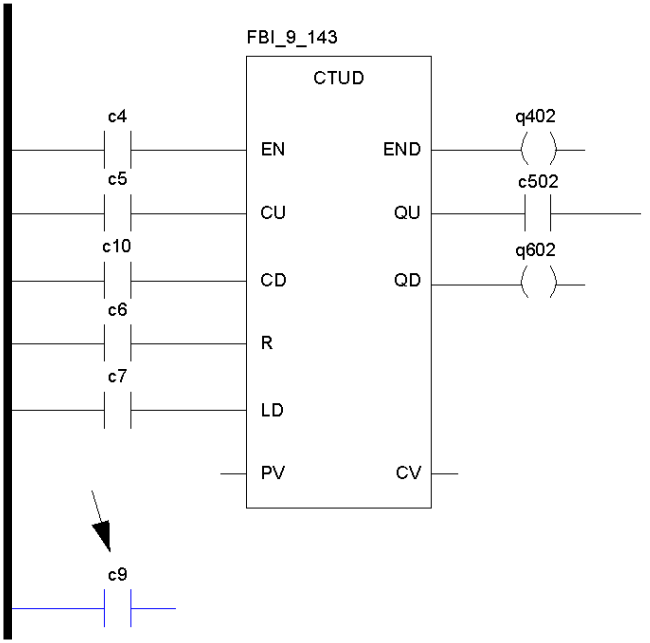
Automatically created connections

In Concept, the contact `c9` is not connected with INPUT PV of the component.



In Unity, the contact `c9` would automatically be connected with INPUT PV because both cells directly border in Concept.

During the conversion from Concept to Unity, the contact `c9` is therefore moved down to avoid the creation of an automatic connection in Unity.



Conversion of the output picture

During the conversion it is desired that the conversion of the picture from Concept to Unity is as exact as possible. To achieve this, the following rules are applied.

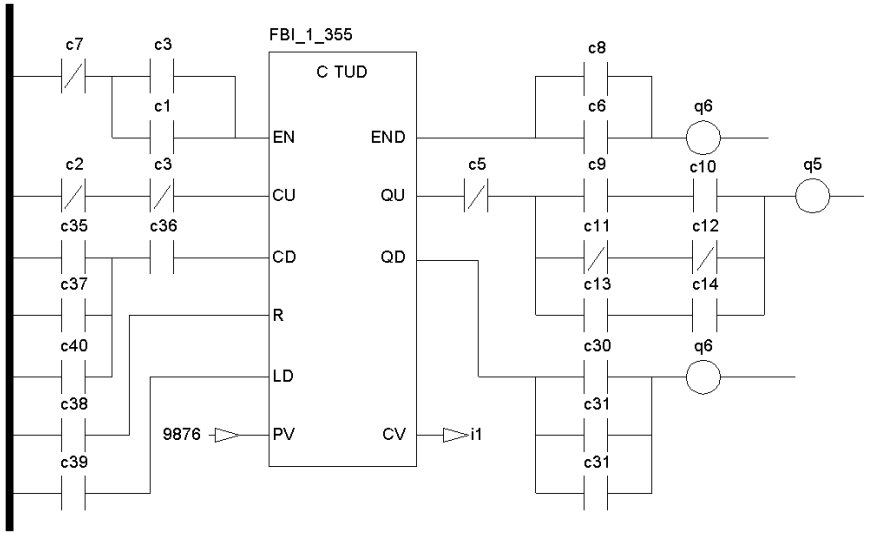
Rules for Object Positioning:

- The distance between two objects must be at least one cell.
- When two FFBs are connected, the minimum distance must equal the number of cells of the first FFB's width.
- The cells in Unity are smaller. If an FFB partially occupies another cell, an additional cell is required for the FFB.
- If an object (contact or coils) has a vertical connection (OR Link), this vertical connection will be located at the end of the cell of the object.
- An additional cell is required if:
  - a vertical connection (OR Link) with an INPUT FFB exists
  - the source FFB has output variables
  - the target FFB has input variables
- A coil may not be directly connected to the left bus bar.

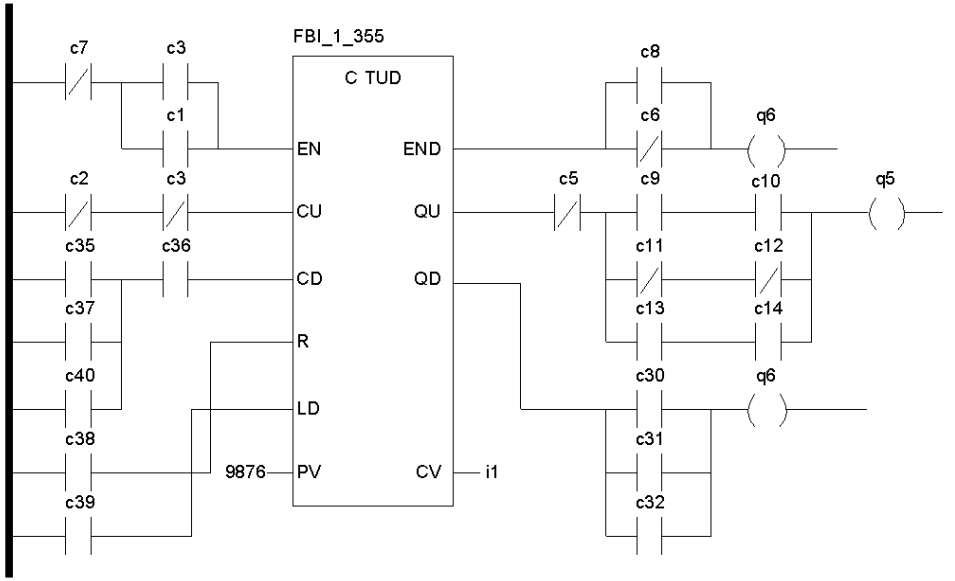
Rules for the conversion of FFB connections:

- FFB connections between variables/constants and FFBs will be ignored. In these cases, Unity will automatically create a connection.
- Purely horizontal FFB connections between objects that are not FFBs will be replaced with horizontal connections with multiple segments.
- When two OR objects are connected, a horizontal connection is first connected to the right side of the source OR object. An FFB connection will then be created between this horizontal connection and the target object. This occurs because the two OR objects would otherwise be combined during the import into Unity.
- Each point of the left bus bar can only be occupied by one connection.

Example of a picture in Concept:



The picture after the conversion to Unity.



The following actions were performed during the conversion according to the rules above:

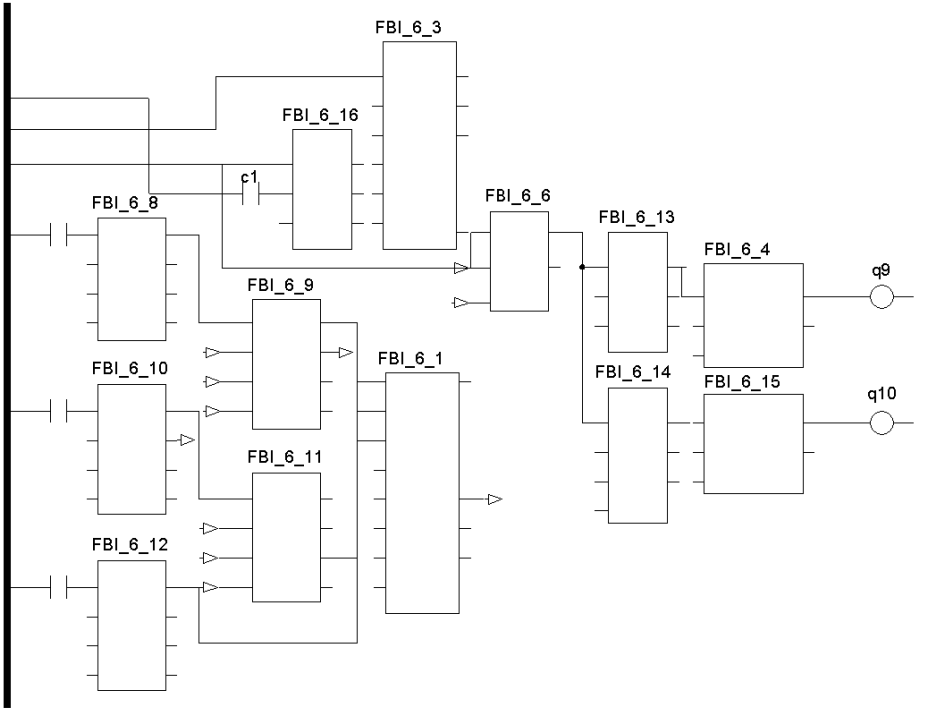
- The space that is occupied by the FFB was expanded to two columns.
- One column each was added at the Input and Output sides of the FFB.
- The connections between coils/contacts and the FFB were realized with FFB connections, not with horizontal connections with multiple segments.

Recognize and disconnect LD Networks

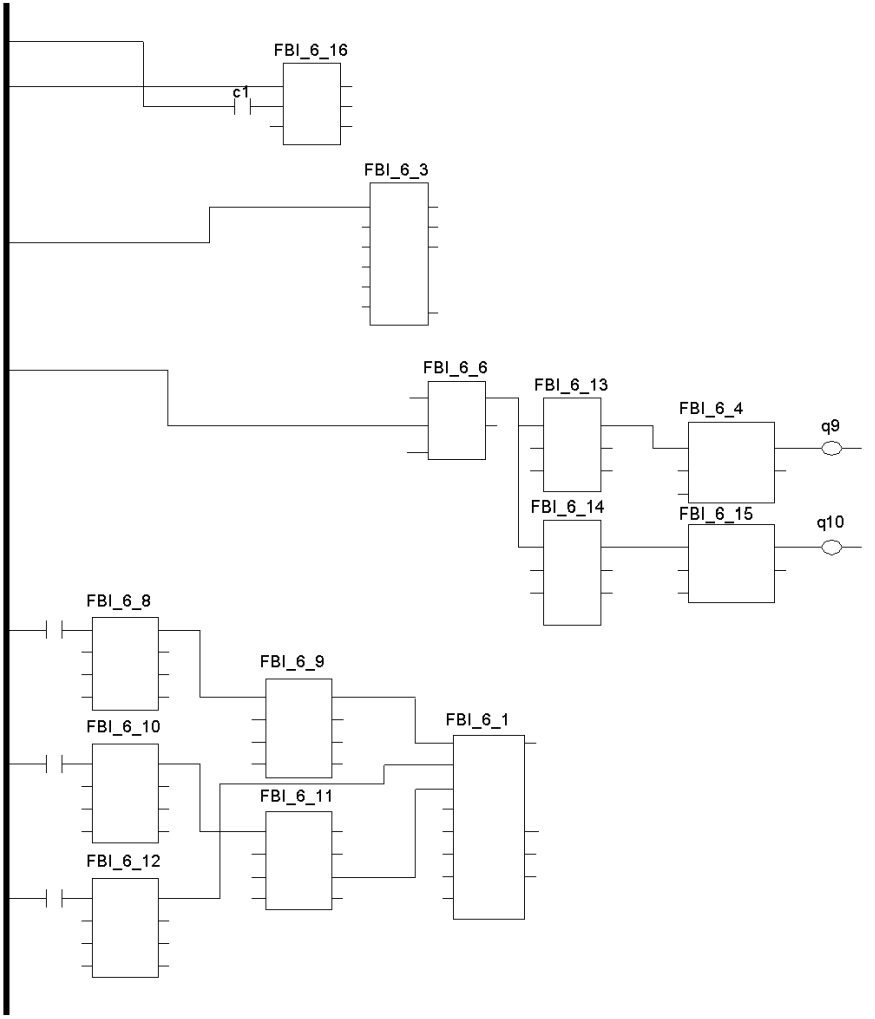
The converter must recognize networks in LD sections during the conversion. To achieve this, the following rules are applied:

- An LD Network is a group of objects that are connected with each other without any other connections to other objects (except the bus bar).
- The minimum distance is always applied to a complete column of a network. This means that if an object of a column requires a certain minimum distance, all other objects are also moved with a higher or equal horizontal position.
- If there are several networks in the same row in Concept, the following network will be moved vertically until it no longer occupies the same rows with the preceding network.
- To avoid undesired automatically created FFB connections, the space that is occupied by an FFB and its connection space will be checked for crossovers. In the event of crossovers the following objects will be moved horizontally.

Schematic diagram of an LD Network in Concept with crossovers



Schematic diagram of an LD Network after the conversion to Unity



Separate LD Networks

IEC LD sections contain many independent graphic areas (networks).

During the conversion of IEC LD sections, additional columns are added to the networks to avoid undesired automatically generated links in Unity Pro.

If the additionally inserted columns were to extend across the entire section, the original graphic would be modified too much. Therefore, the sections are divided in networks during the conversion and additional columns are only inserted for the associated network.

Inserting additional columns may cause a network to exceed its maximum section width and it is then wrapped into the next line.

If this causes networks to vertically overlap, the overlapping of the logic can lead to undesired automatic links in Unity Pro.

Open the **Conversion Settings** tab via **Tools → Options** in Unity Pro to enable/disable the **Separate LD Networks** check box before converting.

- When this checkbox is enabled, recognized networks are moved vertically, which prevents overlapping.
- When this checkbox is not enabled, recognized networks are not vertically moved. The original vertical arrangement of the graphics is maintained, but error messages may occur due to overlapping.

LD Column Break

Inserting additional columns may cause a network to exceed its maximum section width and it is then wrapped into the next line.

Open the **Conversion Settings** tab via **Tools → Options** in Unity Pro to edit the **LD Column Break** option before converting.

The number entered here determines the column after which a network is wrapped to the next column.

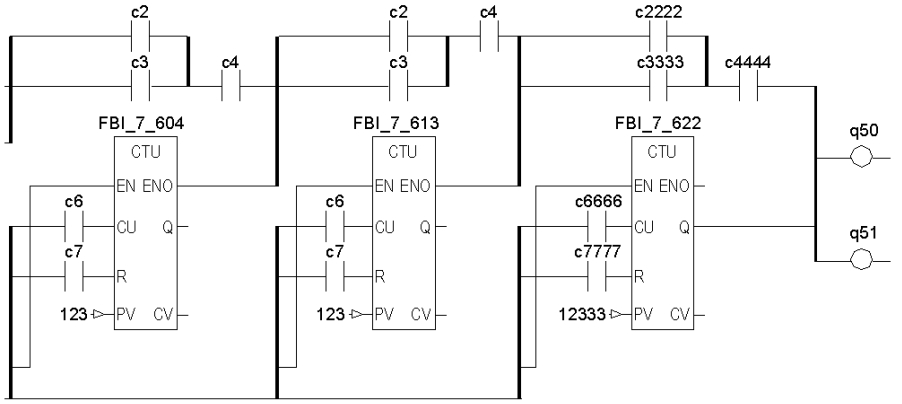
Wrapping networks that are too wide

Since the width of the networks is expanded during the conversion the maximum section width may be exceeded.

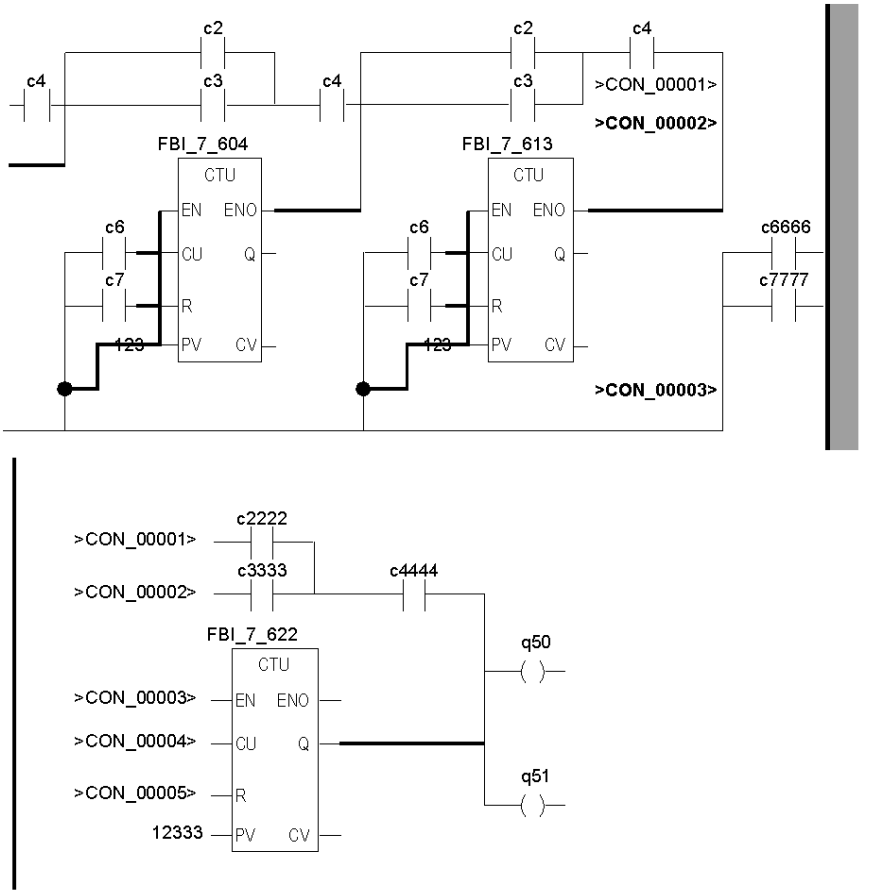
To show the network that is now too wide, the part of the network that reaches beyond the far right edge of the section will be shown in a new row.

The connections are shown as connectors.

Example of an LD Network in Concept.



The wrapped LD Network after the conversion to Unity.



Objects to recognize transitions

The different ways of handling ladder diagram LD objects in Concept (calling an FB) and in Unity Pro (system call) makes the use of State RAM variables (0x/1x register) necessary.

Because of the requirement that several write accesses to the 0x/1x register are possible during a cyclical sweep, there can be differing Online behavior between Concept and Unity Pro.

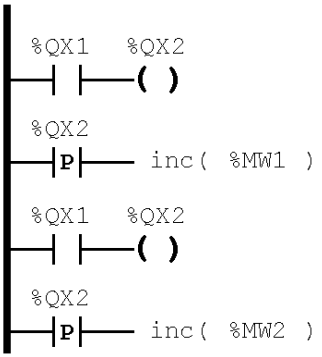
The objects affected are:

- Contact to recognize positive transitions
- Contact to recognize negative transitions

In **Concept** the "Old Value" to recognize a transition will only be updated once per cycle.

In **Unity Pro** the "Old Value" will be updated during every write access.

Example:



Concept: Switch %QX1 from 0 -> 1 and the value of %MW1 **and** %MW2 increase.

Unity Pro: Switch %QX1 from 0 -> 1 and **only** the value of %MW1 increases.

NOTE: Use objects to recognize transitions with a certain variable only once per cycle.

Also see *Located Variable, page 30* and Unity Pro Reference Manual, Use of set and reset coils leads to edge loss *(see Unity Pro, Program Languages and Structure, Reference Manual)*.

Macros

Macros (name begins with @) will be rejected by the converter because macros cannot be implemented in Unity. However, if you try to import an application with macros, the macros will be replaced with Dummy DFBs (indicated with the '~' in the application name).

Error messages regarding these Dummy DFBs will appear during the analysis of the project. To correct these errors, simply remove all DFBs that were created to replace macros.

Programming Language ST/IL

General

For some programming languages there are restrictions to observe when converting a project from Concept to Unity Pro.

Generic EFBs

Only call generic EFBs instances once.

Using Concept 2.2, assign the outputs directly after the EFB call of a variable.

Syntax with Concept 2.5

Only use the new syntax for Concept 2.5 (from Unity V2.0 onwards it is automatically converted).

Syntax with Concept 2.5:

```
GenEFB(in1:=x1, in2:=x2, out1=>x3, out2=>X4;
```

in1, in2, out1 and out2 are type ANY.

Generic EFBs in Concept

List of generic EFBs in Concept:

- COMM library
  - XXMIT
- CONT_CTL library
  - DEADTIME
- EXTENDED library
  - HYST
  - INDLIM
  - LIMD
  - SAH
- LIB984 library
  - FIFO
  - LIFO
  - R2T
  - SRCH
  - T2T
  - GET_3X
  - GET_4X
  - PUT_4X

Declaring EFBs

The declaration of EFBs in Unity Pro is found in the variables editor and no longer in the ST/IL sections as with Concept.

EFBs declared this way are no longer limited to only one section.

Programming Language LL984

General

For some programming languages there are restrictions to observe when converting a project from Concept to Unity Pro.

LL984 is now supported by Unity Pro

NOTE: With Unity Pro 6.0 the LL984 language is supported for Quantum PLCs (but not for Quantum Safety PLCs).
With Unity Pro 6.1 or later the LL984 language is supported for Modicon M340 PLCs with Modicon M340 firmware 2.4 or later.
With Unity Pro 11.0 or later the LL984 language is supported for Modicon M580 PLCs on BME•584040, BME•585040 and BME•586040 CPUs, firmware version greater and equal to 2.1.

Programming Language FBD

General

For some programming languages there are restrictions to observe when converting a project from Concept to Unity Pro.

Macros

When converting a Concept project to Unity Pro, sections created using macros are also converted.

These sections can also be manually copied and modified.

Chapter 3
Language Differences

Overview

This chapter contains information about language differences.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Functions Not Present in Unity	55
EFB Replaced by Function	56
FFBs Not Available For All Platforms	57
`INOUT` Parameters	62
Parameter Type Changed	63
`ANY_ARRAY_WORD` Parameters	64
Unique Naming required	65
Incomplete LD Generation	66
LD Execution Order Changed	67
Edge Detection Difference	71
Constants	72
Indices in ST and IL	73
Calculate with `TIME` and `REAL`	74
`WORD` Assignments to `BOOL` Arrays	75
Topological Address Overlapping	76
Substitute %QD by %MF	77
Structure Alignment Changed	78
Undefined Output on Disabled EFs	79
Variables at Empty Pins	81
The set action remains active, even when the associated step becomes inactive	82
SFC Section Retains its State When Performing an Online Modification	83
`SFCCNTRL` Function Block in Unity Behaves Different to Concept	84
Weekday Numbering	85

Topic	Page
System Timer	86
Initial Values	87
Macros	89

Functions Not Present in Unity

DFB Wrapper

Functions from Concept that are not present in Unity get a DFB wrapper if they are called in ST sections (e.g., `WORD_AS_UDINT`). For example:

```
WAUD(* UDINT *) := WORD_AS_UDINT (LOW := WAUL, (* WORD *) HIGH := WAUH(*
WORD *));
```

. . . looks like this after conversion:

```
WAUD(* UDINT *) := FBI_ST1_75_33 (LOW := WAUL, (* WORD *)HIGH := WAUH(*
WORD *));
```

Manual Correction

`FBI_ST1_75_33` is the instance name of the provided DFB wrapper. However, the call is still invalid for the analyzer because the converter cannot yet do multi-object syntax corrections in ST. (Will be present in V2.0).

You must correct this manually to:

```
FBI_ST1_75_33 (LOW := WAUL, (* WORD *) HIGH := WAUH(* WORD *), OUT =>
WAUD);
```

EFB Replaced by Function

DFB Wrapping

Some standard Concept EFBs are implemented in Unity as functions.

In such cases, a wrapping DFB is provided so that the original interface of the Concept EFB remains valid.

FFBs Not Available For All Platforms

Overview

The FFBs (functions/function blocks) listed below can only be used on Quantum platforms (except `SFC_RESTORE`, see following table).

A subset of these FFBs can be used on M580 platform also.

If Modicon M340 is the target platform, these FFBs appear marked in red and indicated as "type error".

FFBs Not Available

Communication library

Family	FFB	Compatible platforms
Extended	CREAD_REG	Quantum
	CWRITE_REG	
	MBP_MSTR	Quantum, M580
	READ_REG	Quantum
	WRITE_REG	
	MODBUSP_ADDR	
	SYMAX_IP_ADDR	
	TCP_IP_ADDR	
	XXMIT	

I/O Management library

Family	FFB	Compatible platforms
Analog I/O Configuration	I_FILTER	Quantum, M580
	I_SET	
	O_FILTER	
	O_SET	
Analog I/O Scaling	I_NORM	Quantum, M580
	I_NORM_WARN	
	I_PHYS	
	I_PHYS_WARN	
	I_RAW	
	I_RAWSIM	
	I_SCALE	
	I_SCALE_WARN	
	O_NORM	
	O_NORM_WARN	
	O_PHYS	
	O_PHYS_WARN	
	O_RAW	
	O_SCALE	
	O_SCALE_WARN	
Immediate I/O	IMIO_IN	Quantum
	IMIO_OUT	

Family	FFB	Compatible platforms
Quantum I/O Configuration	ACI030	Quantum, M580
	ACI040	
	ACO020	
	ACO130	
	AII330	
	AII33010	
	AIO330	
	AMM090	
	ARI030	
	ATI030	
	AVI030	
	AVO020	
	DROP	
	ERT_854_10	
	NOGSTATUS	Quantum
	QUANTUM	
	XBE	
	XDROP	Quantum, M580

Motion library

Family	FFB	Compatible platforms
MMF Start	CFG_CP_F	Quantum
	CFG_CP_V	
	CFG_CS	
	CFG_FS	
	CFG_IA	
	CFG_RA	
	CFG_SA	
	DRV_DNLD	
	DRV_UPLD	
	IDN_CHK	
	IDN_XFER	
	MMF_BITS	
	MMF_ESUB	
	MMF_INDX	
	MMF_JOG	
	MMF_MOVE	
	MMF_RST	
	MMF_SUB	
	MMF_USUB	

Obsolete Lib library

Family	FFB	Compatible platforms
Extensions/Compatibility	GET_3X	Quantum, M580
	GET_4X	
	PUT_4X	
	IEC_BMDI	

System library

Family	FFB	Compatible platforms
SFC Management	SFC_RESTORE	Quantum, Premium, M580
Hot StandBy	HSBY_RD	Quantum
	HSBY_ST	
	HSBY_WR	
	REV_XFER	Quantum

INOUT Parameters

Manual Correction

INOUT parameter syntax in ST (and IL) must be corrected manually. Examples are shown:

```
Ascii_FIFO_OUT (Pile := AscFifo_Mess); .....

AscFifo_Out := Ascii_FIFO_OUT.DataOut;
```

. . . is manually corrected to:

```
Ascii_FIFO_OUT (Pile := AscFifo_Mess, DataOut => AscFifo_Out);
```

Output Parameters

`INOUT` parameters in ST sections that were output parameters in Concept (e.g., `DataOut` of FIFO) must be moved manually in ST and IL to the parameters inside parentheses associated with the call.

If `INOUT` parameters that were outputs only in Concept are connected only to a link at the output side, they must get a manually declared variable at the input side as well. The link must be deleted if it is not connected to another `IN/OUT` variable. Targets of the deleted link must be assigned to the manually declared variable.

This is done automatically in V2.0.

Change of Variable Type

The converter changes the type of direct variables at `INOUT` parameters of communication blocks to `ARRAY[0..0] OF WORD`.

This must be corrected manually to correspond to the size of the array.

Parameter Type Changed

Change

The parameter type has been changed from type `WORD` to an array of located words.

Explanation

Unity Comm EFBs no longer accept a single `WORD` address for the communication field because more than one `WORD` is written. So the converter introduces an artificial array (shown in the conversion report) that can be reached from the project tree through the appropriate hyperlink:

```
"For var WORD1 type ARRAY[0..0] OF WORD generated"
```

The array has a single word size because the converter can not determine its size. The user, therefore, needs to manually configure the correct array size.

ANY_ARRAY_WORD Parameters

Error Message

For EF/EFB pins that have the type WORD in Concept and have been changed to ANY_ARRAY_WORD in Unity, "Cannot import variables" will be the reported type. Such pins usually have a single register address as a formal parameter in Concept, but it is actually used to point to an array of words for which the size has not been explicitly declared.

Change of Parameter Type

In Unity, an array of words has to be declared for this purpose. This is why the converter changes the type to ARRAY[0..0] OF WORD.

However, the converter cannot determine the required size because a size declaration is absent in the Concept application. Therefore, the converter defines one data element, [0..0], as a replacement for the original variable.

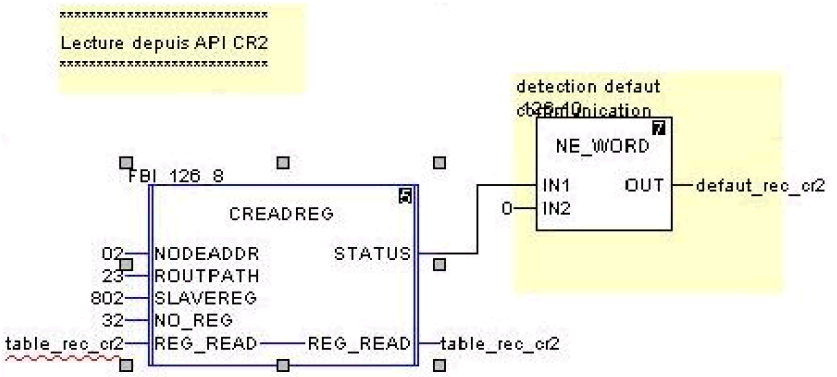
It is up to the user to replace this default range of one element with the number of elements required by the application.

Redefine Back to a One-Dimensional WORD Array

In case the application defined data structures that are mapped to registers that describe the data to be worked with, significant work to redefine this back to a one-dimensional WORD array is required. However, this is necessary for Unity V1.0, for example:

```
{Echanges_CR2 : [MAST]} : (r: 42, c: 7) E1092 data types do not match
('CREADREG.REG_READ:ANY_ARRAY_WORD'<->'table_rec_cr2:peer_Table')
```

Example:



The Unity converter V2.0 will change these EFB parameter types to ANY, avoiding this problem.

Unique Naming required

Unique name

In Concept applications, section names can have the same name as a DDT. That is not the case in Unity.

The converter checks section names to see if they are redundant of DDT names. If so, the converter appends "_Sect" to the section name.

Incomplete LD Generation

LD Generation Not Done Completely

In some cases, LD generation cannot be completed. This can happen when the algorithm allows an object that requires the same position as an existing object. In these cases, the pre-existing object is overwritten.

Messages are issued to make you aware of this:

```
{SAFETY_INTERLOCKS_PLC3 : [MAST]} :
(r: 8, c: 3) E1189 converter error: 'Overwrite happened when generating
LD network - see report'
{SAFETY_INTERLOCKS_PLC3 : [MAST]} : (r: 8, c: 3) E1002 syntax error
```

Details in Conversion Report

In the conversion report, which may be opened after being imported through the hyperlink in the project tree, some additional detail about the message is given:

```
09:29:05.953 > Error: LD Object PTFDTP1_ENABLED with type coil
overwritten
```

The user should compare the conversion result to a printout of the original section and correct the converted section accordingly.

Label or Link Views

If in LD networks, some links seem to have disappeared, it's because for a better reading they have been replaced by labels, to see the links, make a right click and select the **Show as link** option.

LD Execution Order Changed

Different Execution Orders

NOTE: Unity's LD execution order can differ from Concept's. In Unity, one LD network can be completed before the next is started.

The converter follows the Concept execution order in graphical positioning, making the original order visible to the user. However, since Unity calculates the order anew (without the possibility of forcing it from the converter), there can be execution order discrepancies.

Generate ConvError Hints

Open the **Conversion Settings** tab via **Tools → Options** in Unity Pro to enable/disable the **Generate ConvError Hints** check box before converting.

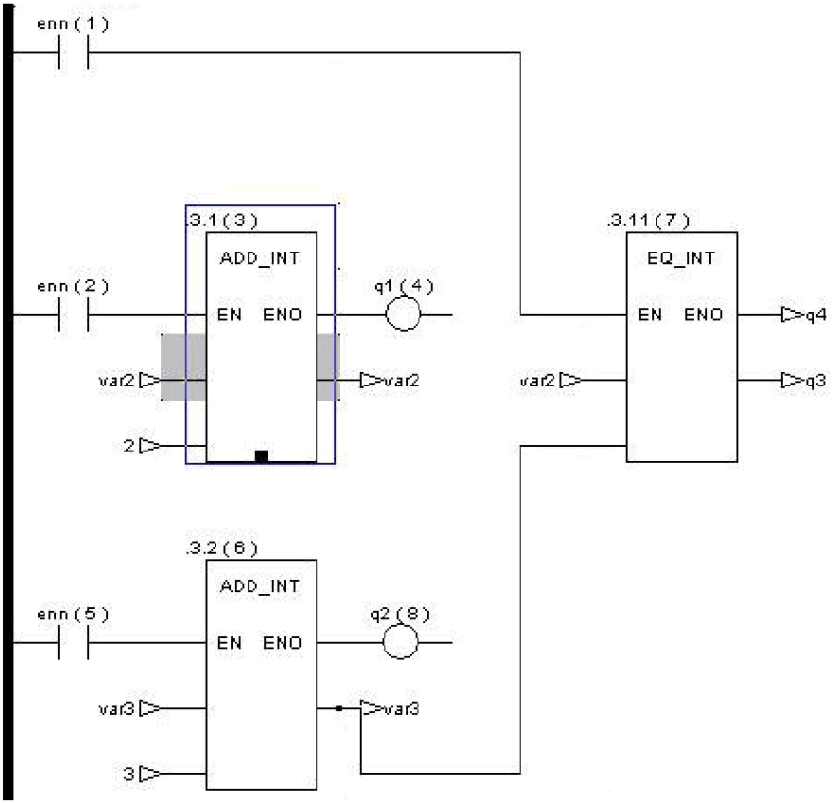
- When this checkbox is enabled, **ConvError** objects are generated in the LD programs during the conversion to draw attention to special issues.
- When this checkbox is not enabled, **no ConvError** objects will be generated during the conversion.

Concept

When analyzing in Concept, the execution order is calculated. The result is shown in parentheses after the instance names in this image.

The selected block is executed in the middle of the other network, even though it has no direct connection to it. Concept calculates the execution order from the block position.

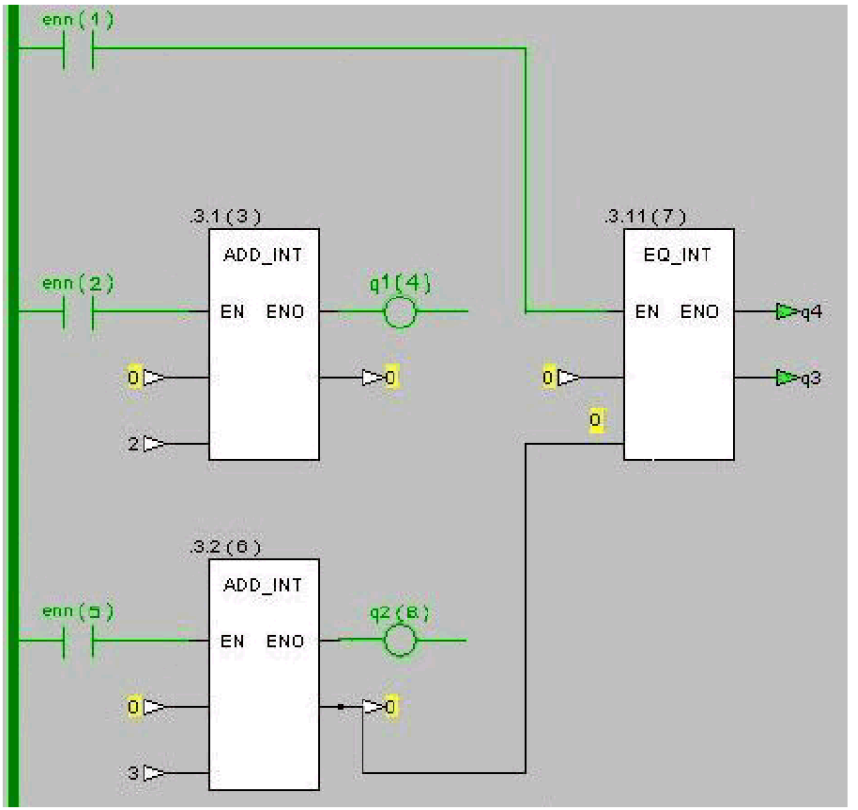
This is the original section as it appears in Concept:



The used variables are initialized in a way that the result of the comparator `EQ_INT` becomes "true" after execution of the first cycle in Concept:

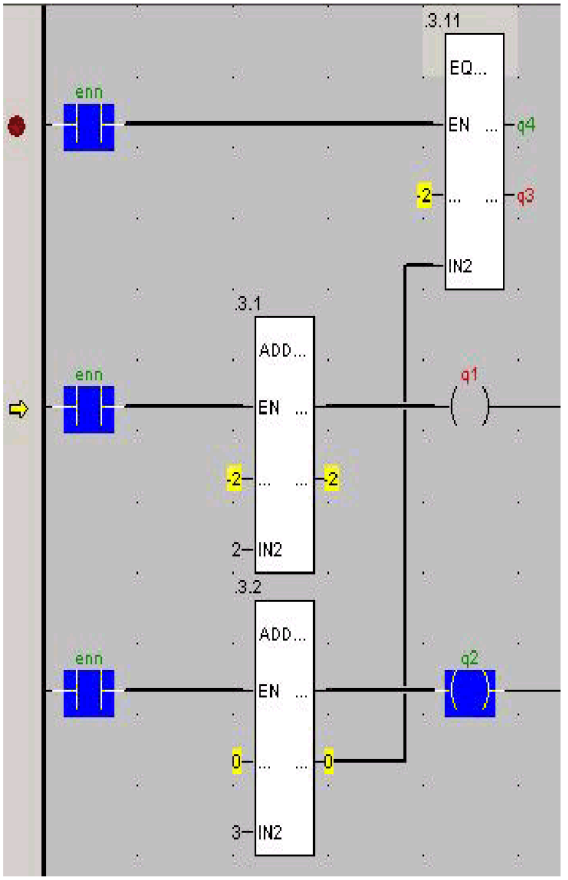
var2	INT		-2
var3	INT		-3

Testing execution in single cycle mode in Concept shows the expected result. The comparator result becomes "true" after the first cycle:



Unity

The converted network reflects the Concept execution order in the graphical position of the blocks:



The image also shows the execution status stopped at a breakpoint in the first cycle. The comparator `EQ_INT` is already executed and will not deliver a "true" result because the first `ADD_INT` integrator block is executed **after** it.

Solution

Replace the connection via a variable by a link to achieve the same result as in Concept.

Edge Detection Difference

General

The edge (positive or negative transition) detection managing is different between **Concept** and **Unity pro**.

Explanation

Because multiple write accesses to the 0x/1x register are possible during a cyclical sweep, there can be differing Online behavior between Concept and Unity Pro.

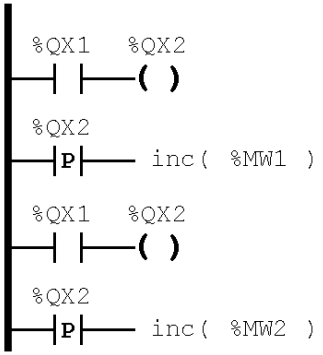
The objects affected are:

- Contact to recognize positive transitions
- Contact to recognize negative transitions

In **Concept** the "Previous Value" used to recognize a transition is only updated once per cycle.

In **Unity Pro** the "Previous Value" is updated at each write access.

Example:



Concept: Switch `%QX1` from 0 -> 1 and the value of `%MW1` **and** `%MW2` increase.

Unity Pro: Switch `%QX1` from 0 -> 1 and **only** the value of `%MW1` increases.

NOTE: Use objects to recognize transitions with a certain variable only once per cycle.

Also see *Located Variable, page 30* and Unity Pro Reference Manual, Use of set and reset coils leads to edge loss *(see Unity Pro, Program Languages and Structure, Reference Manual)*.

Constants

Losing the Read-Only Behavior

Constants are not accepted as private DFB variables. Therefore, they are converted to initialized variables in DFBs, in this way losing the read-only behavior.

Indices in ST and IL

High Resolution

In addition to `INT` now `DINT` will be allowed as array index type in all areas of Unity Pro, but with limited value ranges.

For `DINT` the index may only contain `INT` values (`-32768` ... `32767`).

Calculate with TIME and REAL

Manual Correction

When TIME and REAL variables are multiplied in ST, REAL_TO_DINT must be inserted into the REAL variable manually.

WORD Assignments to BOOL Arrays

Manual Correction

Assignments of HEX WORDS to complete Bool arrays sent to Word registers are possible in Concept, but not in Unity. A manual correction must be done, for example:

```
('AR2_BOOL[0]:BOOL'<->'16#0100:DINT')

('AR2_BYTE[0]:BYTE'<->'16#55AA:DINT')

('AR2_BYTE[0]:BYTE'<->'16#AA55:DINT')
```

Solution

The ST code must be changed to single-component assignments.

The hex word must be split into single bits:

```
AR2_BOOL[17] := true;
```

Topological Address Overlapping

Same Topological Address

In Unity, you are warned (during application analysis) if the same topological address is assigned to multiple variables.

Substitute %QD by %MF

Introduction

Variables that are directly addressed in Concept with %QD can be initialized floating point constants or dual word constants.

When mainly floating point constants appear, the **Substitute %QD by %MF** checkbox should be enabled.

Conversion Settings

Open the **Conversion Settings** tab via **Tools → Options** in Unity Pro to enable/disable the **Substitute %QD by %MF** check box before converting.

- When this checkbox is enabled, %QD variables are converted to %MF variables.
- When this checkbox is not enabled, %QD variables are converted to MW variables.

Structure Alignment Changed

DPM_Time Structure

Unity uses a 2-byte alignment for structures in contrast to Concept (1-Byte) to speed up the access to structure components. This affects system structures mapped to StateRam, because the same structures in Unity can be bigger including some byte gaps.

The concerned structure is `DPM_Time`, which has been redefined for Unity to re-map to the correct hardware addresses.

Concept's DPM_Time definition:

```
sync: BOOL
ms:   WORD
...
```

Unity's DPM_Time definition:

```
sync:   BOOL
ms_lsb: BYTE
ms_msb: BYTE
...
```

Manual Correction

If an application that includes the `DPM_time` structure is converted, the analyze/build process will fail for the redefined structure components (in the above example, `ms_lsb`, `ms_msb`).

The user has to manually change the usage of these structure components in the application accordingly.

Undefined Output on Disabled EFs

Outputs of EFs Not Kept

In case the EN switches from `TRUE` to `FALSE`, the outputs of EFs from the previous cycle are not kept in Unity. This reduces the memory consumption in the PLC. This is different from EFBs, which keep their value from the previous cycle. Concept uses static links to latch the value from the previous cycle.

Execution Behavior Differs Significantly

If a Concept application relies on the outputs of EFs to keep their old values, the execution behavior in UNITY will differ significantly.

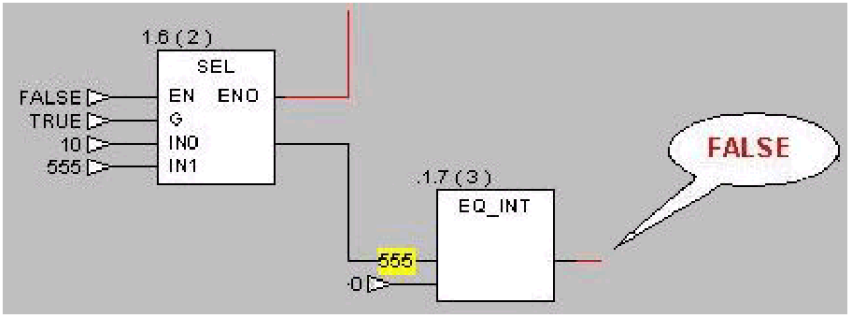
Manual Correction

The application has to be changed manually.

Links from outputs, which are assumed to keep their value, need to be replaced by variables. If the `EN` of an EF is set to false, the EF is not executed and a connected variable is not touched.

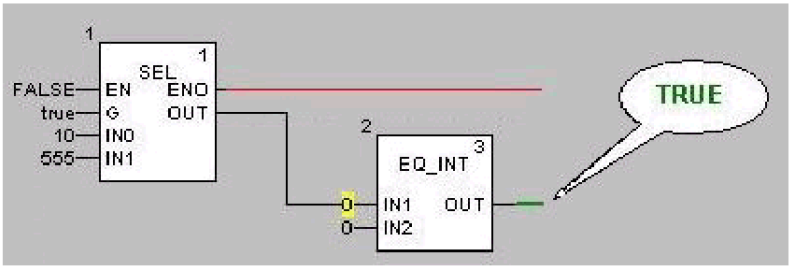
Concept

The output of the disabled `SEL` EF is kept and used as input for the `EQ_INT` function block:



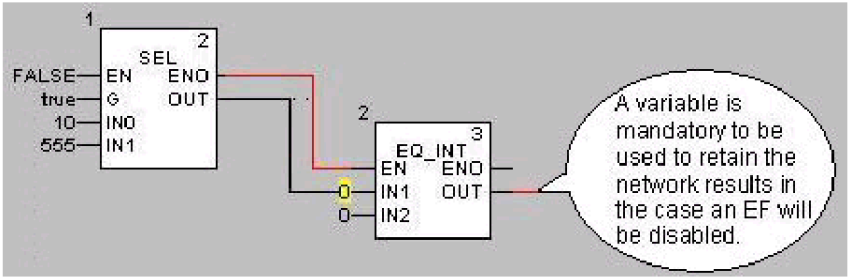
Unity

The output of the disabled SEL EF gets an undefined value, in this case 0. Therefore the output of EQ_INT function block has become true:



Solution

If the EN of the SEL is set to false, the ENO of the EQ_INT is also set to false, but the connected output variable keeps its value from the previous cycle:



NOTE: The use of a variable is mandatory to retain network results in case an EF becomes disabled.

Variables at Empty Pins

Introduction

In Unity Pro it is necessary to fill provided inputs and outputs for derived data types or I/O parameters (this is not necessary in Concept).

If these types are not generic, the converter will fill these initially empty inputs and outputs with variables created by the converter.

Conversion Settings

Open the **Conversion Settings** tab via **Tools → Options** in Unity Pro to enable/disable the **Variables at empty pins** check box before converting.

- When this checkbox is enabled, empty link points will be filled with variables created by the converter.
- When this checkbox is not enabled, empty link points will not be filled with variables created by the converter.

The set action remains active, even when the associated step becomes inactive

In Concept

The action could be reset in other sections.

In Unity Pro

The action only becomes inactive, when it is reset in another step of the current SFC section, using the qualifier R.

NOTE: To get indentical behaviour, the solution is to use "Section" as Action instead of "Variable". In the section you can program SET(bit_xxx) ; and in the section outside the SFC you can program a RESET (bit_xxx) and it will work.

SFC Section Retains its State When Performing an Online Modification

Online Modifications Without Resetting

In Unity it is possible to do online modifications of an SFC chart without resetting it. The SFC chart retains its state and will continue the execution.

NOTE: In Concept, the online modification of an SFC chart usually results in the resetting of the chart.

SFCCNTRL Function Block in Unity Behaves Different to Concept

RESETSFC vs. INIT

In Concept the RESETSFC input of SFCCNTRL resets all action variables of the related SFC section.

In Unity the INIT input of SFCCNTRL (that has a similar function as RESETSFC input in Concept) only resets the action variables that have been set by the SFC step. Action variables, for example, set by user logic or the Animation Table will not be reset.

Weekday Numbering

Different Numbering

In Unity the numbering of weekdays is different than Concept:

Number	Unity	Concept
1	Monday	Sunday
7	Sunday	Saturday

SET_TOD / GET_TOD

Function blocks: `SET_TOD` and `GET_TOD` will be converted to Unity as DFBs, which work in both directions.

Because `SET_TOD` expects a "Concept" numbered weekday and translates it as a Unity coded value. Also the `GET_TOD` reads Unity value and returns to User the Concept value.

System Word %SW49

NOTE: We do not recommend that you mix `GET_TOD` and `SET_TOD` programming with the use of system words (e.g. `%SW49`) in the same application.

System Timer

Concept

Concept's system timer was located on a user-defined register word (16-bit) and incremented at 10 ms.

Unity

Unity provides an incremental timer with 100 ms updating (`%SW18`).

A 10 ms timer can be logically created using the `FREERUN` function (sec timer).

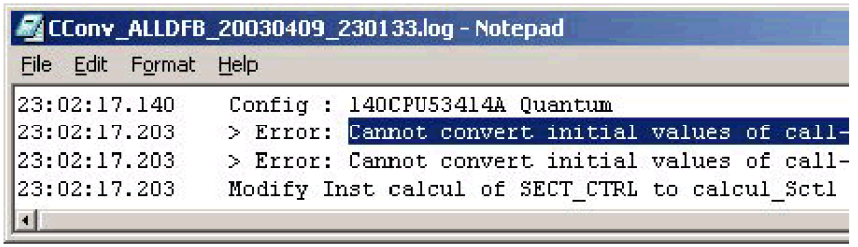
Initial Values

Definition of Initial Values

Concept allows the initial values on DFB pins of a structured array to be defined.

Unity forbids this option for pins of array type. This option is reserved for output pins of structure type.

The converter reflects this with the following error message in the conversion log:




```
Error: Cannot convert initial values of call-by-reference data (pin
Add_PV.in1)
```

Pins to be Connected

At the same time, Unity enforces pins of array type and input pins of structured type to be connected, which in this case leads to analysis errors:

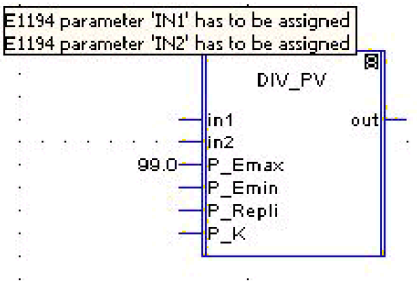
```
{ALL:[MAST]}:   (r:26, c:68) E1194 oarameter ´IN2´has to be assigned
{ALL:[MAST]}:   (r:26, c:68) E1194 oarameter ´IN1´has to be assigned
```

Solution

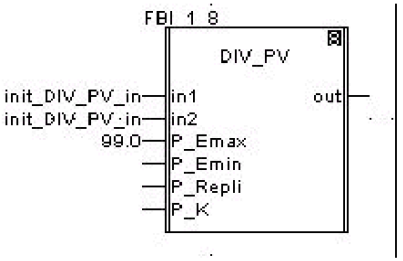
To solve this problem, create a variable of the pin's type and initialize it with the original values.

Connect this constant to the appropriate pin of each DFB instance.

Example



Solution: Add initialized variable.



Macros

Macros Replaced by Dummy DFBs

Macros (name starting with @) are refused by the converter because Unity does not implement macros. However, if you try to import an application containing macros, they will be replaced by dummy DFBs (as indicated by the '~' character in the application name).

While analyzing the project, you will get error messages regarding these dummy DFBs. To correct these errors, simply remove all of the DFBs that were created as a replacements for macros.

`AXx`, `EPARx` Parameters

`AXx` and `EPARx` parameters in Concept's extensible motion blocks are automatically invoked with the newly required array instead of with Unity's formerly present extensible pins. Constants present at the Concept pins are also placed as initialization values to such arrays. However, variables and links must be attached manually with move blocks to these arrays.

Chapter 4
Possible application behavior change

Overview

This chapter contains information about possible application behavior change, when migrating from Concept to Unity Pro.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
General	92
Concept Behavior	93
IEC Demands	94
Unity Behavior	97
Consequences	99

General

Concept

In Concept and Unity Function Block interfaces are implemented with data structures (instance areas) collecting parameters, according to the standard IEC61131 which both systems refer to.

Function Block invocations refer to those data structures. However, Concept does not include output parameters into those instance areas. All parameters of DFB/EFBs (Elementary Function Blocks) are generally handled by reference, therefore output parameters are directly written to by the Function Block code by Concept. Unity DFB/EFBs buffer output parameters in the instance area, as the standard IEC61131 prescribes.

The Concept behavior was used to enable, for example, easy manual mode implementation of closed-loop-control function blocks.

If the output is written to, only once in a cycle, the behavior is the same in both systems.If output values are not written in all invocation cases, but are assigned by several Function Block instances, different behavior between both systems can result.

If the concerned variable is written to by some other control part prior to a Function Block having the same output parameter, invoked in a case where the Function Block does not write to the output, nothing changes in Concept, but in Unity the variable buffer value in the instance area resulting from a previous invocation is assigned to the output parameter.

To detect such cases, multiple assignments to elementary variables or derived data type components from Function Blocks are detected by the Concept converter, if the appropriate option is checked:

- Detected for Elementary and Derived Function Blocks.
- Works in DFB and Program Sections.
- Reports during conversion in the build tab of the output window with textual identification of the concerned locations.
- The same textual report appears in the Conversion report.
- Reports in FBD and LD sections with ‘ConvError’ blocks placed above the concerned Function Blocks.
- On Analysis messages appear in the Analyze/Build tab of the output window, which can be opened by double clicking and opening the concerned section and directly showing the concerned Function Block.

Using this report, the user can adapt this code to ensure common operation, e.g. by changing the DFB outputs to InOut parameters, which offer direct writing also in Unity.

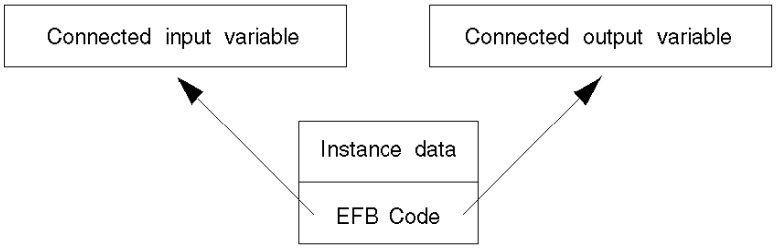
NOTE: If the application uses multi-assignment on EFB outputs, you should carefully read the following chapter to verify that the converted application works in the intended way (EFBs cannot be changed by the user, only new ones can be introduced).

Concept Behavior

Parameters are Handled by Reference

In Concept all function block parameters are handled by reference, means the blocks receives a pointer to the data of every function block pin and works directly on the connected variable.

Connected variables:



Function Block Code

Therefore in Concept it is up to the function block code to decide whether:

- to behave IEC compliant or
- to write input data or
- to read output data or
- not to write output data.

IEC Demands

Function Block

For the purposes of programmable controller programming languages, a function block is a program organization unit which, when executed, yields one or more values.

Multiple, named instances (copies) of a function block can be created.

Each instance shall have an associated identifier (the instance name), and a data structure containing its output and internal variables, and, depending on the implementation, values of or references to its input variables.

All the values of the output variables and the necessary internal variables of this data structure shall persist from one execution of the function block to the next.

Therefore, invocation of a function block with the same arguments (input variables) need not always yield the same output values.

Assignment of a Value

Assignment of a value to an output variable of a function block is not allowed except from within the function block.

The assignment of a value to the input of a function block is permitted only as part of the invocation of the function block.

Unassigned or unconnected inputs of a function block shall keep their initialized values or the values from the latest previous invocation, if any.

Allowable usage of function block inputs and outputs are summarized in table below, using the function block FF75 of type SR.

The examples are shown in the ST language.

Usage	Inside function block	Outside function block
Input read	`IF IN1 THEN ...`	**Not allowed** [1, 2]
Input assignment	**Not allowed** [1]	`FB_INST(IN1:=A,IN2:=B);`
Output read	`OUT := OUT AND NOT IN2;`	`C := FB_INST.OUT;`
Output assignment	`OUT := 1;`	**Not allowed** [1]
In-out read	`IF INOUT THEN ...`	`IF FB1.INOUT THEN...`
In-out assignment	`INOUT := OUT OR IN1;` [3]	`FB_INST(INOUT:=D);`

1 Those usages listed as "not allowed" in this table could lead to implementation-dependent, unpredictable side effects.

2 Reading and writing of input, output and internal variables of a function block may be performed by the "communication function", "operator interface function", or the "programming, testing, and monitoring functions" defined in IEC 61131-1.

3 Modification within the function block of a variable declared in a VAR_IN_OUT block is permitted.

EN and ENO in Function Blocks

For function blocks also an additional Boolean EN (Enable) input or ENO (Enable Out) output, or both, can be provided by the manufacturer or user according to the declarations.

When these variables are used, the execution of the operations defined by the function block shall be controlled according to the following rules:

1. If the value of EN is FALSE (0) when the function block instance is invoked, the assignments of actual values to the function block inputs may or may not be made in an implementation-dependent fashion, the operations defined by the function block body shall not be executed and the value of ENO shall be reset to FALSE (0) by the programmable controller system.
2. Otherwise, the value of ENO shall be set to TRUE (1) by the programmable controller system, the assignments of actual values to the function block inputs shall be made and the operations defined by the function block body shall be executed. These operations can include the assignment of a Boolean value to ENO.
3. If the ENO output is evaluated to FALSE (0), the values of the function block outputs (VAR_OUTPUT) keep their states from the previous invocation.

Not Connected EN Inputs

When EN inputs are left open the concerned blocks are not executed in Concept whereas they would be executed in Unity Pro.

To eliminate this difference the Concept Converter applies a constant boolean value of FALSE to not connected EN inputs. In this way achieving the same behavior as in Concept.

In-Out Variables

In-out variables are a special kind of variable used with program organization units (POUs), i.e., functions, function blocks and programs.

They do not represent any data directly but reference other data of the appropriate type. They are declared by use of the VAR_IN_OUT keyword. In-out variables may be read or written to.

Inside a POU, in-out variables allow access to the original instance of a variable instead of a local copy of the value contained in the variable.

Function Block Invocation

A function block invocation establishes values for the function block's input variables and causes execution of the program code corresponding to the function block body.

These values may be established graphically by connecting variables or the outputs of other functions or function blocks to the corresponding inputs, or textually by listing the value assignments to input variables.

If no value is established for a variable in the function block invocation, a default value is used.

Depending on the implementation, input variables may consist of the actual variable values, addresses at which to locate the actual variable values, or a combination of the two.

These values are always passed to the executing code in the data structure associated with the function block instance.

The results of function block execution are also returned in this data structure.

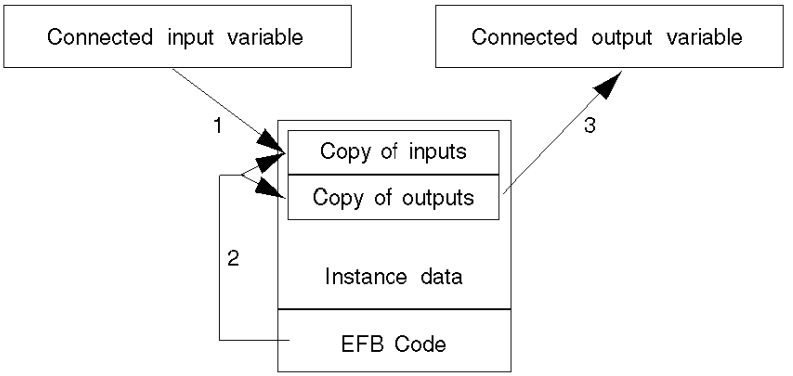
Hence, if the function block invocation is implemented as a procedure call, only a single argument - the address of the instance data structure - need be passed to the procedure for execution.

Unity Behavior

Changed Parameter Handling

To fulfill the IEC demands the normal EDT (Elementary Data Types) parameter handling was changed from Concept to Unity.

The following figure describes the actual implementation in Unity.



The EFBs no longer get pointers to their connected pin variables.

They always get the data by value.

In every scan the application code updates the copy of the input data in the instance data, before the function block is called (1).

The copy of the pin data is located in the instance data of the block and the function block code always works on the instance data (2).

After the function block code execution the application code copies the updated function block output data from the instance data to the connected output variables (3).

This is valid for all EDTs. Derived data types and more complex data types are treated still by reference in some cases.

Addressing Modes

The addressing mode of a Function Block element is directly linked to the type of the element.

The currents known addressing modes are:

- by value (VAL)
- by address (L-ADR)
- by address + Number of elements (L-ADR-LG)

Table with four columns and legend

-	EDT (Except STRING)	STRING	DDT Array	DDT Struct	ANY_ ARRAY	ANY...
Input parameter	VAL	L-ADR-LG	L-ADR-LG	L-ADR	L-ADR-LG	L-ADR-LG
Input_Output parameter	L-ADR [1]	L-ADR-LG	L-ADR-LG	L-ADR	L-ADR-LG	L-ADR-LG
Output parameter	VAL	VAL	L-ADR-LG	VAL	L-ADR-LG	L-ADR-LG
Public Variable	VAL	VAL	-	VAL	-	-
Private Variable	VAL	VAL	-	VAL	-	-
1 Except for BOOL type, the addressing mode is VAL.						

Function Block Invocation

The following rules must be taken into account while invoking a Function Block instance:
- All input_output parameters have to be filled
- All input parameters using the L-ADR or L-ADR-LG addressing modes have to filled
- All output parameters using the L-ADR or L-ADR-LG addressing modes have to filled

All other kind of parameters could be omitted while Function Block Instance invocation.

For input parameters, the following rules are applied (in the given order):
- The values of the previous invocation are used.
- If no previous invocation, the initial values are used.

EFB and DFB output result when not enabled in a program section

When a DFB instance or an EFB instance is not enabled it means that the code is not executed. But if an output is linked to a variable, the output is applied even if the DFB or EFB instance code is not executed, then the variable is set at the value of the DFB or EFB instance output.

EFB and DFB output linked to a variable and to another EFB or DFB

If a DFB instance or EFB instance output, is linked to a variable and at the same time to another FFB, the editor display could show a difference between the link and the variable value.

The reason is:
- A variable is displayed with its value at the end of the MAST task.
- If the variable is written at another place after this display, its value has changed and could be different than the value of the link.

Consequences

Potential Problems

⚠ WARNING
UNEXPECTED APPLICATION BEHAVIOR Take care when an application is migrating from Concept to Unity. **Failure to follow these instructions can result in death, serious injury, or equipment damage.**

NOTE:
Because of this architectural change, when an application is migrated from Concept to Unity you have to evaluate the consequences of the migration, and specially in the following cases:

- **Multi assignment of connected output variables:**
  In Concept there are function blocks, mainly in the closed-loop-control area, which do not write their output values to the connected variables in special operating modes (manual mode).
  In these special modes it was possible to write the variables from other locations inside the application.
  This will work in Unity only, if the variables are written after the function block call.
  If they are written before the function block call, the copy process from the instance data to the connected variables will overwrite this value with the old value from the instance data.
- **Controlling output variables by animation table or HMI:**
  If a block doesn't write his outputs in special operating modes (like manual mode, see above), it was possible to modify the connected output variables by animation tables or HMI.
  This will no longer work in Unity, since the copy process from the instance data to the connected variables of the function block will overwrite the modified value with the old value from the instance data.

Changed EFB Layout

To avoid major problems, a lot of function blocks (mainly in the Motion and CLC area) were changed in their layout from Concept to Unity to ensure a correct mode of operation in the intended way for the function blocks.

The concerned pins were changed from type `OUT` to `IN/OUT`.

In nearly all cases the modification meets better the reality, since it is read from the concerned output pins and so they are in fact `IN/OUTs`.

The following tables summarize the EFBs, where at least one pin was changed from `OUT` to `IN/OUT` during migration from Concept to Unity.

Library CONT_CTL:

Family	Function Block	Concerned Pin
Controller	PI_B	OUT
	PIDFF	OUT
Output Processing	MS	OUT
Setpoint Management	SP_SEL	SP

Library Motion:

Family	Function Block	Concerned Pin
MMF Start	CFG_CP_F	MFB, CFG_BLK
	CFG_CP_V	MFB, CFG_BLK
	CFG_CS	MFB, CFG_BLK
	CFG_FS	MFB, CFG_BLK
	CFG_IA	MFB, CFG_BLK
	CFG_RA	MFB, CFG_BLK
	CFG_SA	MFB, CFG_BLK
	DRV_DNLD	MFB
	DRV_UPLD	MFB
	IDN_CHK	MFB
	IDN_XFER	MFB
	MMF_BITS	MFB
	MMF_ESUB	MFB
	MMF_INDX	MFB
	MMF_JOG	MFB
	MMF_MOVE	MFB
	MMF_RST	MFB
	MMF_SUB	MFB
	MMF_USUB	MFB

Library Obsolete Lib:

Family	Function Block	Concerned Pin
CLC_PRO	ALIM	Y
	COMP_PID	Y, YMAN_N, OFF_N, SP_CAS_N
	DERIV	Y
	INTEG	Y
	LAG	Y
	LAG2	Y
	LEAD_LAG	Y
	PD_OR_PI	Y
	PI	Y
	PID	Y
	PID_P	Y
	PIP	Y
	PPI	Y
	VLIM	Y
Extensions/Compatibility	R2T	OFF
	SRCH	INDEX
	T2T	OFF

Concept Converter Behavior

The Concept Converter normally handles the layout change in the following way, when a Concept application is imported into Unity:

- **Case 1: A variable is connected to the output pin in Concept:**
  The Concept Converter keeps the variable at the output side of the IN/OUT pin and adds the variable additionally at the input side of the pin.
- **Case 2: A link is connected to the output pin in Concept:**
  The Concept Converter removes the link, creates a new variable of the needed type and writes this new variable to the start and end position of the removed link. Additionally the variable is added to the input side of the pin.

Further Potential Problems

The following tables contain blocks, where also some consequences of the architectural change from Concept to Unity may arise in case of multi-assignment, because in Concept:

- The blocks do not write their listed output pin in case of errors inside the block.
- The blocks do not write their listed output pin in `COLD` or `WARM INIT` scan.
- The blocks write their listed output pin conditionally depending from internal mode of operation.

Library CONT_CTL:

Family	Function Block	Concerned Pin
Conditioning	DTIME	OUT
	SCALING	OUT
	TOTALIZER	OUT, INFO
Controller	AUTOTUNE	TRI, INFO
	PI_B	OUT_D, DEV
	PIDFF	OUT_D, INFO
	STEP2	DEV
	STEP3	DEV
Output Processing	MS	OUTD, STATUS
	MS_DB	OUTD, STATUS
	SPLRG	OUT1, OUT2
Setpoint Management	RAMP	SP
	RATIO	KACT, SP
	SP_SEL	LSP_MEM

Library I/O Management:

Family	Function Block	Concerned Pin
Analog I/O Configurationj	I_SET	CHANNEL
	O_SET	CHANNEL
Analog I/O Scaling	I_NORM_WARN	WARN
	I_PHYS_WARN	WARN
	I_SCALE_WARN	WARN
Quantum I/O Configurationj	ACI040	CHANNL1..16
	ACO130	CHANNEL1..8
	AII330	CHANNEL1..8, INTERNAL
	AII33010	CHANNEL1..8
	AIO330	CHANNEL1..8
	ARI030	CHANNEL1..8

Library Motion:

Family	Function Block	Concerned Pin
MMF Start	CFG_CP_F	Q, ERROR
	CFG_CP_V	Q, ERROR
	CFG_CS	Q, ERROR
	CFG_FS	Q, ERROR
	CFG_IA	Q, ERROR
	CFG_RA	Q, ERROR
	CFG_SA	Q, ERROR
	DRV_DNLD	Q, ERROR, IDN_CNT
	DRV_UPLD	Q, ERROR, REG_CNT, DATA_B, LK
	IDN_CHK	Q, ERROR, NOT_EQ
	IDN_XFER	Q, ERROR, OUT_RAW, OUTCONV
	MMF_ESUB	Q, ERROR, RET1, RET2, RET§
	MMF_INDX	Q, ERROR
	MMF_JOG	Q, ERROR
	MMF_MOVE	Q, ERROR
	MMF_RST	Q
	MMF_SUB	Q, ERROR, RET1, RET2, RET§
	MMF_USUB	Q, ERROR, RET1, RET2, RET§

Library Obsolete Lib:

Family	Function Block	Concerned Pin
CLC	DELAY	Y
	PI1	ERR
	PID1	ERR
	PIDP1	ERR
	THREE_STEP_CON1	ERR_EFF
	THREEPOINT_CON1	ERR_EFF
	TWOPOINT_CON1	ERR_EFF
CLC_PRO	COMP_PID	STATUS, ERR
	DEADTIME	Y
	FGEN	Y, N
	INTEG	STATUS
	PCON2	ERR_EFF
	PCON3	ERR_EFF
	PD_OR_PI	ERR, STATUS
	PDM	Y_POS, Y_NEG
	PI	ERR, STATUS
	PID	ERR, STATUS
	PID_P	ERR, STATUS
	PIP	ERR, SP2, STATUS
	PPI	ERR, SP2, STATUS
	PWM	Y_POS, Y_NEG
	QPWM	Y_POS, Y_NEG
	SCON3	ERR_FF
	VLIM	STATUS
Extensions/Compatibility	FIFO	EMPTY, FULL
	LIFO	EMPTY, FULL

NOTE: The pins were not changed, because in normal operation mode of the blocks this has no influence.

Chapter 5
The Conversion Process

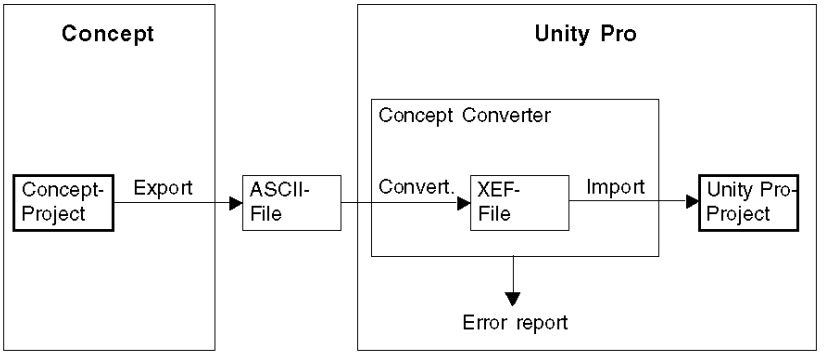
Conversion Process

General

A Concept project is exported from Concept and then converted automatically into a Unity Pro project using the Unity Pro Concept Converter.

Conversion Process

Representation of the conversion process:



Description of the conversion levels:

Level	Description
1	A project is exported from Concept. An ASCII file is created.
2	The Unity Pro Concept Converter is called. The ASCII file is converted into an XEF file.
3	The XEF file is imported into Unity Pro. A Unity Pro project is created.
4	The error report is checked. There must be no errors.
5	The project is now available in Unity Pro and can be generated and then loaded into a PLC or processed in Unity Pro.

Error Report and Analysis

Errors that occur during conversion are logged in an error report and displayed in an output window.

Substitute objects are used in place of objects that cannot be converted. The Unity Pro project can be analyzed using the main menu **Create → Analyze Project**. Subsequently messages are displayed in the output window to find the substitute objects.

The errors displayed in the output window must be corrected manually to ensure the Unity Pro project runs correctly.

Animation tables converting process

In Concept, RDE tables could have different names, the concept converter creates an animation table for each data editor reference file found in the project folder.

For Prowrx32 Data Watch windows, every data Watch window will result in an Animation table.

Chapter 6
Conversion Procedure

Overview

This chapter contains the procedures required to convert a Concept project into a Unity Pro project.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Exporting a Project from Concept	108
Importing a Project into Unity Pro	109
Missing Datatypes at the Beginning of the Import	110
Converting Only Parts of a Concept Application	111
Removing Accidentally Included Concept Macros	112
Initialization Values	113
If the convertedMomentum application contain more than one XMIT block	114

Exporting a Project from Concept

General

A Concept project that should be used in Unity Pro must first be exported from Concept. It is then possible to use the Unity Concept Converter to make the conversion to a Unity Pro project.

Export Project

Perform the following steps to export a project:

Step	Procedure
1	Start the Concept Converter program from the Concept program group.
2	Select **File → Export...,** to open the menu for selecting the export range.
3	Select the export range: • **Project with DFBs:** All project information including the DFBs and data structures (derived data types) used in the project are exported. • **Project without DFBs:** All project information including all data structures (derived data types), but not DFBs and macros, is exported. **Result:** The dialog box for selecting the files to be exported is opened.
4	Select the following file extension: • **Export projects:** Select the extention .prj from the format list box.
5	Select the project and confirm using **OK**. Result: The project is stored in the current directory as an ASCII file (.asc).
6	End the Concept Converter program using **File → Exit**.

Importing a Project into Unity Pro

General

A Concept project that is be used in Unity Pro must first be exported from Concept. It is then possible to use the Unity Concept Converter to make the conversion to a Unity Pro project.

Import Project

Carry out the following steps to convert and import a project:

Step	Procedure
1	Launch Unity Pro.
2	Open the project exported from Concept using **File → Open**. Select the data type CONCEPT PROJECTS (*.ASC). NOTE: Do not use the **project with used DFBs (Re-Connect to Equal)** option when creating the *.ASC file. Unity Pro cannot import the application if this option is used.
3	**Result:** The ASCII file is converted to Unity Pro source file format and imported automatically. Import errors and messages about objects that cannot be converted and have substitute objects in their place, are displayed in an output window.
4	Edit the errors and messages in the output window manually to ensure the Unity Pro project runs correctly.
5	To ensure that a project contains no more errors, select the menu command **Build → Analyse Project** again.

Missing Datatypes at the Beginning of the Import

General

If the dialog at the beginning of the import claims for unknown DDTs, search for local type declarations in this DDT and find out which of these are undefined.

Further, types unused but present in the opened *.asc file are reported as unknown in a dialog during import.

Concept System Data Types

This happens for Concept system data types, which are considered for Concept to be always presented and therefore not included in the export by Concept.

The Converter automatically includes the standard system data types of Concept individually, if they are needed. They are part of the converter command and include file CConv.xml present in the execution directory of Unity.

If the read-only-flag is removed, this file can be extended to include additional data types for user EFB libraries.

Such data type files beyond the global/local ones are placed in the lib subdirectory of Concept to be merged into the Concept application, but these data types do NOT appear in the Concept export file.

Concept *.dty Files

The V1.1 version of the Concept converter will have the capability to add Concept *.dty files, which are stored in the same directory as the *.asc file, to the converted application, as if their content were appearing inside the *.asc file itself.

Converting Only Parts of a Concept Application

General

The Concept converter is prepared to convert complete applications and parts of applications.

If only parts of a Concept application are needed either

- use a reduced application export with Concept (see sections below) or
- use the Conversion Wizard (see *Conversion with the Conversion Wizard, page 19*).

Single DFB

If a single DFB is needed, make a new application with 1 single section and place a call to the desired DFB into this section.

Export the application using the menu item **Export with used DFBs** in Concept.

Convert the resulting **.asc* file in Unity Pro via **File → Open**.

Subset of Sections

To export a subset of sections use the **File → Export → Program: Section(s)** menu in Concept.

Select the source application and the desired sections and follow the user guidance to get a reduced application.

However, if the section contains references to SFC steps, Concept requires to export the referenced SFC section as well.

Convert the resulting **.sec* file in Unity Pro with the Conversion Wizard via **Tools → Convert Partially**.

Subset of Variables

To export a subset of variables first open the **Variable Editor** in Concept and select the desired variables.

After that use the **File → Export → Variables: Text delimited** menu.

Convert the resulting **.txt* file in Unity Pro with the Conversion Wizard via **Tools → Convert Partially**.

Animation Tables

If animation table files are present in the application export directory, the animation tables will be automatically included in the conversion result.

Removing Accidentally Included Concept Macros

General

If a Concept Macro has been included into the Concept export, this Macro is converted as if it were a DFB and appears in the project browser tree as a DFB.

Delete this DFB because Unity Pro does not support Macros.

Initialization Values

General

Initialization values are contained in Concept export in an array, describing the State RAM.

This array is converted in Unity Pro to clusters, i.e. that it is cut into contiguous sequences of non-zero values with single-zero values tolerated.

Each cluster is converted to an individual array with the names `LL_SRAMxxx`.

If the convertedMomentum application contain more than one XMIT block

Rule

The 171 CBU 78090, 171 CBU 98090, or 171 CBU 98091 processor uses a much faster COM port speed than the legacy Momentum PLC.

As a result of this, the XMIT block might not function properly, if the converted Momentum application contain more than one XMIT block.

The fast COM port speed might result in the next XMIT block to execute before the prior XMIT block completes it's operation. If this happens, the block may appear to be functioning incorrectly.

To avoid this simultaneous execution of the XMIT blocks, a time delay or additional logic to test the DONE output of the XMIT block may need to be added to the block start or enable logic.

Part II
Blocks from Concept to Unity Pro

Overview

This part contains a description of the blocks which are not part of Unity Pro as standard.

However, if these blocks were used in Concept they are generated during the project conversion from Concept to Unity Pro in order to map the functionality configured in Concept into Unity Pro on a one to one basis.

What Is in This Part?

This part contains the following chapters:

Chapter	Chapter Name	Page
7	BYTE_TO_BIT_DFB: Type conversion	117
8	CREADREG: Continuous register reading	121
9	CWRITREG: Continuous register writing	129
10	DINT_AS_WORD_DFB: Type conversion	135
11	DIOSTAT: Module function status (DIO)	137
12	GET_TOD: Reading the hardware clock (Time Of Day)	139
13	LIMIT_IND_DFB: Limit with indicator	143
14	LOOKUP_TABLE1_DFB: Traverse progression with 1st degree interpolation	147
15	PLCSTAT: M580 CPU function status DFB	153
16	PLCSTAT: Quantum PLC function status	159
17	READREG: Read register	175
18	RIOSTAT: Module function status (RIO)	183
19	SET_TOD: Setting the hardware clock (Time Of Day)	187
20	WORD_AS_BYTE_DFB: Type conversion	191
21	WORD_TO_BIT_DFB: Type conversion	193
22	WRITEREG: Write register	197

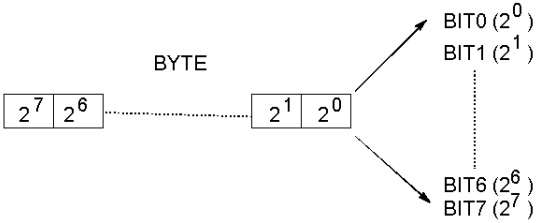
Chapter 7
BYTE_TO_BIT_DFB: Type conversion

Description

Function description

This derived function block converts one input word from the BYTE data type to 8 output values of the BOOL data type.

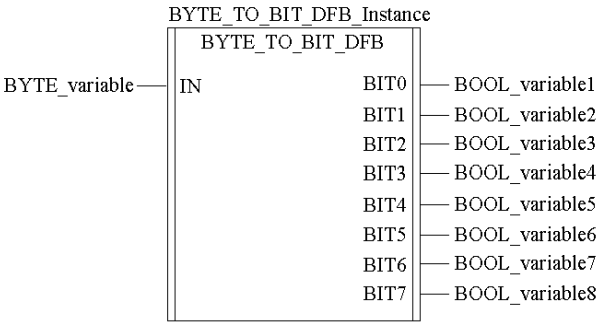
The individual bits of the byte at the input are assigned to the outputs according to the output names.



EN and ENO can be configured as additional parameters.

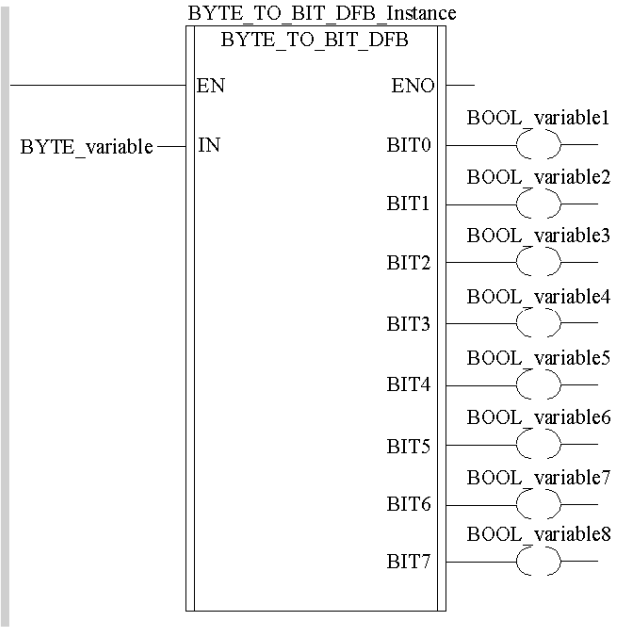
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL BYTE_TO_BIT_DFB_Instance (IN:=BYTE_variable,
    BIT0=>BOOL_variable1, BIT1=>BOOL_variable2,
    BIT2=>BOOL_variable3, BIT3=>BOOL_variable4,
    BIT4=>BOOL_variable5, BIT5=>BOOL_variable6,
    BIT6=>BOOL_variable7, BIT7=>BOOL_variable8)
```

Representation in ST

Representation:

```
BYTE_TO_BIT_DFB_Instance (IN:=BYTE_variable,
    BIT0=>BOOL_variable1, BIT1=>BOOL_variable2,
    BIT2=>BOOL_variable3, BIT3=>BOOL_variable4,
    BIT4=>BOOL_variable5, BIT5=>BOOL_variable6,
    BIT6=>BOOL_variable7, BIT7=>BOOL_variable8) ;
```

Parameter description

Description of the input parameters:

Parameter	Data type	Meaning
IN	BYTE	Input

Description of the output parameters:

Parameter	Data type	Meaning
BIT0	BOOL	Output bit 0
BIT1	BOOL	Output bit 1
:	:	:
BIT7	BOOL	Output bit 7

Chapter 8
CREADREG: Continuous register reading

Introduction

This chapter describes the CREADREG block.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	122
Mode of Functioning	125
Parameter description	126
Modbus Plus Error Codes	127

Description

Function description

This derived function block reads the register area continuously. It reads data from addressed nodes via Modbus Plus.

EN and ENO can be configured as additional parameters.

NOTE: It is necessary to be familiar with the routing procedures of your network when programming a CREADREG function. Modbus Plus routing path structures are described in detail in the Modbus Plus Network Planning and Installation Guide *(see page 13)*.

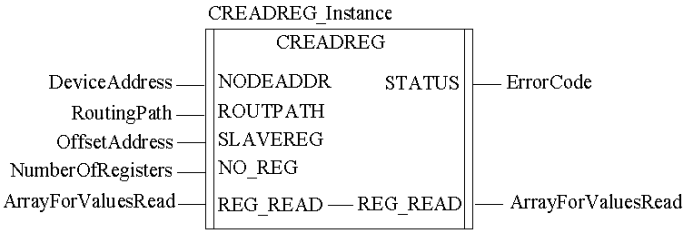
NOTE: This function block only supports the local Modbus Plus interface (no NOM).
If using a NOM work with the block CREAD_REG from the communication block library.

NOTE: This function block does not support TCP/IP- or SY/MAX-Ethernet.
If TCP/IP- or SY/MAX-Ethernet is needed, use the block CREAD_REG of the communication block library.

NOTE: Several copies of this function block can be used in the program. However, multiple instancing of these copies is not possible.

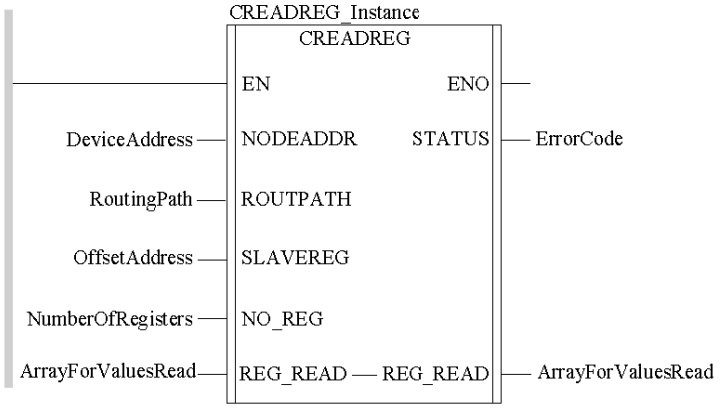
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL CREADREG_Instance (NODEADDR:=DeviceAddress,
    ROUTPATH:=RoutingPath, SLAVEREG:=OffsetAddress,
    NO_REG:=NumberOfRegisters,
    REG_READ:=ArrayForValuesRead,
    STATUS=>ErrorCode)
```

Representation in ST

Representation:

```
CREADREG_Instance (NODEADDR:=DeviceAddress,
    ROUTPATH:=RoutingPath, SLAVEREG:=OffsetAddress,
    NO_REG:=NumberOfRegisters,
    REG_READ:=ArrayForValuesRead,
    STATUS=>ErrorCode;
```

Parameter description

Description of the input parameters:

Parameters	Data type	Meaning
NODEADDR	INT	Device address within the target segment
ROUTPATH	DINT	Routing path to target segment
SLAVEREG	DINT	Offset address of the first 4x register in the slave to be read from
NO_REG	INT	Number of registers to be read from slave

Description of input / output parameters:

Parameters	Data type	Meaning
REG_READ	ANY_ARRAY_WORD	Writing data (For the file to be read a data structure must be declared as a located variable.)

Description of the output parameters:

Parameters	Data type	Meaning
STATUS	WORD	Error Code

Mode of Functioning

Function mode of CREADREG blocks

Although a large number of CREADREG function blocks can be programmed, only four read operations may be active at the same time. It makes no difference whether these operations are performed using this function block or others (e.g. MBP_MSTR, READREG). All function blocks use one data transaction path and require multiple cycles to complete a task.

The complete routing information must be separated into two parts:

- in the NODEADDR of the destination node (regardless of whether it is located in the local segment or in another segment) and
- the routing path, in case there is a link via network bridges.

The resulting destination address consists of these two information components.

The routing path is a DINT data type, which is interpreted as a sequence of two-digit information units. It is not necessary to use "00" extensions (e.g. both routing paths 4711 and 47110000 are valid, for NODEADDR 34 the result is destination address 47.11.34.00.00).

NOTE: This function block puts a heavy load on the network. The network load must therefore be carefully monitored. If the network load is too high, the program logic should be reorganized to work with the READREG function block, which is a variant of this function block that does not operate in continuous mode, but is command driven.

Parameter description

NODEADDR

Identifies the node address within the target segment.

The parameter can be entered as an address, located variable, unlocated variable or literal.

ROUTPATH

Identifies the routing path to the target segment. The two-digit information units run from 01 64 (see *Mode of Functioning, page 125*). If the slave resides in the local network segment, `ROUTPATH` must be set to "0" or must be left unconnected.

The parameter can be entered as an address, located variable, unlocated variable or literal.

SLAVEREG

Start of the area in the addressed slave from which the source data are read. The source area always resides within the 4x register area. `SLAVEREG` expects the source reference as offset within the 4x area. The leading "4" must be omitted (e.g. 59 (contents of the variables or value of the literal) = 40059).

The parameter can be entered as an address, located variable, unlocated variable or literal.

NO_REG

Number of registers to be read from slave processor (1 ... 100).

The parameter can be entered as an address, located variable, unlocated variable or literal.

STATUS

Error code, see *Modbus Plus Error Codes, page 127*

The parameter can be specified as an address, located variable or unlocated variable.

REG_READ

An `ANY_ARRAY_WORD` that is the same size as the requested transmission must be agreed upon (≥ `NO_REG`) for this parameter. The name of this array is defined as a parameter. If the array is defined too small, then only the amount of data is transmitted that is present in the array.

The parameter must be defined as a located variable.

Modbus Plus Error Codes

Form of the function error code

The form of the function error code for Modbus Plus is **Mmss**, which includes:

- **M** is the high code
- **m** is the low code
- **ss** is a subcode

Hexadecimal error code

Hexadecimal error code for Modbus Plus:

Hex. Error Code	Meaning
1001	Abort by user
2001	An operation type that is not supported was specified in the control block
2002	One or more control block parameters were modified while the `MSTR` element was active (this only applies to operations which require several cycles for completion). Control block parameters my only be modified in inactive `MSTR` components.
2003	Illegal value in the length field of the control block
2004	Illegal value in the offset field of the control block
2005	Illegal value in the length and offset fields of the control block
2006	Unauthorized data field on slave
2007	Unauthorized network field on slave
2008	Unauthorized network routing path on slave
2009	Routing path equivalent to own address
200A	Attempting to retrieve more global data words than available
30ss	Unusual response by Modbus slave *(see page 128)*
4001	Inconsistent response by Modbus slave
5001	Inconsistent response by network
6mss	Routing path error *(see page 128)* Subfield m shows where the error occurred (a 0 value means local node, 2 means 2nd device in route, etc) .

ss hexadecimal value in 30ss error code

ss hexadecimal value in 30ss error code:

ss hex. Value	Meaning
01	Slave does not support requested operation
02	Non-existent slave registers were requested
03	An unauthorized data value was requested
05	Slave has accepted a lengthy program command
06	Function cannot currently be carried out: lengthy command running
07	Slave has rejected lengthy program command

ss hexadecimal value in 6mss error code

NOTE: Subfield m in error code 6mss is an `Index` in the routing information that shows where an error has been detected (a 0 value indicates the local node, 2 means the second device in the route, etc.).

The ss subfield in error code 6mss is as follows:

ss hexadecimal value	Meaning
01	No response receipt
02	Access to program denied
03	Node out of service and unable to communicate
04	Unusual response received
05	Router-node data path busy
06	Slave out of order
07	Wrong destination address
08	Unauthorized node type in routing path
10	Slave has rejected the command
20	Slave has lost an activated transaction
40	Unexpected master output path received
80	Unexpected response received
F001	Wrong destination node specified for `MSTR` operation

Chapter 9
CWRITREG: Continuous register writing

Introduction

This chapter describes the CWRITREG block.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	130
Mode of Functioning	133
Parameter description	134

Description

Function description

This derived function block writes continuously to the register area. It transfers data from the PLC via Modbus Plus to a specified slave destination processor.

EN and ENO can be configured as additional parameters.

NOTE: When programming a CWRITREG function, you must be familiar with the routing procedures used by your network. Modbus Plus routing path structures are described in detail in the Modbus Plus Network Planning and Installation Guide *(see page 13)*.

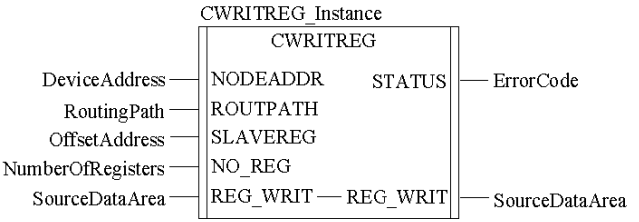
NOTE: This function block only supports the local Modbus Plus interface (no NOM).
If using a NOM work with the CWRITE_REG block from the communication block library.

NOTE: This function block does not support TCP/IP- or SY/MAX-Ethernet.
If TCP/IP- or SY/MAX-Ethernet is needed, use the CWRITE_REG block from the communication block library.

NOTE: Several copies of this function block can be used in the program. However, multiple instancing of these copies is not possible.

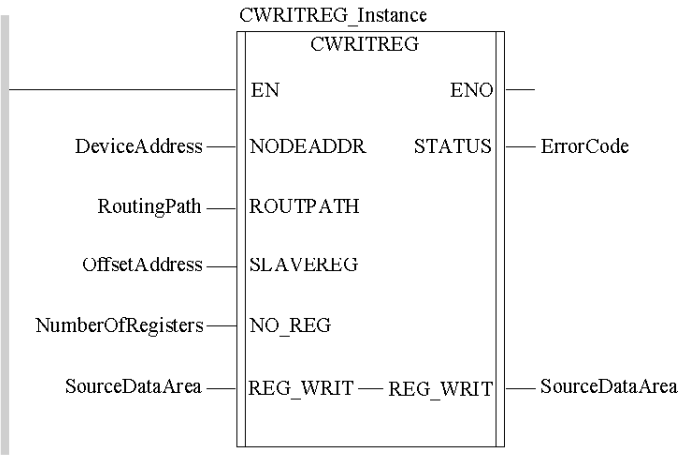
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL CWRITREG_Instance (NODEADDR:=DeviceAddress,
    ROUTPATH:=RoutingPath, SLAVEREG:=OffsetAddress,
    NO_REG:=NumberOfRegisters,
    REG_WRIT:=SourceDataArea,
    STATUS=>ErrorCode)
```

Representation in ST

Representation:

```
CWRITREG_Instance (NODEADDR:=DeviceAddress,
    ROUTPATH:=RoutingPath, SLAVEREG:=OffsetAddress,
    NO_REG:=NumberOfRegisters,
    REG_WRIT:=SourceDataArea,
    STATUS=>ErrorCode) ;
```

Parameter description

Description of the input parameters:

Parameters	Data type	Meaning
NODEADDR	INT	Device address within the target segment
ROUTPATH	DINT	Routing path to target segment
SLAVEREG	DINT	Offset address of the first 4x register in the slave to be written to
NO_REG	INT	Number of registers to be written from slave

Description of input / output parameters:

Parameters	Data type	Meaning
REG_WRIT	ANY_ARRAY_WORD	Source data field (A data structure must be declared as a located variable for the source file.)

Description of the output parameters:

Parameters	Data type	Meaning
STATUS	WORD	Error Code

Mode of Functioning

Function mode of CWRITREG blocks

Although an unlimited number of CWRITREG function blocks can be programmed, only four write operations may be active at the same time. It makes no difference whether these operations are performed using this function block or others (e.g. MBP_MSTR, WRITEREG). All function blocks use one data transaction path and require multiple cycles to complete a task.

If several CWRITREG function blocks are used within an application, they must at least differ in the values of their NO_REG or REG_WRIT parameters.

The complete routing information must be separated into two parts:

- in the NODEADDR of the destination node (regardless of whether it is located in the local segment or in another segment) and
- the routing path, in case there is a link via network bridges.

The resulting destination address consists of these two information components.

The routing path is a DINT data type, which is interpreted as a sequence of two-digit information units. It is not necessary to use "00" extensions (e.g. both routing paths 4711 and 47110000 are valid, for NODEADDR 34 the result is destination address 47.11.34.00.00).

NOTE: This function block puts a heavy load on the network. The network load must therefore be carefully monitored. If the network load is too high, the program logic should be reorganized to work with the WRITEREG function block, which is a variant of this function block that does not operate in continuous mode, but is command driven.

Parameter description

NODEADDR

Identifies the node address within the target segment.

The parameter can be specified as an address, located variable, unlocated variable or literal.

ROUTPATH

Identifies the routing path to the target segment. The two-digit information units run from 01 64 (see *Mode of Functioning, page 133*). If the slave resides in the local network segment, `ROUTPATH` must be set to "0" or must be left unconnected.

The parameter can be specified as an address, located variable, unlocated variable or literal.

SLAVEREG

Start of the destination area in the addressed slave to which the source data are written. The destination area always resides within the 4x register area. `SLAVEREG` expects the destination address as an offset within the 4x area. The leading "4" must be omitted (e.g. 59 (contents of the variables or value of the literal) = 40059).

The parameter can be entered as an address, located variable, unlocated variable or literal.

NO_REG

Number of registers to be written to slave processor (1 ... 100).

The parameter can be specified as an address, located variable, unlocated variable or literal.

REG_WRIT

An `ANY_ARRAY_WORD` that is the same size as the planned transmission must be agreed upon (≥ `NO_REG`) for this parameter. The name of this array is defined as a parameter. If the array is defined too small, then only the amount of data is transmitted that is present in the array.

The parameter must be defined as a located variable.

STATUS

If `MSTR` error code is returned, see *Modbus Plus Error Codes, page 127*

The parameter can be specified as an address, located variable or unlocated variable.

Chapter 10
DINT_AS_WORD_DFB: Type conversion

Description

Function description

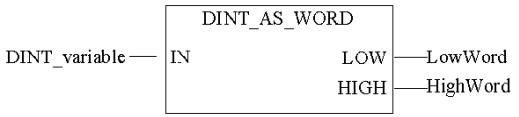
This derived function block converts one input word from the DINT data type to 2 output values of the WORD data type.

The individual words of the DINT input are assigned to the outputs according to the output names.

EN and ENO can be configured as additional parameters.

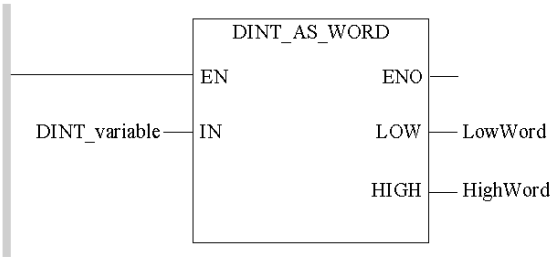
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL DINT_AS_WORD_DFB_Instance (IN:=DINT_variable,
    LOW=>LowWord, HIGH=>HighWord)
```

Representation in ST

Representation:

```
DINT_AS_WORD_DFB_Instance (IN:=DINT_variable,
    LOW=>LowWord, HIGH=>HighWord) ;
```

Parameter description

Description of the input parameters:

Parameters	Data type	Meaning
IN	DINT	Input

Description of the output parameters:

Parameters	Data type	Meaning
LOW	WORD	least significant word
HIGH	WORD	most significant word

Chapter 11
DIOSTAT: Module function status (DIO)

Description

Function description

This function provides the function status for I/O modules of an I/O station (DIO).

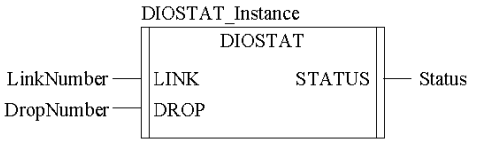
Each module (slot) is displayed as an output "status" bit. The bit on the far left side in "status" corresponds to the slot on the far left side of the I/O station.

NOTE: If a module of the I/O station is configured and works correctly, the corresponding bit is set to "1".

`EN` and `ENO` can be configured as additional parameters.

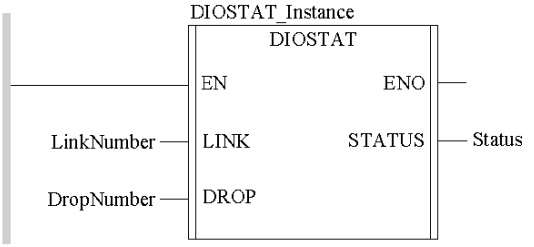
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL DIOSTAT_Instance (LINK:=LinkNumber, DROP:=DropNumber,
    STATUS=>Status)
```

Representation in ST

Representation:

```
DIOSTAT_Instance (LINK:=LinkNumber, DROP:=DropNumber,
    STATUS=>Status) ;
```

Parameter description

Description of the input parameters:

Parameter	Data type	Meaning
LINK	UINT	Link No. (0...2)
DROP	UINT	I/O station no: (1...64)

Description of the output parameters:

Parameter	Data type	Meaning
STATUS	WORD	Status bit pattern *(see page 137)* of an I/O station

Chapter 12
GET_TOD: Reading the hardware clock (Time Of Day)

Description

Function description

This function block searches (together with the other function blocks in the HSBY group) the configuration of the respective PLC for the necessary components. These components always refer to the hardware actually connected.

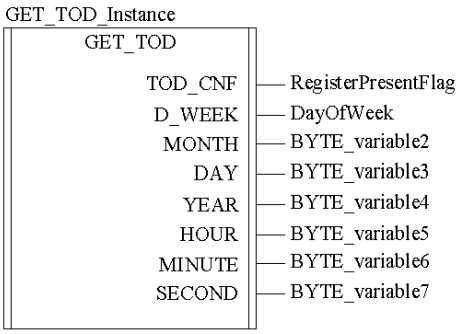
Therefore the correct functioning of this function block on the simulators cannot be guaranteed.

The `GET_TOD` function block reads the hardware clock, if relevant registers are provided with this configuration. If these registers are not present, the `TOD_CNF` output is set to "0".

`EN` and `ENO` can be configured as additional parameters.

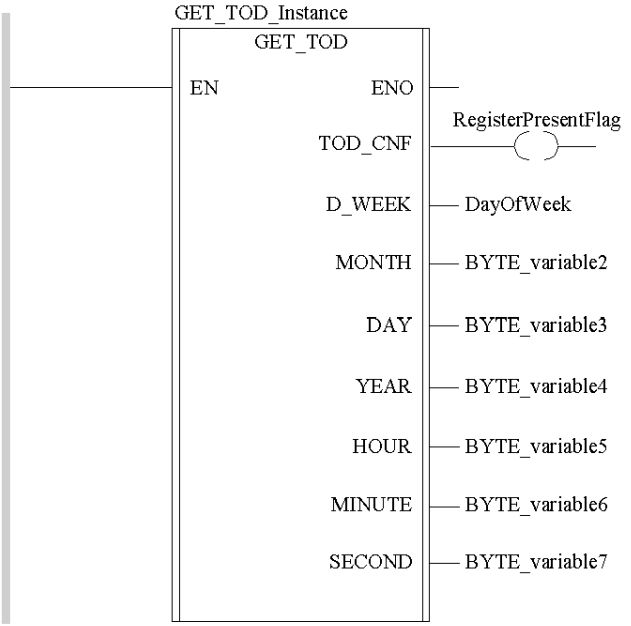
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL GET_TOD_Instance (TOD_CNF=>RegisterPresentFlag,
    D_WEEK=>DayOfWeek, MONTH=>Byte_variable2,
    DAY=>Byte_variable3, YEAR=>Byte_variable4,
    HOUR=>Byte_variable5, MINUTE=>Byte_variable6,
    SECOND=>Byte_variable7)
```

Representation in ST

Representation:

```
GET_TOD_Instance (TOD_CNF=>RegisterPresentFlag,
    D_WEEK=>DayOfWeek, MONTH=>Byte_variable2,
    DAY=>Byte_variable3, YEAR=>Byte_variable4,
    HOUR=>Byte_variable5, MINUTE=>Byte_variable6,
    SECOND=>Byte_variable7) ;
```

Parameter description

Description of the output parameters:

Parameters	Data type	Meaning
TOD_CNF	BOOL	"1" = 4x-register for hardware system clock was found and the clock is operational. "0" = time is set at the moment. In this case the other outputs keep their values.
D_WEEK	BYTE	Weekday, 1 = Sunday .. 7 = Saturday
MONTH	BYTE	Month 1..12
DAY	BYTE	Day 1..31
YEAR	BYTE	Year 0..99
HOUR	BYTE	Hour 0..23
MINUTE	BYTE	Minute 0..59
SECOND	BYTE	Second 0..59

Chapter 13
LIMIT_IND_DFB: Limit with indicator

Description

Function description

This derived function block transfers the unchanged input value (Input) to the Output, if the input value is not less than the minimum value (LimitMinimum) and does not exceed the maximum value (LimitMaximum). If the input value (Input) is less than the minimum value (LimitMinimum), the minimum value will be transferred to the output. If the input value (Input) exceeds the maximum value (LimitMaximum), the maximum value will be transferred to the output.

Additionally, a indication is given if the minimum or maximum value is violated. If the value at the (Input) input is less than the value at the (LimitMinimum) input, the (MinimumViolation) output becomes "1". If the value at the (Input) input is more than the value at the (LimitMaximum) input, the (MaximumViolation) output becomes "1".

The data types of the (LimitMinimum, Input, LimitMaximum) input values and the (Output) output value must be identical.

EN and ENO can be configured as additional parameters.

Formula

Block formula:

OUT = IN, if (IN ≤ MX) & IN ≥ MN

OUT = MN, if (IN < MN)

OUT = MX, if (IN > MX)

MN_IND = 0, if IN ≥ MN

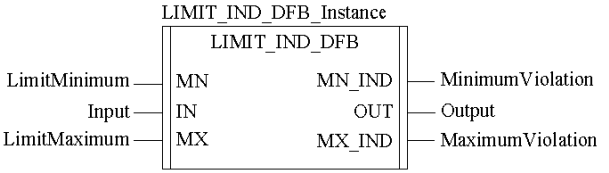
MN_IND = 1, if IN < MN

MX_IND = 0, if IN ≤ MX

MX_IND = 1, if IN > MX

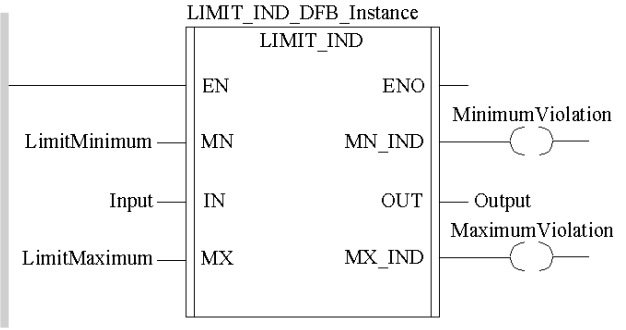
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL LIMIT_IND_DFB (MN:=LimitMinimum, IN:=INPUT,
    MX:=LimitMaximum, MN_IND=>MinimumViolation,
    OUT=>Output, MX_IND=>MaximumViolation)
```

Representation in ST

Representation:

```
LIMIT_IND_DFB (MN:=LimitMinimum, IN:=INPUT,
    MX:=LimitMaximum, MN_IND=>MinimumViolation,
    OUT=>Output, MX_IND=>MaximumViolation) ;
```

Parameter description

Description of the input parameters:

Parameter	Data type	Meaning
LimitMinimum	BOOL, BYTE, WORD, DWORD, INT, DINT, UINT, UDINT, REAL, TIME	Limit of minimum value
Input	BOOL, BYTE, WORD, DWORD, INT, DINT, UINT, UDINT, REAL, TIME	Input
LimitMaximum	BOOL, BYTE, WORD, DWORD, INT, DINT, UINT, UDINT, REAL, TIME	Limit of maximum value

Description of the output parameters:

Parameter	Data type	Meaning
MinimumViolation	BOOL	Display of minimum value violation
Output	BOOL, BYTE, WORD, DWORD, INT, DINT, UINT, UDINT, REAL, TIME	Output
MaximumViolation	BOOL	Display of maximum value violation

Chapter 14
LOOKUP_TABLE1_DFB: Traverse progression with 1st degree interpolation

Introduction

This chapter describes the `LOOKUP_TABLE1_DFB` block.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	148
Detailed description	150

Description

Function description

This function block linearizes characteristic curves by means of interpolation. The function block works with variable support point width.

The number of `XiYi` inputs can be increased to 30 by modifying the size of the block frame vertically. This corresponds to a maximum of 15 support point pairs.

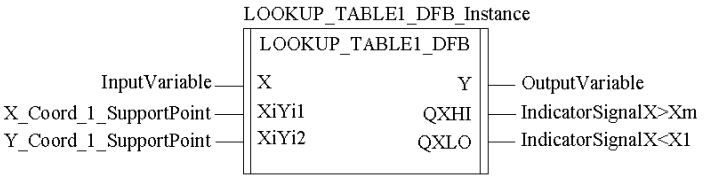
The number of inputs must be even.

The `X` values must be in ascending order.

`EN` and `ENO` can be configured as additional parameters.

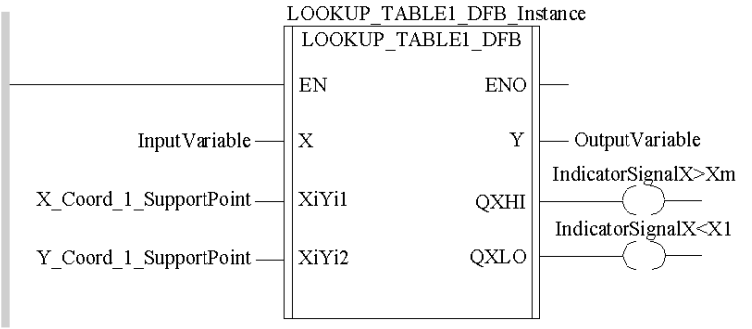
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL LOOKUP_TABLE1_DFB_Instance (X:=InputVariable,
    XiYi1:=X_Coord_1_SupportPoint,
    XiYi2:=Y_Coord_1_SupportPoint, Y=>OutputVariable,
    QXHI=>IndicatorSignalX>Xm, QXLO=>IndicatorSignalX<X1)
```

Representation in ST

Representation:

```
LOOKUP_TABLE1_DFB_Instance (X:=InputVariable,
    XiYi1:=X_Coord_1_SupportPoint,
    XiYi2:=Y_Coord_1_SupportPoint, Y=>OutputVariable,
    QXHI=>IndicatorSignalX>Xm, QXLO=>IndicatorSignalX<X1) ;
```

Parameter description

Description of the input parameters:

Parameter	Data type	Meaning
XiYi1	REAL	X coordinate 1. Support point
XiYi2	REAL	Y coordinate 1. Support point
XiYin	REAL	X coordinate m/2. Support point
XiYim	REAL	Y coordinate m/2. Support point
X	REAL	Input variable

Description of the output parameters:

Parameter	Data type	Meaning
Y	REAL	Output variable
QXHI	BOOL	Indicator: X > Xm
QXLO	BOOL	Indicate X < X1

Detailed description

Parameter description

Each two sequential inputs (XiYi) represent a support point pair. The first input XiYi corresponds to X1, the next one to Y1, the one after that to X2, etc.

For all types of input value in X found between these support points, the corresponding Y output value is interpolated, while the traverse progression between the support points is viewed linearly.

For $X < X_1$ is $Y = Y_1$

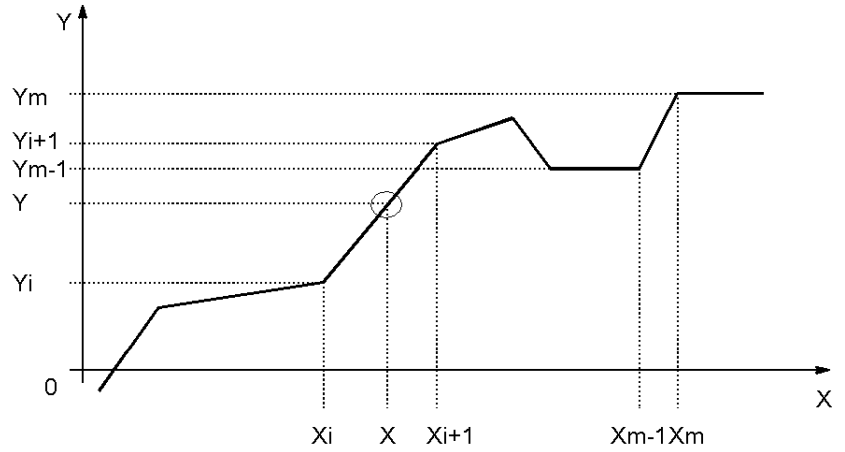
For $X > X_m$ is $Y = Y_m$

If the value at input X is higher than the value of the last support point Xm, the output QXHI becomes "1".

If the value at input X is less than the value of the first support point X1, the output QXLO becomes "1".

Principle of interpolation

Traverse progression with 1st degree interpolation)



Interpolation

The following algorithm applies to a point Y:

$$Y = Y_i + \frac{Y_i + 1 - Y_i}{X_i + 1 - X_i} \times (X - X_i)$$

for $X_i \leq X \leq X_{i+1}$ and i = 1 ... (m-1)

Assuming: $X_1 \leq X_2 \leq ... \leq X_i \leq X_{i+1} \leq ... \leq X_{m-1} \leq X_m$

The X values must be in ascending order.

Two consecutive X values can be identical. This could cause a discrete curve progression.

In this instance, the special case applies:

$Y = 0.5 \times (Y_i + Y_{i+1})$

for

$X_i = X = X_{i+1}$ and i = 1 ... (m-1)

Chapter 15
PLCSTAT: M580 CPU function status DFB

PLC Status DFB

Introduction

Legacy Concept Quantum applications support PLCSTAT, a function block that collects RIO S908 system diagnostics. This function block references system bits and words that are not supported in M580.

A new DFB has therefore been developed for use in M580 applications. The new PLCSTAT DFB is delivered with Unity Pro, but it is not part of the installed library. It is located in the Unity Pro **Extras** folder on your PC.

The DFB is generic in its imported form. It supports both standalone and redundant architectures but is customized for specific applications. Open the ST section within the DFB and follow the customization instructions.

The DFB has an input/output called HSBY_DDDT. For standalone applications, the UMAC tool user creates a dummy Device DDT of type T_M_ECPU_HSBY. This dummy DDDT is assigned to the HSBY_DDDT input/output. For redundant applications, the input/output is assigned to the device DDT HSBY_DDDT that is created when you add a redundant CPU to the configuration.

NOTE: Refer to the *Unity Pro Original LL984 Block Library* for detailed descriptions of the L9_STAT *(see Unity Pro, UnityLL984, Block Library)* and L9_MRTM *(see Unity Pro, UnityLL984, Block Library)* EFBs.

Examples

To use the PLCSTAT DFB, import it into your M580 project from the Unity Pro **Extras** folder. (The DIOSTATE, PLCSTATE, and RIOSTATE DDDTs are imported at the same time.) A generic convError instruction in the ST logic section instructs you to assign the standalone or redundant behavior in the logic.

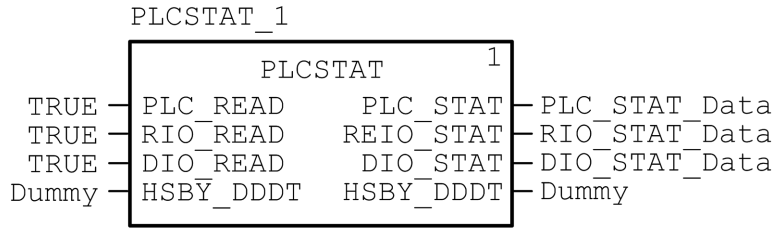
Adjust the logic within the ST section of the DFB by inserting comment tags for each system type. For standalone projects, comment out the convError instruction along with any logic statements between `Second Choice` and `End Second Choice`:

```
(*-- ##### Second Choice: PLC_STAT.word2 for a Hot Standby Application
##### --*)
    (*-- IEC bit 15: set to 1 if hot standby mode --*)
    PLC_STAT.word2 := 16#8000;
    (*-- IEC bit 14: always 0 --*)
    (*-- IEC bit 13: IP or IP+1 = TODO --*)
```

```
    (*-- IEC bit 12: info validity = always valid --*)
    PLC_STAT.word2 := PLC_STAT.word2 or 16#1000;
    (*-- IEC bit 11,10,9: not used, always 0 --*)
    (*-- IEC bit 8: copro fimware mismatch = not significant, always 0
(no copro) --*)
    (*-- IEC bit 7: all firmware (CPU, copro, CRP) mismatch --*)
    if (HSBY_DDDT.FW_MISMATCH) then
        PLC_STAT.word2 := PLC_STAT.word2 or 16#0080;
    end_if;
    (*-- IEC bit 6: CPU-sync link status --*)
    if (NOT(    HSBY_DDDT.LOCAL_HSBY_STS.HSBY_LINK_ERROR
        OR HSBY_DDDT.REMOTE_HSBY_STS.HSBY_LINK_ERROR)) then
        PLC_STAT.word2 := PLC_STAT.word2 or 16#0040;
    end_if;
    (*-- IEC bit 5: unit A or B --*)
    if (HSBY_DDDT.LOCAL_HSBY_STS.PLC_B) then
        PLC_STAT.word2 := PLC_STAT.word2 or 16#20;
    end_if;
    (*-- IEC bit 5: unit A or B --*)
    if (HSBY_DDDT.LOCAL_HSBY_STS.PLC_B) then
        PLC_STAT.word2 := PLC_STAT.word2 or 16#20;
    end_if;
    (*-- IEC bit 4: Application mismatch --*)
    if (     HSBY_DDDT.APP_MISMATCH
        OR HSBY_DDDT.LOGIC_MISMATCH
        OR HSBY_DDDT.OFFLINE_BUILD_MISMATCH) then
            PLC_STAT.word2 := PLC_STAT.word2 or 16#0010;
    end_if;
    (*-- IEC bits 3, 2: remote system state --*)
    if (     HSBY_DDDT.REMOTE_STS_VALID
         AND HSBY_DDDT.REMOTE_HSBY_STS.RUN_PRIMARY) then
          PLC_STAT.word2 := PLC_STAT.word2 or 16#0008; (*-- 10 = primary
--*)
```

```
    elsif (     HSBY_DDDT.REMOTE_STS_VALID
        AND HSBY_DDDT.REMOTE_HSBY_STS.RUN_STANDBY) then
          PLC_STAT.word2 := PLC_STAT.word2 or 16#0008; (*-- 10 = primary
--*)
    elsif (     HSBY_DDDT.REMOTE_STS_VALID
        AND HSBY_DDDT.REMOTE_HSBY_STS.RUN_STANDBY) then
      PLC_STAT.word2 := PLC_STAT.word2 or 16#000C; (*-- 11 = standby --*)
    else
      PLC_STAT.word2 := PLC_STAT.word2 or 16#0004; (*-- 01 = offline --*)
    end_if;
    (*-- IEC bits 1, 0: local system state --*)
    if (HSBY_DDDT.LOCAL_HSBY_STS.RUN_PRIMARY) then
      PLC_STAT.word2 := PLC_STAT.word2 or 16#0002; (*-- 10 = primary --*)
    elsif (HSBY_DDDT.LOCAL_HSBY_STS.RUN_STANDBY) then
      PLC_STAT.word2 := PLC_STAT.word2 or 16#0003; (*-- 11 = standby --*)
    else
      PLC_STAT.word2 := PLC_STAT.word2 or 16#0001; (*-- 01 = offline --*)
    end_if;
    (*-- ##### End Second Choice: PLC_STAT.word2 for a Hot Standby
Application                 ##### --*)
```

This is the PLCSTAT DFB for standalone systems in an FBD section:



For redundant projects, comment out the convError statement and any logic statements between `First Choice` and `End First Choice`:

```
(* --
########################################################################
#################### -- *);
```

```
   (* -- ##### PLC_STAT.word2 contains Hot Standby status if application
is Hot Standby typed, ##### -- *);

    (* -- ##### otherwise in an Standalone Application, this word do not
have significative     ##### -- *);

  (* -- ##### value. In this case it is set to a zero value.
##### -- *);

    (* -- ##### User of the Converter have to choose between the 2
implementations and keep one ##### -- *);

    (* -- ##### (leaving in comment the non-used portion, so available
for a further change)    ##### -- *);

    {ConvError('PLCSTAT DFB type: PLC_STAT.word2 discussion: Choose one
implementation depending of targeted application, Standalone or Hot
Standby.');}

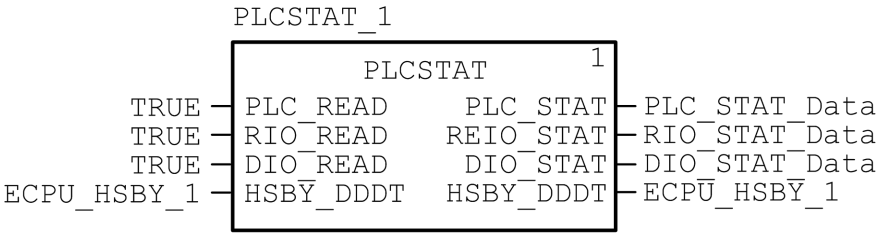
 (*-- ##### First Choice: PLC_STAT.word2 for a Standalone Application
##### --*)

   (*-- Hot standby Status it is not relevant in a Standalone Application
--*)

    PLC_STAT.word2 := 0;

    (*-- ##### End First Choice: PLC_STAT.word2 for a Standalone
Application                  #####
```

This is the PLCSTAT DFB for redundant systems in an FBD section:



This table describes the inputs and outputs of the PLCSTAT DFB:

Pin	Description	
PLC_READ	input	BOOL
RIO_READ	input	BOOL
DIO_READ	input	BOOL
HSBY_DDDT	input/output	HSBY_DDDT is assigned to the redundant DTM that is created when a redundant processor is selected.
PLC_STAT	output	DDT (1-word array)
REIO_STAT	output	DDT (160-word array)
DIO_STAT	output	DDT (106-word array)

Chapter 16
PLCSTAT: Quantum PLC function status

Introduction

This chapter describes the PLCSTAT block for Modicon Quantum platform.

NOTE: For Modicon M580 application, a PLCSTAT DFB *(see page 153)* is available in Unity Pro application folder: **Start menu → All Programs → Schneider Electric → SoCollaborative → Unity Pro → Extras → Converter Quantum M580**.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	160
Derived Data Types	162
PLC status (PLC_STAT)	164
RIO status (RIO_STAT) for Quantum	166
DIO status (DIO_STAT)	168

Description

Function description

This derived function block reads the Quantum PLC internal states and error bits and copies this data to the data structures allocated to the respective outputs.

`EN` and `ENO` can be configured as additional parameters.

Only data with the input bit (`PLC_READ, RIO_READ, DIO_READ`) set to "1" will be read.

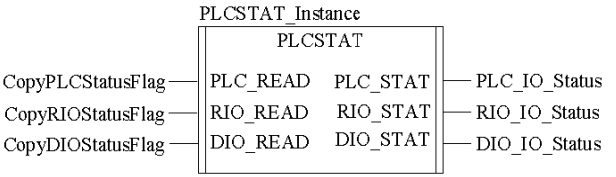
Evaluation

The evaluation of `PLC_STAT` (PLC status), `RIO_STAT` (I/O status) and `DIO_STAT` (I/O communications status) is possible.

NOTE: The name of the output `DIO_STAT` is confusing. This output only relates to the remote I/O Drop Status Information (S908) and not to the Distributed I/O status. To read the distributed I/O status use the function block `DIOSTAT` *(see page 137)*.

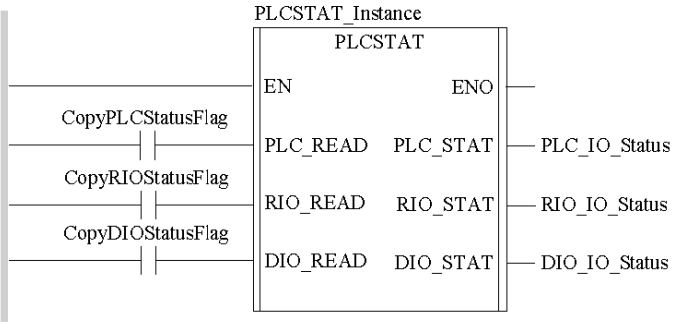
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL PLCSTAT_Instance (PLC_READ:=CopyPLCStatusFlag,
    RIO_READ:=CopyRIOStatusFlag,
    DIO_READ:=CopyDIOStatusFlag,
    PLC_STAT=>PLC_IO_Status, RIO_STAT=>RIO_IO_Status,
    DIO_STAT=>DIO_IO_Status)
```

Representation in ST

Representation:

```
PLCSTAT_Instance (PLC_READ:=CopyPLCStatusFlag,
    RIO_READ:=CopyRIOStatusFlag,
    DIO_READ:=CopyDIOStatusFlag,
    PLC_STAT=>PLC_IO_Status, RIO_STAT=>RIO_IO_Status,
    DIO_STAT=>DIO_IO_Status) ;
```

PLCSTAT parameter description

Description of the input parameters:

Parameters	Data type	Meaning
PLC_READ	BOOL	1 = copies the PLC status from the status table to the output PLC_STAT.
RIO_READ	BOOL	1 = copies the RIO status from the status table to the output RIO_STAT.
DIO_READ	BOOL	1 = copies the DIO status from the status table to the output DIO_STAT.

Description of the output parameters:

Parameters	Data type	Meaning
PLC_STAT	PLCSTATE,	Contains the PLC status.
RIO_STAT	RIOSTATE,	Contains the RIO status (I/O status) for Quantum
DIO_STAT	DIOSTATE,	Contains the DIO status (I/O communication status) **Note:** The name of this output is confusing. This output only relates to the remote I/O Drop Status Information (S908) and not to the Distributed I/O status. To read the distributed I/O status use the function block DIOSTAT *(see page 137)*.

Derived Data Types

Element description PLCSTATE

Description of the PLCSTATE element:

Element	Data type	Meaning
word1	WORD	CPU status
word2	WORD	Hot Standby Status
word3	WORD	PLC status
word4	WORD	RIO Status
word5	WORD	Reserve
word6	WORD	Reserve
word7	WORD	Reserve
word8	WORD	Reserve
word9	WORD	Reserve
word10	WORD	Reserve
word11	WORD	Reserve

Element description RIOSTATE

Description of the RIOSTATE element

Element	Data type	Meaning
word1	WORD	I/O station 1, module rack 1
word2	WORD	I/O station 1, module rack 2
...	...	...
word5	WORD	I/O station 1, module rack 5
word6	WORD	I/O station 2, module rack 1
word7	WORD	I/O station 2, module rack 2
...	...	...
word160	WORD	I/O station 32, module rack 5

Element description DIOSTATE

Description of the DIOSTATE element

Element	Data type	Meaning
word1	WORD	Switch on error numbers:
word2	WORD	Cable A error
word3	WORD	Cable A error
word4	WORD	Cable A error
word5	WORD	Cable B error
word6	WORD	Cable B error
word7	WORD	Cable B error
word8	WORD	Global communication status
word9	WORD	Global cumulative error counter for cable A
word10	WORD	Global cumulative error counter for cable B
word11	WORD	I/O station 1 health status and repetition counter (first word)
word12	WORD	I/O station 1 health status and repetition counter (second word)
word13	WORD	I/O station 1 health status and repetition counter (third word)
word14	WORD	I/O station 2 health status and repetition counter (first word)
...	...	...
word104	WORD	I/O station 32 health status and repetition counter (first word)
word105	WORD	I/O station 32 health status and repetition counter (second word)
word106	WORD	I/O station 32 health status and repetition counter (third word)

PLC status (PLC_STAT)

General information

NOTE: Information corresponds to status table words 1 to 11 in the dialog **PLC status**.

The conditions are true when the bits are set to 1.

PLC status (PLCSTATE: word1)

Bit allocation:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Bit	Allocation
10	Run light OFF
11	Memory protect OFF
12	Battery failed

Hot Standby status (PLCSTATE: word2)

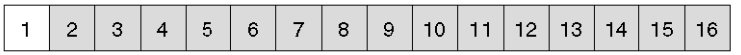
Bit allocation:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Bit	Allocation
1	CHS 110/S911/R911 present and OK
11	0 = CHS shift switch set to A 1 = CHS shift switch set to B
12	0 = PLCs have equal logic 1 = PLCs have unequal logic
13, 14	Remote system condition Dec binary 1 0 1 = Offline 2 1 0 = Primary 3 1 1 = Standby
15, 16	Local system condition Dec binary 1 0 1 = Offline 2 1 0 = Primary 3 1 1 = Standby

PLC status (PLCSTATE: word3)

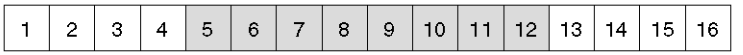
Bit allocation:



Bit	Allocation
1	First cycle

RIO status (PLCSTATE: word4)

Bit allocation:



Bit	Allocation
1	IOP defect
2	IOP timeout
3	IOP Loopback
4	IOP memory disturbance
13-16	00 IO has not responded 01 no response 02 Loopback defect

RIO status (RIO_STAT) for Quantum

General information

NOTE: The information corresponds to status table words 12 to 171 in the PLC status dialog.

The words show the I/O module function status.

Five words are reserved for each of the maximum 32 I/O stations. Each word corresponds to one of maximal 2 possible module racks in each I/O station.

Function display for Quantum hardware

Each of the module racks for Quantum hardware can contain up to 15 I/O modules (except for the first rack which contains a maximum 14 I/O modules). Bit 1... 16 in each word show the corresponding I/O module function display in the racks.

I/O module function status

Bit allocation:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Bit	Allocation
1	Slot 1
2	Slot 2
...	...
16	Slot 16

Conditions for a correct function display

Four conditions must be fulfilled if an I/O module can give a correct function display:

- The data traffic of the slot has to be monitored.
- The slot must be valid for the inserted module.
- Valid communication must be established between the module and the RIO interface at RIO stations.
- Valid communication must be established between the I/O processor in the PLC and the RIO interface at the RIO station.

Status words for the MMI user controllers

The status of the 32 element button controllers and PanelMate units in a RIO network can also be monitored with an I/O function status word. The button controllers are located on slot 4 in a I/O rack and can be monitored at bit 4 of the corresponding status word. A PanelMate on RIO is located on slot 1 in module rack 1 of the I/O station and can be monitored at bit 1 of the first status word for the I/O station.

NOTE: The ASCII keyboard communication status can be monitored with the error numbers in the ASCII read/write instructions.

DIO status (DIO_STAT)

General information

NOTE: The information corresponds to status table words 172 to 277 in the PLC status dialog.

The words contain the I/O communication status (DIO status) Words 1 to 10 are global status words. Of the remaining 96 words, three words are allocated to each of the up to 32 I/O stations.

word1 saves the switch on error numbers. This word is always 0 when the system is running. If an error occurs, the PLC does not start but generates a PLC stop status (word5 from PLC_STAT).

The conditions are true when the bits are set to 1.

Switch on error numbers (DIOSTATE word1)

The conditions are true when the bits are set to 1.

Switch on error numbers:

Code	Error	Meaning (location of error)
01	BADTCLEN	Traffic cop length
02	BADLNKNUM	RIO link number
03	BADNUMDPS	I/O station number in traffic cop
04	BADTCSUM	Traffic cop checksum
10	BADDDLEN	I/O station descriptor length
11	BADDRPNUM	I/O station number
12	BADHUPTIM	I/O station stop time
13	BADASCNUM	ASCII port number
14	BADNUMODS	Module number in I/O station
15	PRECONDRP	I/O station is already configured
16	PRECONPRT	Port is already configured
17	TOOMNYOUT	More than 1024 output locations
18	TOOMNYINS	More than 1024 input points
20	BADSLTNUM	Module slot address
21	BADRCKNUM	Rack address
22	BADOUTBC	Number of output bytes
23	BADINBC	Number of input bytes
25	BADRF1MAP	First reference number
26	BADRF2MAP	Second reference number
27	NOBYTES	No input or output bytes
28	BADDISMAP	I/O marker bit not at 16 bit limit

Code	Error	Meaning (location of error)
30	BADODDOUT	Unmated, odd output module
31	BADODDIN	Unmated, odd input module
32	BADODDREF	Unmated odd module reference
33	BAD3X1XRF	1x-reference after 3x-register
34	BADDMYMOD	Dummy module reference already in use
35	NOT3XDMY	3x-module is not a dummy module
36	NOT4XDMY	4x-module is not a dummy module
40	DMYREAL1X	Dummy module, then real 1x-module
41	REALDMY1X	Real, then 1x-dummy module
42	DMYREAL3X	Dummy module, then real 3x-module
43	REALDMY3X	Real, then 3x-dummy module

Status of cable A (`DIOSTATE: word2, word3, word4`)

Bit allocation for `word2`:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

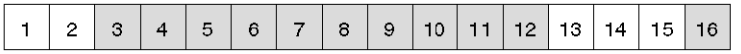
Bit	Allocation
1 - 8	Counts frame fields
9 - 16	Counts DMA receiver overflows

Bit allocation for `word3`:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Bit	Allocation
1 - 8	Counts receiver errors
9 - 16	Counts I/O station receiver failures

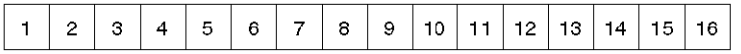
Bit allocation for `word4`:



Bit	Allocation
1	1 = frame too short
2	1 = no frame end
13	1 = CRC error
14	1 = alignment error
15	1 = overflow error

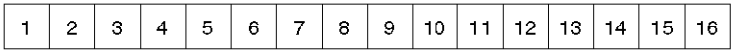
Status of cable B (`DIOSTATE: word5, word6, word7`)

Bit allocation for `word5`:



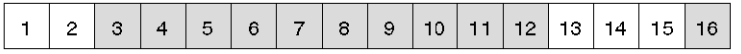
Bit	Allocation
1 - 8	Counts frame fields
9 - 16	Counts DMA receiver overflows

Bit allocation for `word6`:



Bit	Allocation
1 - 8	Counts receiver errors
9 - 16	Counts I/O station receiver failures

Bit allocation for `word7`:



Bit	Allocation
1	1 = frame too short
2	1 = no frame end
13	1 = CRC error
14	1 = alignment error
15	1 = overflow error

Global communication status (DIOSTATE: word8)

The conditions are true when the bits are set to 1.

Bit allocation for word8:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Bit	Allocation
1	Comm. function display
2	Cable A status
3	Cable B status
5 - 8	Communication counter lost
9 - 16	Cumulative repetition counter

Global cumulative error counter for cable A (DIOSTATE: word9)

The conditions are true when the bits are set to 1.

Bit allocation for word9:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Bit	Allocation
1 - 8	Counts recognized errors
9 - 16	Counts zero responses

Global cumulative error counter for cable B(DIOSTATE: word10)

The conditions are true when the bits are set to 1.

Bit allocation for word10:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Bit	Allocation
1 - 8	Counts recognized errors
9 - 16	Counts zero responses

RIO status (DIOSTATE: word11 to word106)

Words 11 to 106 are used to describe the RIO station status, three status words are planned for each I/O station.

The **first** word in each group of three shows the communication status for the corresponding I/O station:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Bit	Allocation
1	Communication health
2	Cable A status
3	Cable B status
5 - 8	Counter for lost communications
9 - 16	Cumulative repetition counter

The **second** word in each group of three is the cumulative I/O station error counter at cable A for the corresponding I/O station:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Bit	Allocation
1 - 8	Minimum one error in words 2 to 4
9 - 16	Counts zero responses

The **third** word in each group of three is the cumulative I/O station error counter at cable B for the corresponding I/O station:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Bit	Allocation
1 - 8	Minimum one error in words 5 to 7
9 - 16	Counts zero responses

NOTE: For PLCs where the I/O station 1 is reserved for the local I/O, words `word11` to `word13` are allocated as follows:

word11 shows the global I/O station status:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Bit	Allocation
1	All modules OK
9 - 16	Counts, how often a module is regarded as not OK, counter overflow is at 255

word12 is used as a 16 bit I/O bus error counter.

word13 is used as a 16 bit I/O repetition counter.

Chapter 17
READREG: Read register

Introduction

This chapter describes the READREG block.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	176
Mode of Functioning	179
Parameter description	180

Description

Function description

With a rising edge at the REQ input, this function block reads a register area from an addressed slave via Modbus Plus.

EN and ENO can be configured as additional parameters.

NOTE: When programming a READREG function, you must be familiar with the routing procedures used by your network. Modbus Plus routing path structures are described in detail in the Modbus Plus Network Planning and Installation Guide *(see page 13)*.

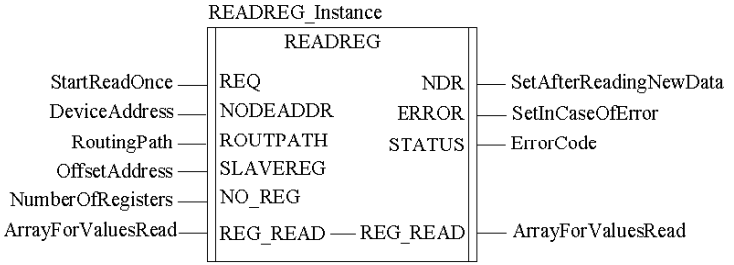
NOTE: This function block only supports the local Modbus Plus interface (no NOM).
If using a NOM, work with the CREAD_REG block from the communication block library.

NOTE: This function block does not support TCP/IP- or SY/MAX-Ethernet.
If TCP/IP- or SY/MAX-Ethernet is needed, use the CREAD_REG block from the communication block library.

NOTE: Several copies of this function block can be used in the program. However, multiple instancing of these copies is not possible.

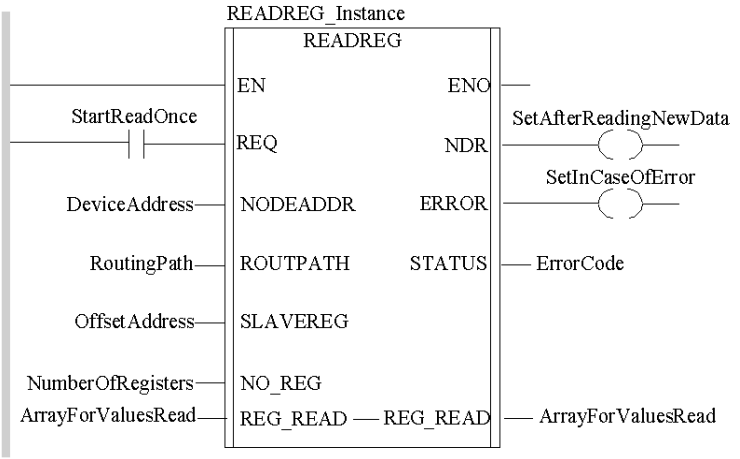
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL READREG_Instance (REQ:=StartReadOnce,
    NODEADDR:=DeviceAddress, ROUTPATH:=RoutingPath,
    SLAVEREG:=OffsetAddress, NO_REG:=NumberOfRegisters,
    REG_READ:=ArrayForValuesRead,
    NDR=>SetAfterReadingNewData, ERROR=>SetInCaseOfError,
    STATUS=>ErrorCode)
```

Representation in ST

Representation:

```
READREG_Instance (REQ:=StartReadOnce,
    NODEADDR:=DeviceAddress, ROUTPATH:=RoutingPath,
    SLAVEREG:=OffsetAddress, NO_REG:=NumberOfRegisters,
    REG_READ:=ArrayForValuesRead,
    NDR=>SetAfterReadingNewData, ERROR=>SetInCaseOfError,
    STATUS=>ErrorCode;
```

Parameter description

Description of the input parameters:

Parameter	Data type	Meaning
REQ	BOOL	With a rising edge at the REQ input, this function block reads a register area from an addressed slave via Modbus Plus.
NODEADDR	INT	Device address within the target segment
ROUTPATH	DINT	Routing path to target segment
SLAVEREG	DINT	Offset address of the first 4x register in the slave to be read from
NO_REG	INT	Number of registers to be read from slave

Description of input / output parameters:

Parameters	Data type	Meaning
REG_READ	ANY_ARRAY_WORD	Writing data (For the file to be read a data structure must be declared as a located variable.)

Description of the output parameters:

Parameters	Data type	Meaning
NDR	BOOL	Set to 1 for one cycle after reading new data
ERROR	BOOL	Set to 1 for one scan in case of error
STATUS	WORD	Error Code

Mode of Functioning

Function mode of READREG_DFB blocks

Although a large number of READREG function blocks can be programmed, only four read operations may be active at the same time. It makes no difference whether these operations are performed using this function block or others (e.g. MBP_MSTR, CREAD_REG). All function blocks use one data transaction path and require multiple cycles to complete a task. The status signals NDR and ERROR report the function block state to the user program.

The complete routing information must be separated into two parts:

- in the NODEADDR of the destination node (regardless of whether it is located in the local segment or in another segment) and
- the routing path, in case there is a link via bridges.

The resulting destination address consists of these two information components.

The routing path is a DINT data type, which is interpreted as a sequence of two-digit information units. It is not necessary to use "00" extensions (e.g. both routing paths 4711 and 47110000 are valid, for NODEADDR 34 the result is destination address 47.11.34.00.00).

Parameter description

REQ

A rising edge triggers the read transaction.

The parameter can be specified as an address, located variable, unlocated variable or literal.

NODEADDR

Identifies the node address within the target segment.

The parameter can be specified as an address, located variable, unlocated variable or literal.

ROUTPATH

Identifies the routing path to the target segment. The two-digit information units run from 01 64 (see *Mode of Functioning, page 179*). If the slave resides in the local network segment, `ROUTPATH` must be set to "0" or must be left unconnected.

The parameter can be specified as an address, located variable, unlocated variable or literal.

SLAVEREG

Start of the area in the addressed slave from which the source data is read. The source area always resides within the 4x register area. `SLAVEREG` expects the source reference as offset within the 4x area. The leading "4" must be omitted (e.g. 59 (contents of the variables or value of the literal) = 40059).

The parameter can be specified as an address, located variable, unlocated variable or literal.

NO_REG

Number of registers to be read from slave processor (1 ... 100).

The parameter can be specified as an address, located variable, unlocated variable or literal.

NDR

Transition to ON state for one program cycle signifies receipt of new data ready to be processed.

The parameter can be specified as an address, located variable or unlocated variable.

ERROR

Transition to ON state for one program cycle signifies detection of a new error.

The parameter can be specified as an address, located variable or unlocated variable.

STATUS

Error code, see *Modbus Plus Error Codes, page 127*

The parameter can be specified as an address, located variable or unlocated variable.

REG_READ

An ANY_ARRAY_WORD that is the same size as the requested transmission must be agreed upon (≥ NO_REG) for this parameter. The name of this array is defined as a parameter. If the array is defined too small, then only the amount of data is transmitted that is present in the array.

The parameter must be defined as a located variable.

Chapter 18
RIOSTAT: Module function status (RIO)

Description

Function description

This function block provides the function status for I/O modules of an I/O station (local/remote I/O).

Quantum I/O or 800 I/O can be used.

An output STATUSx is allocated to each rack. Each module (slot) of this rack is characterized by a bit of the corresponding STATUSx output. The bit on the far left-hand side in STATUSx corresponds to the slot on the far left-hand side of the rack x.

Using STATUS1 to STATUS5:

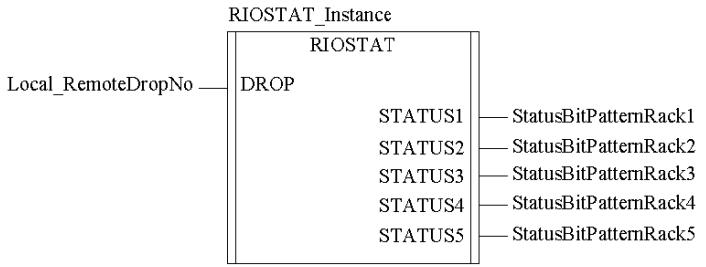
- **Quantum I/O**
  There is only one rack for an I/O station, e.g. only STATUS1 is used.
- **800 I/O**
  There can be up to 5 racks for an I/O station, e.g. STATUS1 corresponds to module rack 1, STATUS5 corresponds to module rack 5.

NOTE: If a module on the module rack has been configured and works correctly, the corresponding bit is set to "1".

EN and ENO can be configured as additional parameters.

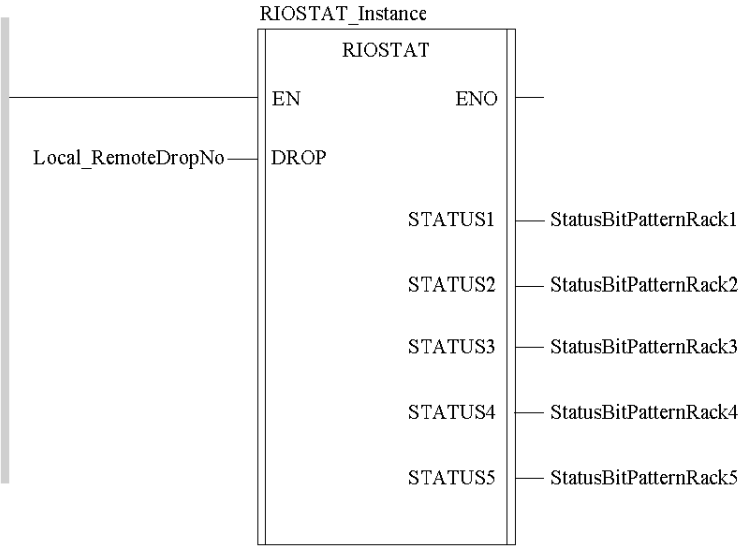
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL RIOSTAT_Instance (DROP:=Local_RemoteDropNo,
    STATUS1=>StatusBitPatternRack1,
    STATUS2=>StatusBitPatternRack2,
    STATUS3=>StatusBitPatternRack3,
    STATUS4=>StatusBitPatternRack4,
    STATUS5=>StatusBitPatternRack5)
```

Representation in ST

Representation:

```
RIOSTAT_Instance (DROP:=Local_RemoteDropNo,
    STATUS1=>StatusBitPatternRack1,
    STATUS2=>StatusBitPatternRack2,
    STATUS3=>StatusBitPatternRack3,
    STATUS4=>StatusBitPatternRack4,
    STATUS5=>StatusBitPatternRack5) ;
```

Parameter description

Description of the input parameters:

Parameters	Data type	Meaning
DROP	UINT	Local/remote I/O station no. (1...32)

Description of the output parameters:

Parameters	Data type	Meaning
STATUS1	WORD	Module rack 1 status bit pattern
STATUS2	WORD	Module rack 2 status bit pattern (800 I/O only)
...	...	...
STATUS5	WORD	Module rack 5 status bit pattern (800 I/O only)

Chapter 19
SET_TOD: Setting the hardware clock (Time Of Day)

Description

Function description

This function block searches (together with the other function blocks in the HSBY group) the configuration of the respective PLC for the necessary components. These components always refer to the hardware actually connected.

Therefore the correct functioning of this function block on the simulators cannot be guaranteed.

The function block sets the hardware system clock, if the corresponding registers are provided within this configuration. If these registers are not present, the `TOD_CNF` output is set to "0".

The function block reads the input values on the `S_PULSE` input at a rising edge and transfers them to the hardware clock.

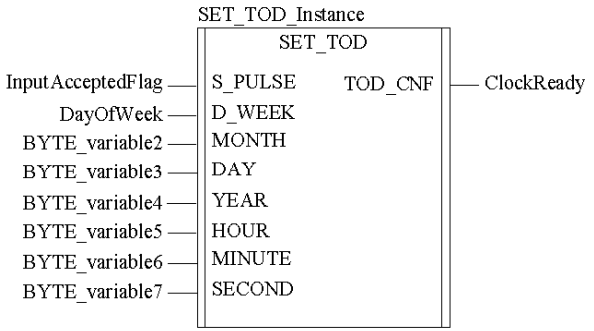
For all input values:

- If the value exceeds the specified maximum value, the maximum is used.
- If the value falls below the specified minimum value, the minimum is used.

`EN` and `ENO` can be configured as additional parameters.

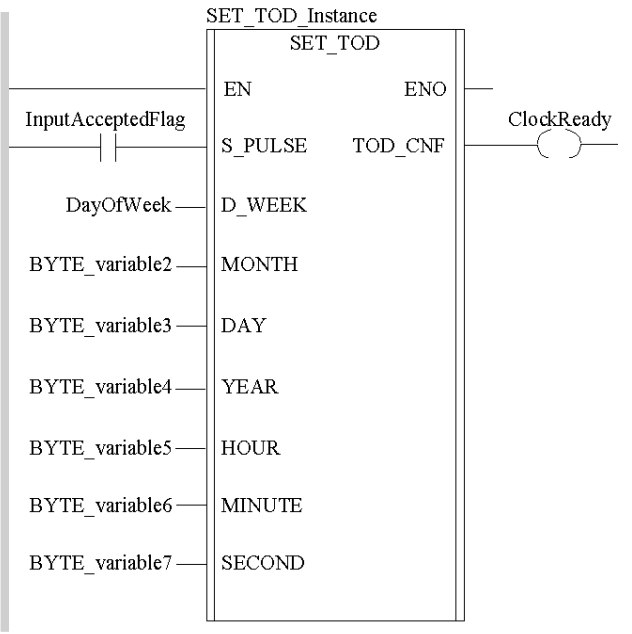
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL SET_TOD_Instance (S_PULSE:=InputAcceptedFlag,
    D_WEEK:=DayOfWeek, MONTH:=Byte_variable2,
    DAY:=Byte_variable3, YEAR:=Byte_variable4,
    HOUR:=Byte_variable5, MINUTE:=Byte_variable6,
    SECOND:=Byte_variable7, TOD_CNF=>ClockReady)
```

Representation in ST

Representation:

```
SET_TOD_Instance (S_PULSE:=InputAcceptedFlag,
    D_WEEK:=DayOfWeek, MONTH:=Byte_variable2,
    DAY:=Byte_variable3, YEAR:=Byte_variable4,
    HOUR:=Byte_variable5, MINUTE:=Byte_variable6,
    SECOND:=Byte_variable7, TOD_CNF=>ClockReady) ;
```

Parameter description

Description of the input parameters:

Parameters	Data type	Meaning
S_PULSE	BOOL	"0 -> 1" = the input values are accepted and written into the clock.
D_WEEK	BYTE	Day of week, 1 = Sunday 7 = Saturday
MONTH	BYTE	Month 1..12
DAY	BYTE	Day 1..31
YEAR	BYTE	Year 0..99
HOUR	BYTE	Hour 0..23
MINUTE	BYTE	Minute 0..59
SECOND	BYTE	Second 0..59

Description of the output parameters:

Parameters	Data type	Meaning
TOD_CNF	BOOL	"1" = %MW register (4x) for the hardware system clock was found and the clock is operational. "0" = Time is currently being set or hardware clock was not found.

Chapter 20
WORD_AS_BYTE_DFB: Type conversion

Description

Function description

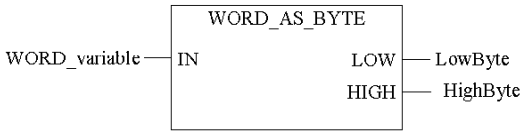
This derived function block converts one input word from the `WORD` data type to 2 output values of the `BYTE` data type.

The individual bytes of the word at the input are assigned to the outputs according to the output names.

`EN` and `ENO` can be configured as additional parameters.

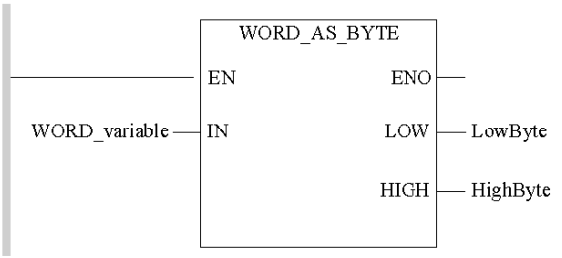
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL WORD_AS_BYTE_DFB_Instance (IN:=WORD_variable,
    LOW=>LowByte, HIGH=>HighByte)
```

Representation in ST

Representation:

```
WORD_AS_BYTE_DFB_Instance (IN:=WORD_variable,
    LOW=>LowByte, HIGH=>HighByte) ;
```

Parameter description

Description of the input parameters:

Parameter	Data type	Meaning
IN	WORD	Input

Description of the output parameters:

Parameter	Data type	Meaning
LOW	BYTE	least significant byte
HIGH	BYTE	most significant byte

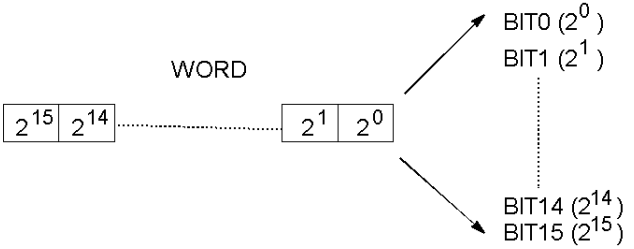
Chapter 21
WORD_TO_BIT_DFB: Type conversion

Description

Function description

This derived function block converts one input word from the WORD data type to 16 output values of the BOOL data type.

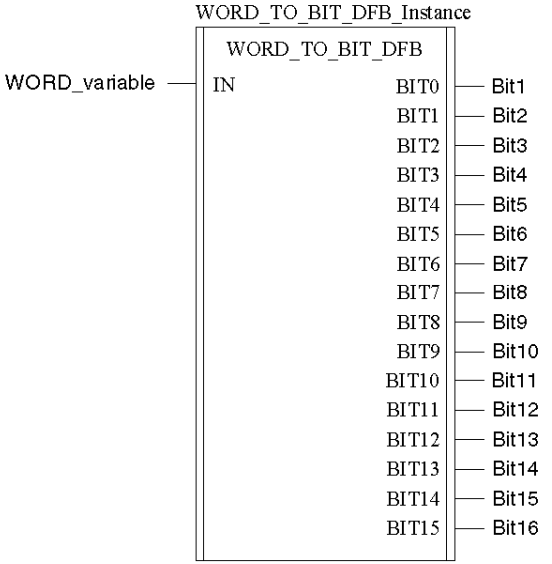
The individual bits of the word at the input are assigned to the outputs according to the output names.



EN and ENO can be configured as additional parameters.

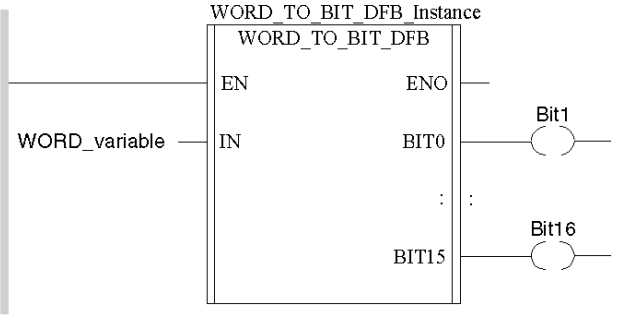
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL WORD_TO_BIT_DFB_Instance (IN:=WORD_variable,
    BIT0=>Bit1, BIT1=>Bit2, BIT2=>Bit3, BIT3=>Bit4,
    BIT4=>Bit5, BIT5=>Bit6, BIT6=>Bit7, BIT7=>Bit8,
    BIT8=>Bit9, BIT9=>Bit10, BIT10=>Bit11, BIT11=>Bit12,
    BIT12=>Bit13, BIT13=>Bit14, BIT14=>Bit15, BIT15=>Bit16)
```

Representation in ST

Representation:

```
WORD_TO_BIT_DFB_Instance (IN:=WORD_variable,
    BIT0=>Bit1, BIT1=>Bit2, BIT2=>Bit3, BIT3=>Bit4,
    BIT4=>Bit5, BIT5=>Bit6, BIT6=>Bit7, BIT7=>Bit8,
    BIT8=>Bit9, BIT9=>Bit10, BIT10=>Bit11, BIT11=>Bit12,
    BIT12=>Bit13, BIT13=>Bit14, BIT14=>Bit15,
    BIT15=>Bit16) ;
```

Parameter description

Description of the input parameters:

Parameter	Data type	Meaning
IN	WORD	Input

Description of the output parameters:

Parameter	Data type	Meaning
BIT0	BOOL	Output BIT0
BIT1	BOOL	Output BIT1
:	:	:
BIT15	BOOL	Output BIT15

Chapter 22
WRITEREG: Write register

Introduction

This chapter describes the `WRITEREG` block.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Description	198
Mode of Functioning	201
Parameter description	202

Description

Function description

With a rising edge at the REQ input, this function block writes a register area from the PLC to an addressed slave via Modbus Plus.

EN and ENO can be configured as additional parameters.

NOTE: When programming a WRITEREG function, you must be familiar with the routing procedures used by your network. Modbus Plus routing path structures are described in detail in the Modbus Plus Network Planning and Installation Guide *(see page 13)*.

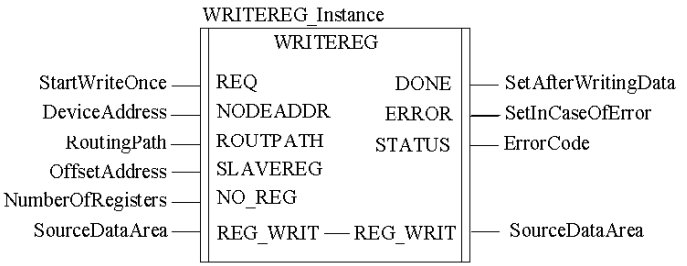
NOTE: This derived function block only supports the local Modbus Plus interface (no NOM). If using a NOM, work with the WRITE_REG block from the communication block library.

NOTE: This derived function block also does not support TCP/IP- or SY/MAX-Ethernet. If TCP/IP- or SY/MAX-Ethernet is needed, use the WRITE_REG block from the communication block library.

NOTE: Several copies of this function block can be used in the program. However, multiple instancing of these copies is not possible.

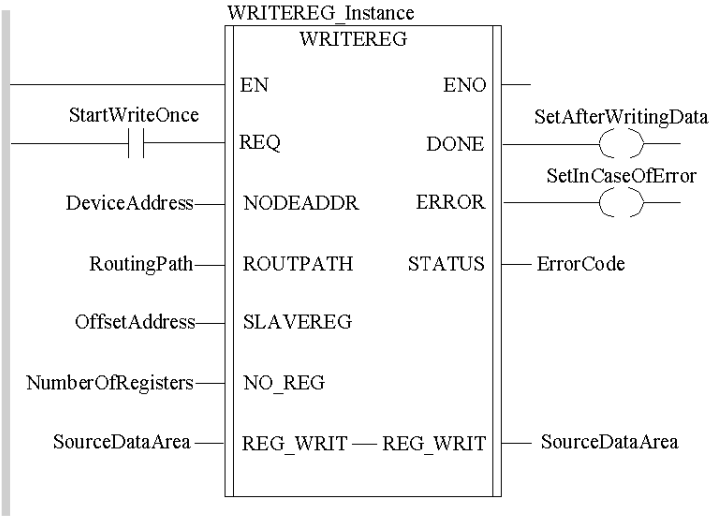
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL WRITEREG_Instance (REQ:=StartWriteOnce,
    NODEADDR:=DeviceAddress, ROUTPATH:=RoutingPath,
    SLAVEREG:=OffsetAddress, NO_REG:=NumberOfRegisters,
    REG_WRIT:=SourceDataArea,
    DONE=>SetAfterWritingData,ERROR=>SetInCaseOfError,
    STATUS=>ErrorCode)
```

Representation in ST

Representation:

```
WRITEREG_Instance (REQ:=StartWriteOnce,
    NODEADDR:=DeviceAddress, ROUTPATH:=RoutingPath,
    SLAVEREG:=OffsetAddress, NO_REG:=NumberOfRegisters,
    REG_WRIT:=SourceDataArea,
    DONE=>SetAfterWritingData,ERROR=>SetInCaseOfError,
    STATUS=>ErrorCode) ;
```

Parameter description

Description of input parameters:

Parameter	Data type	Meaning
REQ	BOOL	With a rising edge at the REQ input, this function block writes a register area from the PLC to an addressed slave via Modbus Plus.
NODEADDR	INT	Device address within the target segment
ROUTPATH	DINT	Routing path to target segment
SLAVEREG	DINT	Offset address of the first 4x register in the slave to be written to
NO_REG	INT	Number of registers to be written from slave

Description of input / output parameters:

Parameters	Data type	Meaning
REG_WRIT	ANY_ARRAY_WORD	Source data field (A data structure must be declared as a located variable for the source file.)

Description of the output parameters:

Parameters	Data type	Meaning
DONE	BOOL	Set to 1 for one scan after writing data
ERROR	BOOL	Set to 1 for one scan in case of error
STATUS	WORD	Error Code

Mode of Functioning

Function mode of WRITEREG blocks

Although a large number of WRITEREG function blocks can be programmed, only four write operations may be active at the same time. It makes no difference whether these operations are performed using this function block or others (e.g. MBP_MSTR, CWRITE_REG). All function blocks use one data transaction path and require multiple cycles to complete a task.

If several WRITEREG function blocks are used within an application, they must at least differ in the values of their NO_REG or REG_WRIT parameters.

The status signals DONE and ERROR report the function block state to the user program.

The complete routing information must be separated into two parts:

- in the NODEADDR of the destination node (regardless of whether it is located in the local segment or in another segment) and
- the routing path, in case there is a link via network bridges.

The resulting destination address consists of these two information components.

The routing path is a DINT data type, which is interpreted as a sequence of two-digit information units. It is not necessary to use "00" extensions (e.g. both routing paths 4711 and 47110000 are valid, for NODEADDR 34 the result is destination address 47.11.34.00.00).

Parameter description

REQ

A rising edge triggers the write transaction.

The parameter can be entered as an address, located variable, unlocated variable or literal.

NODEADDR

Identifies the node address within the target segment.

The parameter can be entered as an address, located variable, unlocated variable or literal.

ROUTPATH

Identifies the routing path to the target segment. The two-digit information units run from 01 64 (see *Mode of Functioning, page 201*). If the slave resides in the local network segment, `ROUTPATH` must be set to "0" or must be left unconnected.

The parameter can be entered as an address, located variable, unlocated variable or literal.

SLAVEREG

Start of the destination area in the addressed slave to which the source data is written. The destination area always resides within the 4x register area. `SLAVEREG` expects the destination address as an offset within the 4x area. The leading "4" must be omitted (e.g. 59 (contents of the variables or value of the literal) = 40059).

The parameter can be entered as an address, located variable, unlocated variable or literal.

NO_REG

Number of registers to be written to slave processor (1 ... 100).

The parameter can be entered as an address, located variable, unlocated variable or literal.

REG_WRIT

An `ANY_ARRAY_WORD` that is the same size as the planned transmission must be agreed upon (≥ `NO_REG`) for this parameter. The name of this array is defined as a parameter. If the array is defined too small, then only the amount of data is transmitted that is present in the array.

The parameter must be defined as a located variable.

DONE

Transition to ON state for one program scan signifies data have been transferred.

The parameter can be entered as an address, located variable or unlocated variable.

ERROR

Transition to ON state for one program cycle signifies detection of a new error.

The parameter can be specified as an address, located variable or unlocated variable.

STATUS

Error code, see *Modbus Plus Error Codes, page 127*

The parameter can be specified as an address, located variable or unlocated variable.

Part III
Import / Export of LL984 Applications

Overview

This part describes the specifics of the LL984 Editor.

For standard features please refer to *Import / Export (see Unity Pro, Operating Modes)*.

What Is in This Part?

This part contains the following chapters:

Chapter	Chapter Name	Page
23	Import	207
24	Export	245

Chapter 23
Import

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Importing an LL984 Application	208
Importing PLCs	216
Importing Specific CPU Configurations	219
Importing Hot Standby Configurations	226
Importing I/O Modules	228
Importing Communication Settings	234
Importing Descriptors and Comments	237
Import Restrictions	239
Importing Application Parts with the Conversion Wizard	243

Importing an LL984 Application

Introduction

When you import an application to Unity Pro from other programming software, some functions may behave differently.

For example:
- Cold start behavior of Unity Pro LL984 applications is different from legacy LL984 applications.
- Imported Concept IEC coils behave differently than imported LL984 coils.
- Unit Pro MATH functions might not behave the same as legacy LL984 (overflow, underflow, etc.).

⚠ WARNING

UNINTENDED EQUIPMENT OPERATION

- After you imported an application program, edit the result carefully to check that all the logic elements have been ported completely and are located correctly in the program.
- Test the application before running it to check that it performs as expected.
- If necessary, edit the program to add logic elements that are missing, remove code that is not necessary, and replace logic elements that behave differently from their intended purpose.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

General Message after Import

After the import check the log file *(see page 215)* for error and warning messages.

You have to **Analyze** and **Build** the project.

Import

To import an LL984 application use the **File → Open** menu command.

Please refer to **Open** dialog *(see Unity Pro, Operating Modes)*.

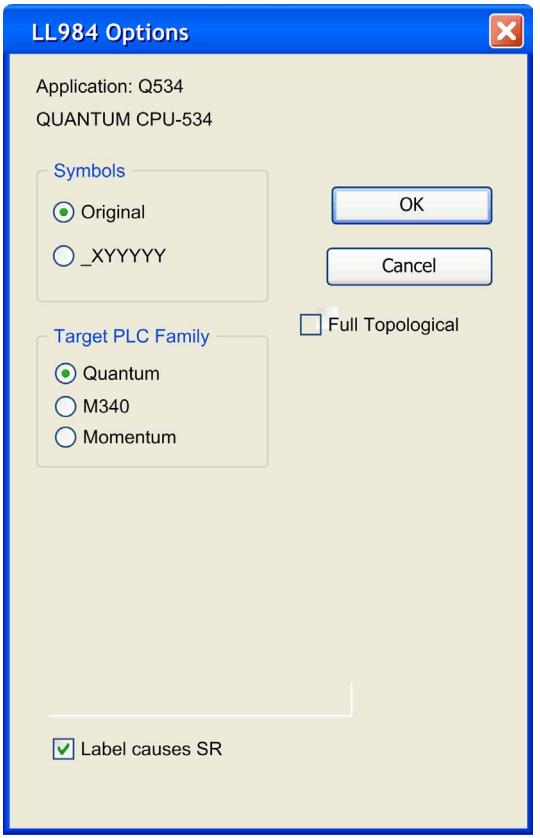
You can import the following legacy LL984 applications:
- Concept Projects (**.ASC*)
- Modsoft Projects (**.CFG*)
- ProWORX32 Databases (**.PWX*)
- ProWORX Nxt (**.DCF*)

NOTE: Imported MSTR functions are not designed to work with the Modicon M340 CPUs. After an application is converted to Unity Pro, it must be manually updated. The MSTR function must be replaced by the READ_VAR and WRITE_VAR functions.

For further information please refer to Import / Export of LL984 Applications *(see page 205)*.

LL984 Options Dialog (-> Target PLC Quantum)

After selecting a legacy LL984 application file, the following **LL984 Options** dialog is displayed. The following graphic shows an example with a legacy Quantum LL984 source application and a Quantum selected as target PLC family.



Element	Description
Application	Displays the name of the legacy application to be imported.
Source PLC Family	Displays the PLC family of the legacy application to be imported.
OK	Starts the import.
Cancel	Displays a message `...`"`Convert" step failed`, no application is imported, and the following message is generated in the import log file: "`Conversion terminated by User`".
Full Topological	This check box is only used to convert applications to Modicon M340 PLCs.

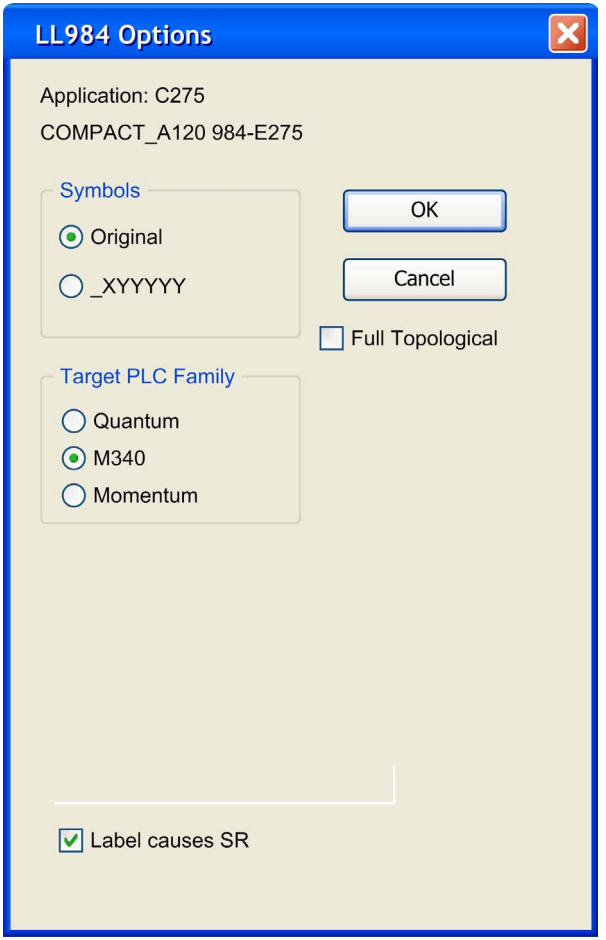
Element	Description
Symbols	
Original	Select this option to use the original variable names of the legacy application in your new Unity Pro application.
_XYYYYY	Select this option to automatically convert the legacy variable names. The new variable names will be composed according to the following list: ● **_**: an underscore will be the leading sign of the converted variables. ● **X**: represents the data type in the memory area. ❍ **0**: %M (memory bit) ❍ **1**: %I (discrete input) ❍ **3**: %IM (input word) ❍ **4**: %MW (memory word) ● **YYYYY**: represents the address number. For example **_400001** stands for%MW1.
Target PLC Family	
Quantum	Select this option to use the Quantum as target PLC.
M340	Select this option to use the Modicon M340 as target PLC. The Modicon M340-specific parameters for full topological addressing are listed in a separate table below.
Momentum	Select this option to use the the Modicon Momentum under Unity Pro as target PLC.
Label causes SR	Select this option to import a segment containing labels as an SR segment, even if its scheduling does not express this. Labels are allowed only in SR segments. For details please refer to Calling Subroutines *(see LL984 Editor, Reference Manual, LL984 Specifics)*.
Shift 984 Remote	This additional option appears if the legacy application CPU family is a 984 family. If selected, a Quantum local rack is added to the application and all drop numbers are increased by one until an unused drop number is reached, keeping the old 984 rack as the first remote rack (drop number 2) without the old CPU. If the option is not selected, conversion attempts to translate the local rack and its modules to Quantum, as far as possible. A common situation encountered is that 984 local drops often contain more racks than allowed with Quantum.
Long Comments are in project and in directory	This additional dialog box displays the path where the long comments are stored if the legacy application contains long comments. This path is identical to the path where the legacy application is stored plus one additional directory for the long comments. Long comments appear twice in the conversion: ● Inside the internal project database of the data editor. ● As text files in the additional application path directory for long comments. To back up all information of the application, only the files in the additional application path directory are to be saved. To achieve consistency between data editor and text files, comment in both places must be edited together.

LL984 Options Dialog (-> Target PLC Modicon M340)

Converting a legacy LL984 application to a Modicon M340 application you can determine the kind of addressing. By default Mixed topological / State RAM addressing is used (option **Full Topological** not selected).

Mixed Topological / State RAM Addressing (-> Modicon M340)

The following graphic shows an example for converting a legacy Compact application to a Modicon M340 application with Mixed topological / State RAM addressing (option **Full Topological** not selected).



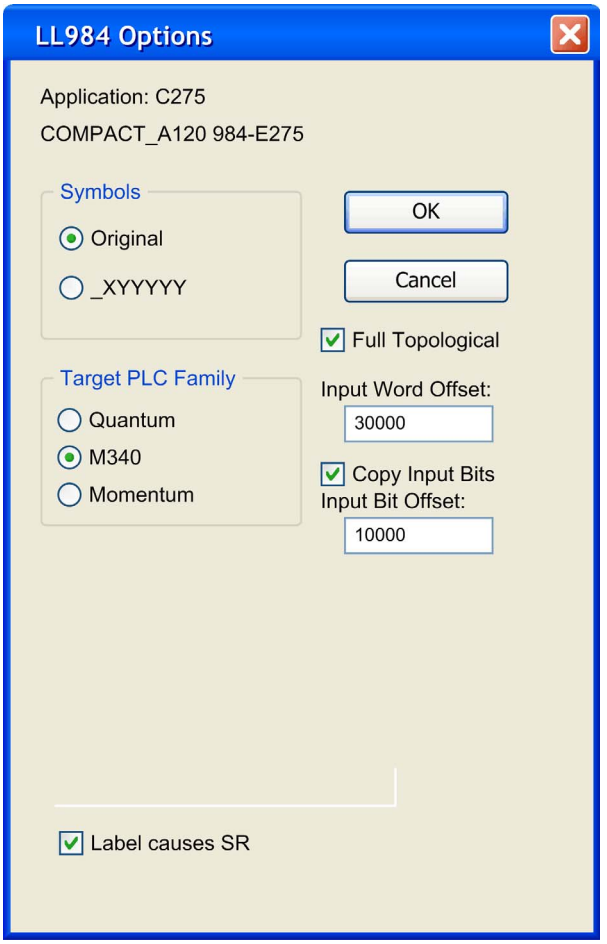
NOTE: If you want to import a legacy LL984 Compact application which uses Modbus request to communicate with an HMI, you have to use State RAM addressing to preserve the Modbus exchange between PLC and HMI.

Full Topological Addressing (-> Modicon M340)

To convert a legacy LL984 application to a Modicon M340 application with full topological addressing, the option **Full Topological** has to be selected.

- All word variables get a declaration with a symbol and the topological address in the data editor, if not already present.
- **Input Words** are copied to a mirror area of memory words, allowing SCADA access.
- Discrete words are directly addressed with topological addresses in the programs, if no symbols are defined for them, showing in this way directly their hardware connection to the controller.
- If SCADA access is required to discrete inputs, the input bits can be copied as well to a mirror area of memory bits by checking **Copy Input Bits**.
- SCADA access for inputs with **Full Topological** checked needs modified addresses in the SCADA system – the offsets in the dialog must be added to the original offsets and the memory areas must be used instead of the input areas.

The following graphic shows an example for converting a legacy Compact application to a Modicon M340 application with full topological addressing.



For converting a legacy LL984 application to a Modicon M340 application with full topological addressing, the following additional parameters are used:

Element	Description
Full Topological	Activate this option to convert input word addresses (3x) of the legacy LL984 application to a separate, mirrored area of the %MW areas of the target Modicon M340 application. If this option is not activated, the legacy LL984 application is converted into a State RAM Modicon M340 application.
Input Word Offset	Enter the offset for the input word addresses (3x) in the mirrored area of the %MW area of the target Modicon M340 application.
Copy Input Bits	Activate this option to additionally convert input bit addresses (1x) to the mirrored area of the %M area of the target Modicon M340 application.
Input Bit Offset	Enter the offset for the input bit addresses (1x) in the mirrored area of the %M area of the target Modicon M340 application.

NOTE: With this **Full Topological** option activated you must manually change the access to the %I and %IW addresses in the SCADA components.

Converting a Legacy Compact Application to Modicon M340 with Full Topological Addressing

During conversion of a legacy Compact application to a Modicon M340 application with full topological addressing, the import automatically creates separate ST code sections for input and output operations with copying commands to transfer I/O data to the mirrored memory areas.

Please refer to *Module Emulation* *(see page 231)*.

Import Behavior

During import of a legacy LL984 application, problems may be detected:

- controller family not supported
- I/O family not supported
- I/O module not supported
- rack count exceeded
- instruction/function block not supported
- non-coil in column 11

Generally such an issue does not stop the import.

Instead a message is displayed for each of these detected problems in the **Import/Export** tab of the **Output Window** at the bottom of the Unity Pro user interface.

Double-click such a message in the **Output Window** to access the respective dialog in your application.

If detected problems occur the following is done during import:

- Not supported controllers or I/O families are replaced by a default Quantum/Modicon M340 configuration.
- Not supported I/O modules are ignored in case of Quantum as target PLC family.
- Exceeding racks and their modules are ignored.
- Not supported instructions are represented by empty DFBs that look identical to the LL984 instruction. In such a case proceed as described under *Not Supported Legacy LL984 Instructions and User Loadables* *(see page 239)*.
- Equation networks with questionable conversions (for example implicit type cast) will contain the imported equation statement and an additional message.
  Equation networks which are not compliant with Modicon M340 variable alignment rules, will cause a build error during **Analyze**.
  For details please refer to Equation Networks *(see page 240)*.
- The import of a legacy application that contains duplicated identifiers is aborted. For details please refer to Duplicated Identifiers *(see page 241)*.

If parts of a legacy application are not imported, after **Analyze** and **Build** of the project an appropriate message is displayed. In this case you have to adapt the application manually. (For example in case of Modicon M340 by selecting **Mixed topological and State RAM** in the *Memory Tab (see Unity Pro, Operating Modes)* of the modules and subsequently fixing the related variables).

Import Log File

After import a message informs you that the conversion and import is finished and reminds you to have a look at the import log file.

The import log file is stored in the root directory of your application.

You have to **Analyze** and **Build** the project.

Importing PLCs

Introduction

During import PLCs used in legacy LL984 applications are replaced by PLCs available in Unity Pro.

In the **LL984 Options** dialog box you can select either Quantum or Modicon M340 as the target PLC family.

For some legacy PLCs, it is only possible to create an default configuration without I/Os in the selected target PLC family. This is indicated in the import log file.

The default configuration
- for Modicon M340 is a local rack with CPU (BMX P34 2020, firmware V2.4) and power supply.
- for Quantum is a local rack with CPU (140 CPU 534 14A) and power supply.

984 PLCs

A 984 controller can either be:
- replaced with a default Modicon M340 configuration without I/Os,
- replaced by a Quantum local rack with a CPU, power supply and remote I/O controller inserted. I/Os having direct equivalents are translated with module specific parameters.

Legacy LL984 PLC	Unity Pro Quantum PLC
984A/S908 984B/S908	140 CPU 534 14A
984A/S901 984B/S901 984X	140 CPU 534 14A, no I/Os
38x48x, 68x78x	140 CPU 534 14A
484Replacement	140 CPU 534 14A, no I/Os

NOTE: The Legacy LL984 PLCs 984A, 984B and 984X do not have local I/Os.
Importing such a legacy PLC having for example Series 800 I/Os as RIO drops 1-32 will insert the first RIO drop as a Quantum local I/O and the following 2-32 RIO drops as 1-31 Quantum RIO drops.
You have to adapt your Unity Pro hardware configuration manually.
As a workaround it is recommended to adapt your legacy LL984 application by changing the PLC to a Quantum PLC (with the appropriate I/Os) before importing the application to Unity Pro.
If the legacy LL984 application which is imported contains a 800 I/O local drop with more than 2 racks (e.g. 4 racks) it will be translated to a local Quantum drop with the appropriate number of racks (e.g. 4 racks).
But as in Unity Pro a local Quantum drop can only have a maximum number of 2 racks, this Quantum drop is invalid. This is indicated by a message in the **Import/Export** tab of the **Output Window**.
As a solution you have to drag-and-drop the modules of rack 3 and 4 to racks 1 and 2 and subsequently delete rack 3 and 4. During drag-and-drop Unity Pro will adapt the logic.

Quantum PLCs

Quantum PLCs used in legacy LL984 applications can either be:

- replaced by a default Modicon M340 configuration without I/Os,
- replaced/upgraded by the next Quantum Unity Pro equivalent:
  - 140 CPU 311 10
  - 140 CPU 434 12A/U
  - 140 CPU 534 14A/U
  - 140 CPU 658 60
  - 140 CPU 671 60
  - 140 CPU 672 60
  - 140 CPU 672 61
  - 140 CPU 678 61

Please also refer to *Unity Pro Hot Standby* *(see page 226)*.

Compact PLCs

If you select the option Modicon **M340** as target PLC, Compact PLCs are replaced by Modicon M340 CPU BMX P34 2020, firmware V2.4.

- For Compact 16 Bit PLCs the board behavior is **not** emulated.
- For Compact 32 Bit PLCs the board behavior is emulated as far as possible.

If you select the option **Quantum** as target PLC, Compact PLCs are replaced by Quantum 140 CPU 534 14A.

If the legacy LL984 application which is imported contains a Compact drop with more than 2 racks (e.g. 4 racks) it will be translated to a local Quantum drop with the appropriate number of racks (e.g. 4 racks).

But as in Unity Pro a local Quantum drop can only have a maximum number of 2 racks, this Quantum drop is invalid. This is indicated by a message in the **Import/Export** tab of the **Output Window**.

As a solution you have to drag-and-drop the modules of rack 3 and 4 to racks 1 and 2 and subsequently delete rack 3 and 4. During drag-and-drop Unity Pro will adapt the logic.

Micro PLCs

Micro PLCs can either be translated with a default Quantum configuration or with a default Modicon M340 configuration.

Momentum PLCs

Momentum PLCs can either be translated with a default Quantum configuration or with a default Modicon M340 configuration.

584 PLCs

Converting a legacy 584 application ends up with NoConf.

ProWORX or Modsoft applications with a Modicon 584 PLC cannot be imported correctly:

- segment order may be changed
- Traffic cop may not be imported correctly
- Port parameter may not be imported correctly

As a workaround, it is recommended to convert the ProWORX 584 application to a ProWORX Quantum application before importing the application to Unity Pro.

584 PLC segment order different

If in the source project there is a 584 PLC which does not support a segment scheduler, after an import the segment order may not be correct. In this case, you must adjust the segment order in the **Project Browser**.

Other PLCs

Other PLCs can either be translated with a default Quantum configuration or with a default Modicon M340 configuration.

Importing Specific CPU Configurations

Introduction

With legacy LL984 applications it was possible to configure some special CPU properties.

During import of a legacy LL984 application the following CPU properties are converted:

- Battery Coil
- Timer Register
- Time of Day Clock

Special Unity Pro ST sections are generated with code that reproduces these properties.

The ST sections generated in your new Unity Pro application are:

- **LL984_OS_Wrapper** in the **MAST** task
  This section provides code to support the **Battery Coil**, **Time of Day Clock** and in parts the **Timer Register** properties.
- **Timer0** in the **Timer Events** task
  This section provides additional code to support the **Timer Register** property.

To provide necessary information before the application gets executed these special sections will be executed before the application sections are executed.

You can modify or remove the code in these sections.

For the Watch Dog Time property see section below *(see page 224)*.

Using System Bits (%SM) and System Words (%SW)

Information concerning the above mentioned properties can be read from system bits (%SM) and system words (%SW).

Please refer to *System Objects (see Unity Pro, System Bits and Words, Reference Manual)*.

You can adapt your application using these system bits and system words and subsequently delete the automatically generated sections.

Battery Coil

If the battery coil is configured in a legacy LL984 application, the PLC operating system fills the addressed coil with the battery status every scan.

This battery status provides the same information as the %S68 bit in a Unity Pro PLC.

Therefore, if a battery coil is configured in the legacy LL984 application, the **LL984_OS_Wrapper** section contains code to copy the %S68 content to the configured battery coil (%M).

The needed code is:

```
%Mx:=%S68;
```

where `x` stands for the number of the configured battery coil in the legacy LL984 application.

Battery Coil with Modicon M340 as Target System

If the battery coil is configured in a legacy LL984 application, but the target system is a Modicon M340, then no code is generated for the battery coil because the Modicon M340 is not equipped with a battery.

A message in the import log file indicates that the Modicon M340 is not equipped with a battery and that the power up behavior may differ from the legacy LL984 application.

Timer Register

The timer register is a 10ms based free running timer.

In the legacy LL984 application the configured timer register gets updated in the system timer interrupt every 10ms.

To get a similar behavior in Unity Pro the **Timer0** event section has the following properties.

Property	Value
Time number	0
Time base	10 ms
Preset	1
Phase	0

Please also refer to *Properties dialog box for Timer Event Sections (see Unity Pro, Operating Modes)*.

The corresponding `ITCNTRL` function block is called in the **LL984_OS_Wrapper** section.

The **LL984_OS_Wrapper** section provides a free running 10 ms counter in the configured timer %MW.

The cyclic application modifies this value and during the next timer execution, the modified value will be incremented.

The **Timer0** event section provides the timer register in Unity Pro.

The needed code in the **LL984_OS_Wrapper** section is:

```
ITCNTRL (ENABLE := true (*BOOL*),
       RESET := false (*BOOL*),
       HOLD := false(*BOOL*),
       EVENT := 0(*BYTE*),
       STATUS => TIMER0_STATUS_WORD(*WORD*),
       VALUE => TIMER0_TIME_REGISTER(*TIME*));
IF(%S21 = true) THEN
%MWx:=0;
END_IF;
```

The needed code in the **Timer0** event section is:

```
%MWx:=%MWx+1;
```

where x stands for the number of the configured timer register in the legacy LL984 application.

Time of Day Clock

In some legacy LL984 PLCs it is possible using the Time of Day registers to read and write data to the Real Time Clock chip.

The legacy LL984 TOD (Time of Day) data have the following (American) format:

Register	Control Register	Description
4XXXX	Discrete 1 (MSB)	1 = set clock values
	Discrete 2	1 = read clock values
	Discrete 3	1 = done flag
	Discrete 4	1 = error flag
4XXXX +1	Day of week (1 - 7)	1 = Sunday
4XXXX +2	Month (1 - 12)	-
4XXXX +3	Day (1 - 31)	-
4XXXX +4	Year (00 - 99)	-
4XXXX +5	Hours (0 - 23)	-
4XXXX +6	Minutes (0 - 59)	-
4XXXX +7	Seconds (0 - 59)	-

In dependence of the required action (read or write) the application has to set a control bit in the first Time of Day register.

In Unity Pro PLCs the Real Time Clock is handled using the system objects %S50 and %SW49 to %SW53.

The Unity Pro system words %SW49 to %SW53 have the following format:

System Word	Description
%SW49	day of the week ● 1 = Monday ● 2 = Tuesday ● 3 = Wednesday ● 4 = Thursday ● 5 = Friday ● 6 = Saturday ● 7 = Sunday
%SW50	Seconds (16#SS00)
%SW51	Hours and Minutes (16#HHMM)

System Word	Description
%SW52	Month and Day (16#MMDD)
%SW53	Year (16#YYYY)

The **LL984_OS_Wrapper** section will emulate the legacy LL984 Time of Day function using the %S50 and %SW49 to %SW53 system objects.

In case a read is required the **LL984_OS_Wrapper** section will read the content from %SW49 to %SW53 convert and write this information to the configured Time of Day word (%MW).

Then the performed read action gets shown in the control word in bit 3.

In case the %S51 shows a Real Time Clock error the bit 4 in the control word will be set.

In case a write to the Real Time Clock is required the values in the Time of Day word are validated.

If the values are not correct, the error flag in the control word will be set.

If case the values are valid, the data gets prepared in an unlocated word array and the FFB `WRTC_DT`, which writes the PLC internal Real Time Clock, is called.

To show a successfully write the Done flag in the Time of Day Control word is set.

The needed code in the **LL984_OS_Wrapper** section is:

```
IF((%MWx AND 16#8000) = 16#8000) THEN
      (* Write the clock with help of the WRTC_DT EF *)
      %S18 :=FALSE;
      (* Validation from the clock parameters *)
      if(   (%MWx+4 > 99)  (* Year > 99 *)
         OR (%MWx+2 > 12)  (* Month > 12  *)
         OR (%MWx+3 >31 )  (*Day  > 31    *)
         OR (%MWx+5 >23 )  (* Hours > 23  *)
         OR (%MWx+6 >59 )  (* Minutes > 59*)
         OR (%MWx+7 >59 ))  (* Seconds > 59*)
      THEN
     (* One or more of parameter contains wrong values  *)
      %MW10 := 16#1000;
      ELSE
         UNITY_LL984_INTARRAY[1] := (*INT*) SHL(INT_TO_BCD(%MW17 ),8);
         UNITY_LL984_INTARRAY[2] := (*INT*) SHL(INT_TO_BCD(%MW15 ),8);
         UNITY_LL984_INTARRAY[2] := UNITY_LL984_INTARRAY[2] OR (*INT*)
\ INT_TO_BCD(%MW16);
         UNITY_LL984_INTARRAY[3] := (*INT*) SHL(INT_TO_BCD(%MW12),8);
         UNITY_LL984_INTARRAY[3] := UNITY_LL984_INTARRAY[3] OR (*INT*)
\ INT_TO_BCD(%MW13);
         UNITY_LL984_INTARRAY[4] := (*INT*) INT_TO_BCD(%MW14+2000);
         UNITY_LL984DT := ARINT_TO_DT(UNITY_LL984_INTARRAY);
         WRTC_DT (UNITY_LL984DT);
         if(%S18 = TRUE)  THEN
```

```
                %MW10 := 16#1000;
          ELSE
            %MW10 := 16#2000;
           END_IF;
       END_IF;
ELSE
(*************** Real Time Clock Read*****************)
IF((%MWx AND 16#4000) = 16#4000) THEN
(* Read the content from SW49..%SW53 and convert the content
    to a LL984 TOD format *)
       IF(%S51 = TRUE) THEN
          %MW10 := 16#1000; (* Show an error because the %SW49
           till %SW50 are not valid yet *)
          ELSE
          %MWx+1 :=%SW49; (* Day of the week content *)
          %MWx+2 :=BCD_TO_INT(SHR(%SW52,8) AND 255);
           (* Month from the %SW52 higbyte *)
          %MWx+3 :=BCD_TO_INT(%SW52 AND 255);
            (*day from the %SW52 Lowbyte  *)
          %MWx+4 :=BCD_TO_INT(%SW53) MOD 100;
             (*Year only from 0 till 99*)
          %MWx+5 :=BCD_TO_INT(SHR(%SW51,8) AND 255);
              (* Hours from the %SW51 higbyte *)
          %MWx+6 :=BCD_TO_INT(%SW51 AND 255);
               (*Minutes from the %SW51 lowbyte *)
          %MWx+7 :=BCD_TO_INT(SHR(%SW50,8));(* Seconds *)
           %MWx :=16#6000; (* Show the application that
                 the read from the time of day clock is done,
                  but let the read command bit true *)
        END_IF;
  END_IF;END_IF;
```

where `x` stands for the first number of the configured Time of Day register in the legacy LL984 application.

NOTE: The **Day of the Week** setting of Time of Day Clock is configurable in legacy LL984applications. In Unity Pro with LL984 this system setting (%SW49) is derived from the defined date and is not settable when using the LL984 configured TOD clock command word **Set Time Bit**. This can be compensated by modifying the ST logic in the **LL984_OS_Wrapper** section.
(For example code `%MWx+1 := (%SW49 + 1) MOD 7;` to get Sunday as 1st weekday, Monday as 2nd, etc.).

Watch Dog Time

The watchdog time in legacy LL984 applications is used to stop the PLC if an expected scan time is exceeded.

Since in Unity Pro the imported LL984 sections will run in the **MAST** task, the **MAST** task watchdog time is the Unity Pro equivalent to the legacy LL984 application´s watchdog time.

During import

- the watchdog time configured in the legacy LL984 application is converted to the **MAST** task watchdog time
  When the configured watchdog time is 0 (which in the legacy LL984 application means 250 ms), the **MAST** task watchdog time is set to 250 ms.
- the **MAST** task is configured as a cyclic task, since this cyclic mode is very similar to the cyclic scan of a legacy LL984 application.

If a watchdog overflow occurs the PLC stops immediately (HALT state).

The system bit %S11 indicates a watchdog overflow. It is set to 1 by the system when the **MAST** task cycle time becomes greater than the watchdog.

The system word %SW11 provides the watchdog time value in ms. This value can not be modified by the program.

The maximum value of the **MAST** task watchdog is 1500 ms.

NOTE: Watch Dog Time settings of imported legacy LL984 applications greater than 150 (150*10 ms =1500 ms), will cause the **MAST** task watch dog time to be set at its maximum limit of 1500 ms.

Please also refer to *Displaying and Modifying Task Properties (see Unity Pro, Operating Modes)*.

I/O Mapped Status Word

NOTE: Compact I/O mapped status word support is not generated during import. In case the Compact application was using the information stored in the I/O mapped status register, you have to modify the application to use the standard Modicon M340 module status information.

NOTE: This section only provides information for you to be considered.

In Compact legacy LL984 systems the I/O mapped status word is given in the status table for all modules and refers to the configured slot in the rack.

For Compact PLCs the first word in the dialog I/O map contains the status word of both the first slots (CPU) at the same time. The 3x-reference for the status information is freely definable. E.g. if the address begins with 300101 the status word at 300103 is for the slot 3 in the primary rack.

Partitioning of 3x register I/O mapped status area (PLC input)

3x	Word	Slot	Backplane
3x	1	CPU slot	1. backplane
3x + 1	2	CPU slot	
3x + 2	3	1. I/O module	
3x + 3	4	2. I/O module	
3x + 4	5	3. I/O module	
3x + 5	6	4. I/O module	2. backplane
...	...	...	
3x + 9	10	8. I/O module	
3x + 10	11	9. I/O module	3. backplane
...	...	...	
3x + 14	15	13. I/O module	
3x + 15	16	14. I/O module	4. backplane
...	...	...	
3x + 19	20	18. I/O module	

Structure of I/O mapped status word

Bit	Meaning
0 - 14	module depended, in most cases not used
15	0 = module is healthy 1 = module is not healthy (reports various errors messages which are viewable on the input status word of the module once identified)

Sections with Emulation Code

After import these sections contain the instructions which are necessary for example to copy the BMXAMO0410 status bits to the DAU204 status word, or to copy the BMXAMI0410 status bits to the ADU206/256 status word.

You can read/write access these sections, but show at the beginning is the remark:

```
(* Do not change arbitrarily *)
```

Please also refer to *Module Emulation* *(see page 231)*.

Importing Hot Standby Configurations

Introduction

In legacy LL984 applications the Quantum Hot Standby extension allows you to set the type of state RAM transfer between 140 CHS 110 00 modules in a redundant controller system.

An initial command register and the non-transfer area can be defined. It is also possible to define fragments from the state RAM which should be transferred in each scan, and some fragments which should transferred with multiple scans.

This extension is only available for Quantum controllers Version 2.x that contain the CHS loadable in the configuration.

Unity Pro Hot Standby

The Unity Pro Hot Standby control system uses the 140 CPU 671 60, 140 CPU 672 60, 140 CPU 672 61 or 140 CPU 678 61 PLC providing a CPU with an integrated coprocessor.

The legacy CPU (1 slot) and the 140 CHS 110 00 extension module (1 slot) are replaced by the new 140 CPU 671 60, 140 CPU 672 60, 140 CPU 672 61, or 140 CPU 678 61 PLC (2 slots).

In a Unity Pro Hot Standby control system the state RAM, except the non-transfer area, becomes transferred each scan.

Therefore the non-transfer area parameter becomes converted to a Unity Pro Hot Standby setting.

The command register and the status register allow the monitoring and controlling of Hot Standby operations.

The status register is the third word in the non-transfer area.

Since Unity Pro supports only one non-transfer area in the %MW area, all configured non-transfer areas in the 0x,1x, 3x references are not imported.

In case the source application uses the reference specific exclusion and additional state RAM transfer features, the application must be adapted to the Unity Pro Hot Standby needs.

In general only the settings made in the Quantum Hot Standby configuration extension will be imported.

For detailed information please refer to the Hot Standby documentation *(see Modicon Quantum, Hot Standby System, User Manual)*.

Hot Standby Settings by Means of Loadables

Hot Standby settings made in legacy LL984 applications by means of the `CHS` loadable, or in older LL984 series by means of the `HSBY` loadable, are not imported and must be manually adjusted in the Hot Standby tab of the CPU module dialog.

The following message is displayed:

```
Hot standby configuration with CHS loadable is not supported. Hot Standby
adaption must be made manually.
```

In this case the application must be adapted to the Unity Hot Standby needs.

- The PLC, automatically inserted during import, must be replaced by the appropriate Hot Standby PLC 140 CPU 671 60, 140 CPU 672 60, 140 CPU 672 61 or 140 CPU 678 61.
- The Hot Standby parameters have to be adjusted manually.
- The `HSBY` or `CHS` DFBs, created during import, must be removed to allow a build of the application.

Importing I/O Modules

Introduction

Legacy LL984 systems associate images of dynamic module data to the State RAM.

This association is expressed in the Traffic Cop dialogs.

Quantum Series

Quantum I/O modules of legacy LL984 applications are replaced by the same Unity Pro Quantum I/O modules, if available.

Non-available modules are substituted by appropriate modules. Please refer to Quantum Modules *(see page 250)*.

Non-supported Quantum modules are ignored during import. A message in the import log file reminds you to replace manually these modules after import.

A legacy LL984 Traffic Cop is converted to Unity Pro for Quantum.

For Quantum systems, Traffic Cop content can be mapped one to one because Unity Pro provides for Quantum similar dialogs.

Special cases:

- Quantum 140 XBE100-00 modules not in the right place
  Depending on the application, after an import two 140 XBE 100 00 modules may be placed in the same rack. Unity Pro displays a message during the **Build**.
  Correct the configuration by putting a 140 XBE 100 00 in seperate racks.
- Empty Quantum DIO rack is not converted to Unity Pro
  If a configured Quantum DIO drop only contains a 140 CRA 2xx module, this DIO drop is not imported. If empty drops are needed, create them in Unity Pro with just a 140 CRA2xx module in each rack.

Modicon Compact / A120 Series I/O (General)

During import the converter tries to find for each Compact I/O module an equivalent in the Modicon M340 or Quantum family (depending on the selected target PLC family). Please refer to Compact Modules *(see page 268)*.

Modicon Compact / A120 Series I/O (Modicon M340 as Target PLC Family)

For any unknown or unsupported Compact module that is not converted, a message is generated in the import log file. The unknown / unsupported modules are replaced by dummy modules. Any such replacement is indicated by a message in the output window during **Analyze**. Double-click a message to navigate to the affected module in the **PLC bus** configuration window, substitute the dummy module and adapt the logic if required.

Some Compact modules are converted to 2 Modicon M340 modules

(For example, DAU208 is converted to two AMO0414 modules using voltage sensors *(see page 268)*). These additional modules are integrated in the rack in consecutive order. The slot numbers of the subsequent modules in the rack are correspondingly increased.

Reduced Modules

When importing Compact I/O modules with Modicon M340 as target PLC family, for compatibility reasons and to avoid address overlapping some modules are configured with a reduced number of I/Os in Unity Pro during import. Please refer to *Reduced Modules (see page 273)*.

Traffic Cop

For each module the traffic cop data are evaluated and traffic cop parameters like State RAM address and type (discrete or register/word), are used as configuration data of the Modicon M340 module.

There are 2 different Compact families, which also have 2 different types of traffic cop.

- 32-Bit CPUs: 984-E258 to 984-E285
- 16-Bit CPUs: 948-0120 to 948-E255

The usage of I/O modules is different between these 2 families:

- With 32 Bit Compact CPUs most analog and expert modules get configured with help of module specific configuration dialogs.
- With 16 Bit Compact CPUs, I/O modules are configured at runtime with user logic.

Emulation of Compact modules (I/O configuration parameters) is limited to projects which contain a 32-Bit Compact CPU.

Module specific channel I/O blocks

Depending on the different Compact module parameter data the converter will only set the Modicon M340 module specific channel I/O blocks which contain amongst others scaling and limit parameters.

When a channel cannot be configured to perform like it has been with the Compact family a `ConvError` block is inserted into a converter generated ST section, informing you that there is an issue to be solved by hardware and/or program logic changes.

In this case the channel is configured with a default configuration (same configuration as when manually inserting the module into the **PLC bus** configuration window.

During **Analyze** a `ConvError` message is displayed in the output window. Double-click the message to navigate to the ST section which contains the `ConvError` block.

In this section the `ConvError` block appears under a topic comment which contains the topological address of the channel.

You can retrieve the configuration of the module by opening the module with the topological address.

You need to configure the module with Modicon M340 possibilities and potentially change the program logic.

After solving the issue you can acknowledge this by deleting the `ConvError` block.

No Compact status word emulation

With exception for DAU204 there will be no Compact status word emulation in the Unity Pro target application.

Therefore you have to check the application and communication, if such status information was used.

No analog current channel value adaptation

There is no analog current channel value adaptation.

Compact has a special behavior. For some analog modules In cases like a range limit or a broken wire was detected, the analog current channel value is set to a pre-defined value. E.g. the ADU205 15 Bit sign value is set to -32768 in case an under range has occurred.

⚠ WARNING
UNINTENDED EQUIPMENT OPERATION
Check if the setting of the analog channel to a predefined value is compatible with the application.
Failure to follow these instructions can result in death, serious injury, or equipment damage.

As in Modicon M340 analog current channel values will not be adapted, you have to check the application concerning the utilization of the analog current channel values.

Module dependent parameters

The generated Modicon M340 module parameter are module dependent and are described in chapter *Special Conversion for Compact I/O Modules* *(see page 275)*.

Specific Configuration Dialogs

If an ST section has been created during import to emulate Compact configuration behavior, the converter creates a specific I/O module configuration with specific configuration dialogs *(see page 231)*.

Please also refer to *Memory Tab (see Unity Pro, Operating Modes)*.

The **Memory Management** area is grayed and the choice is fixed to **Mixed Topological / State RAM**.

In the **State RAM** area **Format** and **Coding** are defined by the import.

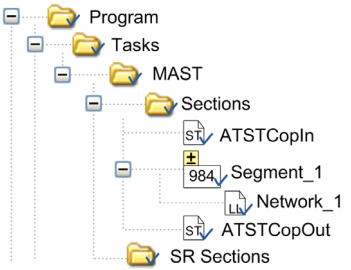
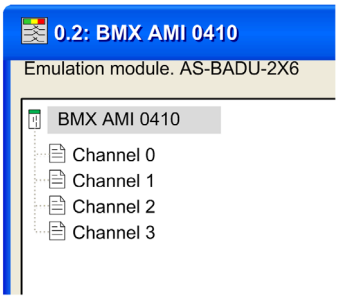
You can only delete these modules. Copy and paste of these modules is not possible.

Module Emulation

When importing a legacy Compact LL984 application into Unity Pro with Modicon M340 as target PLC family and the configuration of the Compact I/O modules cannot be converted directly into a Modicon M340 module configuration, then in some cases ST Sections will be created to emulate Compact configuration behavior.

This paragraph describes the module emulation in case of converting Compact I/O modules to Modicon M340 modules.

The sequence of LL984 segments in the MAST task is introduced by import with an ST section **ATSTCopIn** dedicated for pre-processing of input module data and terminated with an ST section **ATSTCopOut** for post-processing of output module data.

Emulation Segments	Configuration Dialog with Module Emulation Hint
	

In the header of the configuration dialog of a module with pre- or post-processing, the original module name is shown after the introduction **Emulation module**.

For orientation, both ST sections show a list of comments, for each module the topological position, the generated module name and the original module name.

For example: `(* 0.0 BMXP342020 (984-E258): *).`

Following the module comment line, module specific processing is implemented.

For example:

- ADU256, DAU204: construction of the specific module status word
- ConvError notes for module channels which cannot be automatically converted, e.g. due to manual-only configured original modules (e.g. ADU 216) or non-available resolution
- ADU 204-254: thermocouple scaling

If the address conversion option **Full Topological** is selected, copying code for the module data between topological address and mirror areas in `%M` or `%MW` memory is placed into these sections as well.

Modicon Compact / A120 Series I/O (Quantum as Target PLC Family)

Compact I/O modules are replaced by Quantum modules.

Please refer to Compact Modules *(see page 268)*.

Since most Quantum I/O modules provide larger data size than the Compact I/O modules, in many cases an overlap can occur and you may have to adjust the application.

Not supported Compact modules are ignored during import.

In case there is no equivalent I/O module available in the Quantum family the slot will be left empty and a hint will be generated in the log file.

A message in the import log file reminds you to replace these modules after import by yourself.

Special Cases:

EHC10500 parameters may not be correctly set after conversion of a ZAE 20

In this case, after application import you get a message as:

```
{Device (1.4) 140 EHC 105 00}: Parameter <DEFAULT TO RELATIVE SET> out
of range (value (0) not in [1 to 1])
```

To correct this issue you have to delete the module and reconfigure it, with the proper parameters from the legacy application.

Application containing 140MSB10100 or/and 140EIA92100 ends up with NoConf

When a legacy Compact project containing a B MOT201 board is imported, Unity Pro PLC goes in `NoConf` state after download.

During configuration import the B MOT201 is converted to a 140 MSB101-00 Module. This module is not correctly converted. Unity Pro OS detects a wrong parameter size and goes in `NoConf` state.

This issue may also appear using 140 EIA921-00 modules.

As a workaround you have to delete the module from the Unity Pro configuration, create it again and manually enter the parameters from the legacy application.

Series 800

Available direct equivalents for series 800 are replaced by the same module.

Logically equivalent, but electrically different modules are treated commonly.

Please refer to Series 800 Modules *(see page 256)*.

8 Bit module must be mapped on 16 bit addresses

After import of a legacy LL984 application to Unity Pro, in some cases you get the message:

```
S800 8 Bit module must be mapped on a 16(+1) bit boundary
```

In this case you have to map the 8 Bit module on S800 drop to 16 bit addresses. Unity Pro handles the data mapping on 16 bit boundaries.

Invalid Unity Pro Drop Configuration

If an imported legacy LL984 application contains a drop with more than 2 racks of Series 800 I/O modules, it is translated to a local Quantum drop with the same number of racks. This configuration is invalid with Unity Pro as a local Quantum drop can only have a maximum of 2 racks. This is indicated by a message in the **Import/Export** tab of the **Output** window.

As a solution, for a Quantum drop with 3 racks, drag-and-drop the rack 3 modules to racks 1 and 2 and then delete rack 3. During drag-and-drop Unity Pro automatically adapts the logic.

Series 200/500

Series 200/500 I/O modules are generically replaced by two 800 Series I/O modules.

Unity Pro and legacy LL984 systems support these I/O families generically, so logically similar modules are treated commonly.

Please refer to Series 200/500 Modules *(see page 262)*.

Series 200 I/Os are not imported properly

During import of a ProWORX or Modsoft application with a Modicon 984B controller, the Traffic cop may not be imported properly.

The imported project will contain more channels with I/O modules than configured in the source project.

As a workaround it is recommended to replace the PLC of your legacy LL984 application with a Quantum PLC before importing the application to Unity Pro.

SY/MAX

Available direct equivalents for SY/MAX I/O modules are replaced.

Please refer to SY/MAX Modules *(see page 265)*.

Not supported SY/MAX I/O modules are ignored during import.

A message in the import log file reminds you to replace these modules after import by yourself.

Series 300, DCP

Series 300 and DCP I/O families are not supported.

Parameter <NOT USED>

After import of a legacy LL984 application to Unity Pro, in some cases you get the message:

```
Parameter <NOT USED> out of range [value 0] not in [1 to 1]
```

Delete the module configuration and reconfigure it with the correct parameters from the legacy application.

Importing Communication Settings

Introduction

Communication features are offered in legacy LL984 applications as configuration extensions.

Only the following extensions are converted during import:
- TCP/IP communication
- Peer Cop
- I/O Scanner
- SY/MAX Ethernet

TCP/IP Communication

The TCP/IP extension enables the controller to work with a TCP/IP communication card.

Controllers support different numbers of TCP/IP communications cards:
- Quantum 113, Revision 2 and 213 Revision 2 controllers support 2 cards
- Quantum 424, Rev. 2 supports up to 6 cards
- M1E Momentum controllers only support 1 card and the head number is fixed to 1.

During import of a legacy LL984 application the following TCP/IP parameters are converted:
- card number (1-6)
- slot number (1-16)
- IP address (1-255).(1-255).(1-255).(1-255) from the IP Address, Subnet Mask, and Gateway IP fields.
  0.0.0.0 indicates an undefined address.
- Framing Type (Ethernet II or IEEE 802.3)
- IP Address Selection
  - Extension: On power up, PLC reads its TCP/IP addressing information from this extension.
  - BOOTP: On power up, the PLC requires a BOOTP server to supply TCP/IP addressing information.

For further information please refer to *Quantum TCP/IP Configuration*.

Peer Cop

The Peer Cop extension provides data transfer between 2 or more controllers on a peer-to-peer network, as well as linking multiple networks using the S985 communication card.

Peer Cop configures data blocks to be continuously transferred (once per scan) between nodes on a Modbus Plus network.

Data can be broadcast to all nodes on a single link (Global I/O) or between specific Modbus Plus nodes on a link (Specific I/O).

A maximum of 32 data registers or 512 (for example, 32*16) I/O points can be transferred to or read from a controller at a time.

Peer Cop is supported by the by Compact with A145 and the E-Series CPUs, Momentum and Quantum controllers equipped with NOK Modbus+ adapters.

Up to three Links of the Peer Cop can be configured and edited.

During import of a legacy LL984 application the Peer Cop parameters are converted to Unity Pro.

For further information please refer to *Peer Cop Configuration (see Quantum with Unity Pro, Modbus Plus Network Modules, User Manual)*.

I/O Scanner

During import of a legacy LL984 application (with Quantum as target PLC family) the following I/O Scanner parameters are converted:

- Transaction type
  - Direct
    Creates a single transaction in the local device. Data is transferred regardless of the programming of the remote device. This option is simpler than Link Client/Server, as the target device needs no additional program to verify its operation.
  - Link Client/Server
    Creates a pair of matching transactions, one in each device. The server makes a request from a client, which then responds to that request. This is a more complex option than Direct transactions to set up and maintain.
- IP address in the local device IP address field in the standard IP format (1-255).(1-255).(1-255).(1-255).
  For Quantum, select the slot that the NOE (or similar Ethernet adapter) card resides in.
- Target PLCs with which to communicate.
  These target PLCs are called remote devices. At least one remote device must be defined before proceeding - IP address of the PLC for the remote device.
  If using a Quantum PLC, select the slot that the NOE (or similar Ethernet adapter) card resides.
- Transactions
  To transfer data from one device to another a transaction is required.
  Up to 128 transactions may be created, except if you are using the M1E PLCs, then only 64 transactions can be supported.
  - Function (Read/Write)
  - From and To (4xxxx addresses)
  - Length (Number of register fields in the Read from Remote and Write to Remote device area).

For further information please refer to *Configuring the Quantum I/O Scanner (see Quantum with Unity Pro, TCP/IP Configuration, User Manual)*.

NOTE: With Unity Pro it is not allowed to set the parameters **Health Timeout** and/or **Repetitive Rate** to zero. (With legacy LL984 application it was possible to set these parameters to zero).

Device Control Block is not imported for ProWORX projects.

Importing a legacy LL984 application with a configured Ethernet I/O Scanner configuration, the Device Control Block configuration is not imported.

After the import, reenter the Device Control Block address in the Unity Pro Ethernet configuration window.

SY/MAX Ethernet

The SY/MAX configuration extension allows you to properly access and configure up to 6 SY/MAX RIO cards.

This extension is only available when using Quantum controllers, Revision 2 or later.

During import of a legacy LL984 application the following SY/MAX Ethernet parameters are converted:

- card number (1-6)
- Module Drop Number (-1 to 99). Set to -1 if the drop is not defined.
- Backplane Slot (number of the slot (0 to16) that the RIO card inhabits.
  Set to 0 to remove a module.
- Retry Count (1 to 255)
- Timeout (1 ms to 65535 ms)

For further information please refer to *Quantum Ethernet Modules (see Quantum with Unity Pro, Experts and Communication, Reference Manual)*.

NOTE: If the content from SY/MAX configuration extension is not imported to the Unity Pro target application, you have to check and adjust the NOE311/NOE351 configuration parameter in the Unity Pro target application.

Importing Descriptors and Comments

Introduction

During importing an application *(see page 208)* the following elements of a legacy LL984 application are imported:

- Descriptors
- Short comments
- Long comments
- Page titles

Descriptors

The descriptors (1-9) of variables are imported and can be displayed and edited using the **Data Editor** or the **Data Properties** dialog.

Please refer to Configuring the Data Editor Columns *(see LL984 Editor, Reference Manual, LL984 Specifics)*.

NOTE: The symbol descriptors in a Modsoft project are converted into short comments in a Unity LL984 project.

NOTE: Bit-Descriptors from Modsoft or ProWORX are not converted as they do not have an equivalent in a Unity LL984 project.

Short Comments

The short comment of a variable is imported and displayed as the **Comment** property of the variable.

It can be displayed and edited using the **Data Editor** or the **Data Properties** dialog.

The **Comment** can also be displayed in the LL984 Editor. Please refer to Network Display *(see LL984 Editor, Reference Manual, LL984 Specifics)*.

A short comment associated to a network is imported as a network comment.

Long Comments

During import of an application you can select on of the 2 options:

- **1:1 In Project**
- **1:n In Hyperlinked Files**

Please refer to Additional LL984 Options Dialog *(see page 209)*.

If you select

- **1:1 In Project**, the long comment of a variable is imported and displayed as the **Custom** property of the variable.
  A long comment associated to a network is imported as a network comment.

- **1:n In Hyperlinked Files**, the long comments are converted to text files.
  - If there is a already a comment in the variable **Comments** column, this comment will be colored in blue and works as a hyperlink to the text file.
  - If there is no comment in the variable **Comments** column, a relative path colored in blue, is displayed working as a hyperlink to the text file.

  A long comment associated to a network is imported as a hyperlink for the network with **User Name** equal to the network name and **Target Path** equal to the text file, where the long comment is stored.
  This hyperlink can be activated via the **Project Browser**.
  The hyperlinks are available in the variable **Comments** column of the Data Editor and in the **Data Properties** dialog.

The **Comment** can also be displayed in the LL984 Editor. Please refer to Network Display *(see LL984 Editor, Reference Manual, LL984 Specifics)*.

NOTE: Long comments will not be included in upload information.

Hyperlink for a Network Long Comment

In case a hyperlink for a network long comment is not created after import you have to create it manually.

The long comment files are stored at the location which was selected during import.

The name of the long comment files will be *_LCx*, where *x* is the long comment number.

In the hyperlink properties dialog (accessible via the network context menu) it is possible to browse to the selected long comment folder and select the wanted long comment file.

Page Titles

The **Page Title** of a legacy LL984 network will be imported as network name.

The **Page Title** of a legacy LL984 network can be up to 64 characters long.

As in Unity Pro network names are limited to 32 characters, the network names are truncated and a numerical suffix is automatically appended to make it unique.

Blank characters are replaced by underscores.

You can edit the network name using its **Properties** dialog via context menu (right mouse click) or directly in the **Project Browser**.

The unchanged **Page Title** is displayed in the network comment.

Import Restrictions

Macros

ProWORX32, ProWORX Nxt, Modsoft SFC and Modsoft macros are not converted nor supported.

Data Watch Windows

During import, Data Watch Windows used in legacy LL984 applications, are converted to animation tables in Unity Pro.

Please refer to *Forcing Variables via Animation Tables (see LL984 Editor, Reference Manual, LL984 Specifics)* and *Animation Tables (see Unity Pro, Operating Modes)*.

Animation Tables

Whether animation tables are imported or not depends on your legacy LL984 application:

- Concept
  With Concept all animation tables (*.RDE) in the application folder are imported to Unity Pro, but with no connections to the application you are importing.
- Modsoft
  Modsoft animation tables (*.RFD) are not imported to Unity Pro.
  You have to enter them manually.
- ProWORX32
  With ProWORX32 only the last opened animation table, which is stored with the application file, is imported to Unity Pro. External files (*.DWA) are not imported to Unity Pro.
- ProWORX Nxt
  ProWORX Nxt animation tables (*.DWW) are not imported to Unity Pro.
  You have to enter them manually.

Hot Standby

Import supports only Quantum Hot Standby settings made by means of the Hot Standby configuration extension.

For more information please refer to *Importing Hot Standby Configurations* *(see page 226)*.

Not Supported Legacy LL984 Instructions and User Loadables

If the imported LL984 Ladder Logic application contains unsupported LL984 logic instructions or user loadables, the following occurs:

1. The import log identifies the unsupported instruction(s) and/or loadable(s) with a message: `An unsupported instruction or loadable found.`
2. The import places an empty DFB that looks identical to the LL984 instruction in the same location(s) in logic.

3. The empty DFB contains ST logic:
   ```
   {ConvError ('Unsupported instruction or loadable found. Please see the
   import log for more details);};.
   ```
   This code is not executable and causes a notification during **Analyze** of the project.

If empty DFBs are part of the imported application, proceed as follows:

Step	Action
1	Delete or comment out the empty DFB ST to eliminate these notifications during the analyzlis of the application.
2	Create replacement DFB logic using available IEC instructions if the unsupported instruction or user loadable is still required in the application.
3	Contact the local Schneider Electric representative if assistance is necessary.

You can locate the empty DFB's logic instances using one of several methods:
- Use the **Search** icon in the tool bar.
- Double-click the DFB in the **Project Browser**.
- Right-click the DFB in **DFB Types** and select **Initialize search** from the context menu.

For legacy LL984 instructions that are not supported by Unity Pro please refer to *Not Implemented Function Blocks (see Unity Pro, UnityLL984, Block Library)*.

Equation Networks

If the imported LL984 Ladder Logic application contains equation networks with questionable conversions (for example implicit type cast), the following occurs:
- The respective **Equation Network Block** contains the imported equation statement and an additional message:
  ```
  Please check the integrity and correctness of this equation before
  proceeding.
  ```
- This message causes a notification during **Analyze** of the application.
- After verification of the **Equation Network Block´s** correctness, you must delete this message to achieve a successful **Build**.

After import of an equation network, data type suffix of `INT` variables `S` is different from the suffix created by the LL984 Editor.

The generated suffixes are:

Data Type	Import	LL984 Editor
INT S	I	-
DINT L	DI	-
UDINT	UL	UD
REAL	F	R
WORD	W	W
DWORD	DW	DW

Importing Equation Networks to Unity Pro for Modicon M340

Modicon M340 has specific variable alignment rules, which are different from Quantum.

Depending on the legacy LL984 application, the variable type alignment of the imported equation network might not match the Modicon M340 alignment rules if you:

- import a legacy LL984 application containing equation networks
- select Modicon M340 as target platform

In this case, a build error is created during **Analyze** of the application.

Modify the equations to match the Modicon M340 alignment rules, to achieve a successful **Build**.

Modicon M340 alignment:

- `BOOL` and `BYTE` types are aligned on either even or uneven bytes.
- `INT`, `WORD` and `UINT` types are aligned on even bytes.
- `DINT`, `UDINT`, `REAL`, `TIME`, `DATE`, `TOD`, `DT` and `DWORD` types are aligned on double words.

Duplicated Identifiers

If the imported LL984 Ladder Logic application contains duplicated identifiers, i.e for example a network and a segment have the same name, the import is aborted.

The following message is displayed:

```
Object creation failure. Duplicate identifier.
```

You have to open the application with your legacy LL984 software (e.g. Concept, ProWORX32 etc.) and you have to take care for unique identifiers.

After that you can import your legacy application again.

Drop Status Word

In case I/O mapped status words are configured for Quantum or Compact I/O drops, the Concept or ProWORX software prevents a double usage of 3x references from this I/O mapped status area.

The size of the area depends on the highest configured slot number (E.g.: If in slot number 8 a module is configured the usage as input words of the first four 3x references from the module status area is prevented).

In case of Unity Pro with a Quantum PLC, independent of the highest configured slot, 8 %IW words for the primary rack and additional 8 %IW words for a secondary rack are reserved and cannot be used as input references.

Because of this, it is possible that after import an I/O module is mapped to a %IW address which was allowed in the legacy application but which leads to an error message during **Build** of the Unity Pro application.

In this case you must remap the I/O modules or the I/O mapped status area, and adapt the application concerning these changed %IW addresses.

AS-i Proworx application, conversion rules

If the AS-i Proworx application uses Input/Output bits mapping, follows the procedure given in this *FAQ* to convert and adapt the application in Unity Pro.

Importing Application Parts with the Conversion Wizard

Introduction

The conversion wizard is an integrated part of Unity Pro.

You can use it to

- convert applications, exported out of legacy applications (PL7 and Concept) to Unity Pro
- convert legacy applications partially or as a whole
- remap I/O objects (channels, variables etc.) during conversion by means of the wizard
- adapt the hardware configuration of the new application concurrently in Unity Pro
- modify the amount of used memory in the CPU

For detailed information please refer to Conversion Wizard *(see Unity Pro, Operating Modes)*.

Chapter 24
Export

Exporting/Importing LL984 Segments/Networks

Introduction

You can export/import LL984 segments and networks from the **Structural View** of the **Project Browser**.

Restrictions

In contrast to the **Export** dialog for other language sections, the dialog for LL984 segments/networks does not provide the **With SR** check box, because the subroutine networks called in the exported networks are not exported.

This is done because the called subroutine number can be given as a parameter at runtime, so you do not know in advance which subroutines are really called.

If you want to use the called subroutines in another application, you must export and re-import the whole LL984 subroutine segment.

Files Generated

Files with the following extensions are generated during export:

- **.X9S* for LL984 segments
- **.X9N* for LL984 networks

The created files also include the definitions of variables and variable types used inside the exported segments and networks.

Exporting LL984 Segments/Networks

Step	Action
1	Select the element to be exported (LL984 segment/network).
2	Right-click and select **Export** from the context menu or use the **File → Export** menu command.
3	Select the destination directory for the export and enter the file name.
4	Select / deselect the **With DDT**/**With DFB** options.
5	Click the **Export** button.
6	A message in the **Import/Export** tab of the output window informs you when export is completed.

Importing LL984 Networks

To import a **.X9N* file (LL984 network) you have to select an LL984 segment node in the **Structural View** of the **Project Browser**

Right-click and select **Import** from the context menu or use the **File → Import** menu command.

In case of naming conflicts for the network name or the used variables and variable types an **Import Trouble Report** dialog will be displayed. Please refer to Managing Conflicts *(see Unity Pro, Operating Modes)*.

Importing LL984 Segments

To import a **.X9S* file (LL984 segment) you have to select **Sections** or **SR Sections** in the **Structural View** of the **Project Browser**.

Right-click and select **Import** from the context menu or use the **File → Import** menu command.

In case of naming conflicts an **Import Trouble Report** dialog will be displayed.

Importing a **.X9S* file to the **SR Sections** node can end up in the existence of more than one LL984 segment under **SR Sections**.

During **Analyze** this will be checked and will lead to an analyze error.

In this case move the LL984 subroutine networks needed into one LL984 SR segment and delete the other LL984 SR segments.

Even if the **Ladder Logic (LL984)** option under **Tools → Project Settings → Program → Languages** is not set, **.X9S* files can be imported without error messages and the corresponding segments and networks will be created. But in this case later during **Analyze** an `incorrect language` message will be raised for these segments and networks.

In the context menu of segments the menu entry to import networks is disabled in case the **Ladder Logic (LL984)** option is not set.

Part IV
I/O Modules Conversion Table

Overview

This part describes the supported I/O modules and conversion specificities of the supported modules.

What Is in This Part?

This part contains the following chapters:

Chapter	Chapter Name	Page
25	Supported / Not Supported I/O Modules	249
26	Special Conversion for Compact I/O Modules	275

Chapter 25
Supported / Not Supported I/O Modules

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Quantum Modules	250
Series 800 Modules	256
Series 200/500 Modules	262
SY/MAX Modules	265
Compact Modules	268

Quantum Modules

Supported Quantum Modules

Analog input

Legacy Module	Description	Substituted by Unity Pro Module
ACI030-00	Analog 8 Channel Unipolar Input	available in Unity Pro
ACI040-00	16 CH Analog Current Module	available in Unity Pro
AII330-00	I.S. Analog Input	available in Unity Pro
AII330-10	I.S. 8CH Analog Input Current	available in Unity Pro
ARI030-10	8 Channel RTD	available in Unity Pro
ATI030-00	8 Channel Thermocouple	available in Unity Pro
AVI030-00	8 Channel Bipolar, Analog Input	available in Unity Pro

Analog output

Legacy Module	Description	Substituted by Unity Pro Module
ACO020-00	4-20 mA Analog Output Module	available in Unity Pro
ACO130-00	8 Channel Output Module	available in Unity Pro
AIO330-00	I.S. Analog Output	available in Unity Pro
AVO020-00	Analog Voltage Output Module	available in Unity Pro

Digital input

Legacy Module	Description	Substituted by Unity Pro Module
DAI340-00	24 VAC ISO 16 Input Module	available in Unity Pro
DAI353-00	24/48 VAC 32 Input Module	available in Unity Pro
DAI440-00	48 VAC 2x8 Input Module	available in Unity Pro
DAI453-00	48 VAC 32 Input Module	available in Unity Pro
DAI540-00	115 VAC 16 Input Module Isolated	available in Unity Pro
DAI543-00	2x8 115 VAC Input Module	available in Unity Pro
DAI553-00	115 VAC 32 Input Module	available in Unity Pro
DAI740-00	230 VAC 16 Input Module	available in Unity Pro
DAI753-00	230 VAC 4x8 Input Module	available in Unity Pro
DCF077-00	GPS or DTS Time Module	available in Unity Pro
DDI153-10	5 VDC 4x8 Input Module	available in Unity Pro
DDI353-00	24 VDC 32 Input Module	140DDI35310
DDI353-10	24 VDC True Low 32 Input Module	available in Unity Pro

Legacy Module	Description	Substituted by Unity Pro Module
DDI364-00	24 VDC 6x12 Fast Input Module	available in Unity Pro
DDI673-00	125 VDC 24 Point I/P Module	available in Unity Pro
DDI841-00	10-60 VDC 16 Input Module	available in Unity Pro
DDI853-00	10-60 VDC 32 Input Module	available in Unity Pro
DII330-00	I.S. Digital Input	available in Unity Pro
DSI353-00	24 VDC 32 Point Input Module	available in Unity Pro
HLI340-00	Hi-Speed/Latch/Interrupt	available in Unity Pro

Digital output

Legacy Module	Description	Substituted by Unity Pro Module
DAO840-00	24-230 VAC 16 Output	available in Unity Pro
DAO840-10	24-115 VAC 16 Output	available in Unity Pro
DAO842-10	100-230 AC 16 Output Module	available in Unity Pro
DAO842-20	24-48 VAC 16 Output	available in Unity Pro
DAO853-00	24-230 VAC 4x8 Out	available in Unity Pro
DDO153-10	5 VDC 4x8 Output Module	available in Unity Pro
DDO353-00	24 VDC 32 Output Module	available in Unity Pro
DDO353-01	24 VDC 32 Point Output Module	available in Unity Pro
DDO353-10	24 VDC True Low 32 Output Module	available in Unity Pro
DDO364-00	24 VDC True High 96 Output Module	available in Unity Pro
DDO843-00	10-60 VDC 16 Output	available in Unity Pro
DDO885-00	125 VDC 12 Point O/P Module	available in Unity Pro
DIO330-00	I.S. Digital Output	available in Unity Pro
DRA840-00	16 Output Relay	available in Unity Pro
DRC830-00	8 Output ISO Relay	available in Unity Pro
DVO853-00	10-30 VDC Verified Output Module	available in Unity Pro

Generic analog

Legacy Module	Description	Substituted by Unity Pro Module
ACI050-00	32 CH Analog Current In	GENANAIO
ACI051-00	32 CH Analog Voltage/Current	GENANAIO
ACI052-00	32 CH Analog Voltage/Current	GENANAIO
AUI040-00	16 Channel Universal Input Module	GENANAIO
AVI050-00	32 CH Analog Voltage In	GENANAIO

Mixed

Legacy Module	Description	Substituted by Unity Pro Module
AMM090-00	Analog In/Out 4Ch/2Ch	available in Unity Pro
DAM390-00	16/8 Bidir 24 VAC	140DAM59000
DAM490-00	16/8 Bidir 48 VAC	140DAM59000
DAM590-00	16/8 Bidir 120 VAC	available in Unity Pro
DCP	Distributed Control Module	GENNOM
DDM390-00	16/8 Bidir 24 VDC	available in Unity Pro
DDM690-00	125 VDC 4 Input/4 Output HPO Module	available in Unity Pro
EHC105-00	High Speed Counter 5 Channel	available in Unity Pro
EHC202-00	High Speed Counter	available in Unity Pro
EIA921-00	1 Channel AS-I Module Interface	available in Unity Pro
ERT854-10	32 Point Smart Digital Input	available in Unity Pro
ESI062-10	2 Channel ASCII Interface	available in Unity Pro
MSB101-00	Motion Inc Enc	available in Unity Pro
MSC101-00	Motion Enc/Res	available in Unity Pro
NOA611-00	INTERBUS-S Master Module	GENNOM
NOA611-10	INTERBUS-S Master with PCP	GENNOM
NOG111-00	Bitbus Master	GENNOM

Communication

Legacy Module	Description	Substituted by Unity Pro Module
CRA211-10	DIO Drop MB+	available in Unity Pro
CRA211-20	DIO Drop MB+	available in Unity Pro
CRA212-10	DIO Drop MB+	available in Unity Pro
CRA212-20	DIO Drop MB+	available in Unity Pro
CRA931-00	RIO Drop S908	140CRA93X00
CRA932-00	RIO Drop S908	140CRA93X00
CRP811-00	Profibus DP Interface Module	PTQPDPMV1
CRP931-00	RIO Head S908	140CRP93X00
CRP932-00	RIO Head S908	140CRP93X00
NOE211-00	Ethernet TCP/IP Twisted Pair	140NOE77111
NOE251-00	Ethernet TCP/IP Fiber Optic	140NOE77111
NOE311-00	Ethernet SY/MAX Twisted Pair	available in Unity Pro
NOE351-00	Ethernet SY/MAX Fiber Optic	available in Unity Pro

Legacy Module	Description	Substituted by Unity Pro Module
NOE771-00	Ethernet TCP/IP 10/100 Megabit	available in Unity Pro
NOE771-01	Ethernet TCP/IP 10/100 Megabit	available in Unity Pro
NOE771-10	Factorycast Web Server	140NOE77111
NOE771-11	Ethernet TCP/IP 10/100 Megabit	available in Unity Pro
NOM212-10	MB+ Drop Interface Card	140NOM2XX00
NOM2XX-00	MB+ Drop Interface Card	available in Unity Pro
NWM100-00	Factory Cast HMI Web Server	available in Unity Pro
P910	-	J892/P8XX

Controller

Legacy Module	Description	Substituted by Unity Pro Module
140CPU65150	controller	available in Unity Pro
140CPU65160	controller	available in Unity Pro
140CPU65260	controller	available in Unity Pro
140CPU67160	controller	available in Unity Pro
434/534	Controller	140CPU53414A/U
CPU-113-2	Controller	140CPU31110
CPU-113-3	Controller	140CPU31110
CPU-213-4	Controller	140CPU31110
CPU-424-X	Controller	140CPU43412A/U
CPU-434	Controller	140CPU43412A/U
CPU-534	Controller	140CPU53414A/U

Rack

Legacy Module	Description	Substituted by Unity Pro Module
XBP-002	-	available in Unity Pro
XBP-003	-	available in Unity Pro
XBP-004	-	available in Unity Pro
XBP--004	-	available in Unity Pro
XBP-006	-	available in Unity Pro
XBP--006	-	available in Unity Pro
XBP-010	-	available in Unity Pro
XBP--010	-	available in Unity Pro

Legacy Module	Description	Substituted by Unity Pro Module
XBP-016	-	available in Unity Pro
XBP--016	-	available in Unity Pro

Power supply

Legacy Module	Description	Substituted by Unity Pro Module
CPS-111	115/230V AC Power Supply 3A	available in Unity Pro
CPS-114	115/230V AC Power Supply 10A	available in Unity Pro
CPS114-20	115/230V AC Power Supply 10A	available in Unity Pro
CPS-124	115/230V AC Power Supply RED 8A	available in Unity Pro
CPS124-20	115/230V AC Power Supply RED 8A	available in Unity Pro
CPS-211	24 V DC Power Supply 3A	available in Unity Pro
CPS-214	24 V DC Power Supply 8A	available in Unity Pro
CPS-224	24 V DC Power Supply RED 8A	available in Unity Pro
CPS-414	48 V DC Power Supply SUM 8A	available in Unity Pro
CPS-424	48 V DC Power Supply RED 8A	available in Unity Pro
CPS-511	125V DC Power Supply 3A	available in Unity Pro
CPS-524	125V DC Power Supply 8A	available in Unity Pro
CPS524-20	125V DC Power Supply 10A	available in Unity Pro
XBE-100	-	available in Unity Pro

Backplane expander

Legacy Module	Description	Substituted by Unity Pro Module
XBE100-00	Backplane Expander	available in Unity Pro

Not Supported Quantum Modules

Mixed

Legacy Module	Description
AMM090-0X	Analog-4 Ch 4-20mA Output
DEVNET-08	16 Register Devicenet Scanner
DEVNET-32	64 Register Devicenet Scanner
EHC204-00	High Speed Counter 4 Channel
EHC208-00	High Speed Counter 8 Channel
GPS100-00	IRIG-B Time Sync Interface
I2T010-00	I2T 10 Input / 10 Output

Legacy Module	Description
I2T016-00	I2T 16 Input / 16 Output
MPM204-00	4 Channel Meter Proving Module
NOL911-XX	LonWorks Interface
NOP911-00	Profibus FMS Interface Module
QSPXM	Seriplex Master
SERX53-00	Sequence Of Events Recorder

Communication

Legacy Module	Description
NOE511-00	Ethernet MMS Twisted Pair
NOE551-00	Ethernet MMS Fiber Optic

Counter

Legacy Module	Description
MCI186X	MultiTurn Resolver Interface
MCI18X1X2	Resolver Interface Module
MCI18X3X4	Single Turn Interface

Motion

Legacy Module	Description
MMB102-00	Two axis motion with incremental
MMB104-00	Four axis motion with incremental
MMC120-0X	2-Axis Motion Control
MMD102-00	Two axis absolute motion
MMD104-00	Four axis absolute motion

Miscellaneous adapter

Legacy Module	Description
CHS110-00	Hot Standby
HRT100-00	HART Serial Communications Card
MDCCRDL-6	Bus Module, Cradle
QUCM-SE	NRD Universal Comm
XCP900-00	Battery Backup

Series 800 Modules

Supported Series 800 Modules

Analog input

Legacy Module	Description	Substituted by Unity Pro Module
B863-001	4 Channel Register Input	B863
B863-032	4 Channel Register Input	B863
B865-001	8 Channel Register Input	B865
B869-001	8 Channel Register Input	B869
B873	4 Channel Analog Input	B873
B873-001	4-20mA,1-5V 4 Channel Analog Input	B873
B873-002	4-20mA,1-5V 4 Channel Analog Input	B873
B873-011	-10 V to 10 V 4 Channel Analog Input	B873
B873-012	-10 V to 10 V 4 Channel Analog Input	B873
B873-200	V/A, Thermo, RTD, Strain Gauge Input	B873
B875	8 channel. Analog Input	B875
B875-001	4-20mA, 1-5V 8 Channel Analog Input	B875
B875-002	4-20mA, 1-5V 8 Channel Analog Input	B875
B875-011	-10 V to 10 V 8 Channel Analog Input	B875
B875-012	-10 V to 10 V 8 Channel Analog Input	B875
B875-101	Fast selectable 8 chan. Analog Input	B875
B875-102	Fast selectable 8 chan. Analog Input	B875
B875-111	Select. 8 channel Differential Input	B875
B875-200	V/A, Thermo, RTD, Strain Gauge Input	B875
B877-111	Select. 16 channel Single Ended Input	B877

Analog output

Legacy Module	Description	Substituted by Unity Pro Module
B846	Analog MUX	B846
B846-001	Analog MUX	B846
B846-002	Analog MUX	B846
B862-001	4 Channel Register Output	B862
B864-001	8 Channel Register Output	B864
B868-001	8 Channel Register Output	B868
B872	Selectable 4 Channel Voltage Output	B872

Legacy Module	Description	Substituted by Unity Pro Module
B872-002	4-20 mA,1-5V 4 Channel Analog Output	B872
B872-011	Selectable 4 Channel Voltage Output	B872
B872-100	4-20mA 4 Channel Current Output Module	B872
B872-200	Selectable 4 Channel Voltage Output	B872

Digital input

Legacy Module	Description	Substituted by Unity Pro Module
B803-008	115 VAC 8 Point Input Module	B803
B805-016	115 VAC 16 Point Input Module	B805
B807	32 Point Input Module	B807
B807-032	115 VAC 32 Point Input Module	B807
B807-132	115 VAC 32 Point Input Module	B807
B809-016	230 VAC 16 Point Input Module	B807
B817	16 Point Isolated Input Module	B817
B817-116	115 VAC 16 Point Isolated Input Module	B817
B817-216	230 VAC 16 Point Isolated Input Module	B817
B819-032	230 VAC 32 Point Input Module	B819
B819-232	230 VAC 32 Point Input Module	B817
B821	8 Point Input	B821
B821-008	10-60 VDC 8 Point Input	B821
B821-108	10-60 VDC 8 Point Input	B821
B825-016	24 VDC 16 Point Input	B825
B827-032	24 VDC 32 Point Input	B827
B829-116	5V TTL 16 Input	B829
B833-016	24 VDC 16 Point Input	B833
B835	8 Point Input	B835
B837-016	24 VAC/DC 16 Point Input	B837
B849-016	48 VAC/DC 16 Point Input Module	B849
B853-016	115 VAC/125 VDC 16 Input	B853
B855-016	12 VDC 16 Point Input	B855
B863-03w	4 Channel Register Input	B863
B881-001	24 VDC 16 Point Latched Input	B881

Digital output

Legacy Module	Description	Substituted by Unity Pro Module
B802-008	115 VAC 8 Point Output Module	B802
B804	115 VAC 16 Point Output Module	B804
B804-016	115 VAC 16 Point Output Module	B804
B804-116	115 VAC 16 Point Output Module	B804
B806	32 Point Output Module	B806
B806-032	115 VAC 32 Point Output Module	B806
B806-124	24 VAC 32 Point Output Module	B806
B808-016	230 VAC 16 Point Output Module	B808
B810-008	115 VAC 8 Isolated Output Module	B810
B814	8 Point Output Module	B814
B814-001	NO Power Relay 8 Point Output Module	B814
B814-002	NC Power Relay 8 Point Output Module	B814
B814-108	NO/NC Power Relay 8 Point Output Module	B814
B818-032	24 VDC 32 Point Output	B818
B820-008	10-60 VDC 8 Point Output	B820
B824-016	24 VDC 16 Point Output	B824
B826-032	24 VDC 32 Point Output	-
B828-016	5V TTL 16 Point Output	B828
B832-016	24 VDC 16 Point Output	B832
B834	8 Point Output Module	B834
B836-016	12-250 VDC 16 Point Output Module	B836
B838-032	24 VDC 32 Point Output	B838
B840-108	NO/NC Reed Relay 8 Point Output Module	B840
B842-008	NO/NC Reed Relay 8 Point Output Module	B842
B881-108	115 VAC 8 Point Protected Output Module	B881
B882-032	24 VDC Diagnostic Output Module	B882

Mixed

Legacy Module	Description	Substituted by Unity Pro Module
B881-508	125 VDC 8 Point True High Output Module	B881
B882-239	0-30 kHz 2 High Speed Up-Counter Module	B882
B883	10 Point Input Module	B883
B883-001	0-50 kHz 2 High Speed UP/Down Counter	B883
B883-101	4 kHz CAM ABS Encoder Input, 8 Disc Out	available in Unity Pro
B883-111	1 kHz CAM with Velocity compensation	available in Unity Pro
B883-200	10 Thermocouple Input Module	B883
B883-201	8 RTD Input Module	B883
B885-100	Motion Module	B885
B885-110	Motion Module	B885
B886-000	High Speed Logic Solver	B886
B887-000	12 Register Bi-directional	B887
B888-100	Datalogic CM1000 AutoID interface	available in Unity Pro
B984-100	High Speed Logic Solver	B984
B984-102	High Speed Logic Solver	B184
DCP	Distributed Control Module	GENNOM
B819	230 VAC 32 Point Input Module	B819
B863	4 Channel Register Input	B863
B881	8 Point Protected Output Module	B881

Communication

Legacy Module	Description	Substituted by Unity Pro Module
J890-001	RIO Single	P890300
J890-002	RIO Redundant	P890300
J892-001	RIO-ASCII Single	J892/P8XX
J892-002	RIO-ASCII Redundant	J892/P8XX

Rack

Legacy Module	Description	Substituted by Unity Pro Module
H810-100	4 slot secondary	available in Unity Pro
H810-208	4 slot alone	available in Unity Pro
H810-209	4 slot primary	available in Unity Pro
H819-100	7 slot secondary	available in Unity Pro
H819-103	7 slot primary	AS-H819-209
H819-209	7 slot primary	available in Unity Pro
H827-100	11 slot secondary	available in Unity Pro
H827-103	11 slot primary	available in Unity Pro
H827-209	11 slot primary	available in Unity Pro

Power supply

Legacy Module	Description	Substituted by Unity Pro Module
P800-003	Power Supply 115/230 VAC	P890300
P802-001	Power Supply 24VDC	P890300
P810-000	Power Supply 115/230 VAC	P890300
P830-000	Power Supply 115/230 VAC 24VDC	P890300
P840-000	Power Supply 115/230 VAC	P890300
P884-001	Power Supply 115/230 VAC	P890300
P890-000	Power Supply 115/230 VAC 24VDC	P890300
P892-000	Power Supply 115/230 VAC 24VDC	P890300
XBE-100	-	available in Unity Pro

Not Supported Series 800 Modules

Mixed

Legacy Module	Description
B884-002	2 Loop, PID Control Module
B885	Motion Module
B885-002	ASCII/BASIC Module
MMC188-40	Motion Module
S985-XXX	Modbus Plus Interface

Communication

Legacy Module	Description
D908-110	Distributed Control Single
D908-120	Distributed Control Dual
S908-110	RIO Proc Single
S908-120	RIO Proc Dual
S911-800	Hot Standby

Controller

Legacy Module	Description
984-380	Controller
984-381	Controller
984-381E	Controller
984-385	Controller
984-385E/D	Controller
984-480	Controller
984-480E	Controller
984-485	Controller
984-485E	Controller
984-680	Controller
984-685	Controller
984-685E	Controller
984-780	Controller
984-785	Controller
984-785E	Controller
984-785EQ	Controller
984-785L	Controller
B816	-

Series 200/500 Modules

Supported Series 200/500 Modules

Digital input

Legacy Module	Description	Substituted by Unity Pro Module
500-IN	500 Series Input	B805
B231	115 VAC Input	B805
B233	24 VDC Input	B805
B235	230 VAC Input	B805
B237	5 VDC Input for TTL Logic	B805
B243	Analog Input	B805
B245	230 VAC Isolated Input	B805
B247	115 VAC Isolated Input	B805
B256	Analog Input	B805
B258	Analog Input	B805
B271	48 VAC Input	B805
B275	10-60 VDC Input	B805
B551	115 VAC Input	B805
B553	9-56 VDC Universal Input - True High	B805
B555	220 VAC Input	B805
B557	5 VTTL Input	B805
B559	9-56 VDC Universal Input - True Low	B805
B561	9-160 VDC Input	B805
B581	Absolute Encoder	B805
B583	Intrinsically Safe Latched Input	B805

Digital output

Legacy Module	Description	Substituted by Unity Pro Module
500-OUT	500 Series Output	B804
B230	115 VAC Output	B804
B232	24 VDC Output	B804
B234	230 VAC Output	B804
B236	5 VDC Outputs for TTL Logic	B804
B238	24 VDC High Current Output	B804
B239	High Speed Pulse Signals	B804

Legacy Module	Description	Substituted by Unity Pro Module
B244	230 VAC Isolated Output	B804
B246	115 VAC Isolated Output	B804
B248	10-60 VDC Output	B804
B260	Analog Voltage Output	B804
B262	Analog Current Output	B804
B266	Reed Isolated Output	B804
B268	Reed Isolated Output	B804
B270	48 VAC Output	B804
B274	Reed Isolated Output	B804
B276	Reed Isolated Output	B804
B550	115 VAC Output	B804
B552	9-56 VDC Universal Output - True High	B804
B554	220 VAC Output	B804
B556	5VTTL Output	B804
B558	9-56 VDC Universal Output - True Low	B804
B560	9-150 VDC Output	B804
B592	Reed Relay Output - Normally Open	B804
B596	Reed Relay Output - Normally Closed	B804

Mixed

Legacy Module	Description	Substituted by Unity Pro Module
500-IN/OUT	500 Series Mixed	B846
B570	5VDC BCD Multiplexer	B846
B571	Complement of BCD Multiplexer	B846
B572	Analog Output	B846
B573	Analog Input	B846
B574	4-20 mA Analog Output	B846
B575	Stepper Motor Control	B846
B577	Analog Input	B846
B579	Dual High Speed Counter	B846

Not Supported Series 200/500 Modules

Digital input

Legacy Module	Description
200-IN	200 Series Input

Digital output

Legacy Module	Description
200-OUT	200 Series Output

SY/MAX Modules

Supported SY/MAX Modules

Analog input

Legacy Module	Description	Substituted by Unity Pro Module
RIM121	4-Function Analog Input	8030RIM121_125
RIM123	8 Channel High Speed Analog Input	8030RIM123
RIM125	16-Function Analog Input	8030RIM121_125
RIM126	8 Channel Analog/Thermo Input	8030RIM126
RIM127	12 Channel RTD Input Module	8030RIM127
RIM131	High Speed Counter Module	8030RIM131
RIM144	Multiplexed BCD Input Module	8030RIM144

Analog output

Legacy Module	Description	Substituted by Unity Pro Module
ROM121	4-Function Analog Output	8030ROM121
ROM122	4-Function Isolated Output	8030ROM122
ROM131	Stepper Motor Controller Module	8030ROM131
ROM141	Multiplexed BCD Output Module	8030ROM141

Digital input

Legacy Module	Description	Substituted by Unity Pro Module
RDI116	16 Channel Input	8030RIM101_361
RDI132	32 Channel Input	8030RIM331
RDI1XX	Input Module	8030RIM331
RIM101	16-Function 120V AC/DC Input	8030RIM101_361
RIM301	85-140 VAC 16 Input Module	8030RIM301
RIM331	32-Function 24V DC Input	8030RIM331

Legacy Module	Description	Substituted by Unity Pro Module
RIM361	16-Function 240V AC/DC Input	8030RIM101_361
RIM731	64-Function 24V AC/DC Input	8030RIM731

NOTE: Take care, Sy/Max RIM 101 & ROM 221 modules = bit inversion after conversion.

Converting as following:
- Square D --> Quantum Concept
- Square D --> Quantum Unity Pro

In both cases, the bits order is reversed. For Unity Pro case, to create a specific DFB (in order to reorder all bits) can be helpful.

Digital output

Legacy Module	Description	Substituted by Unity Pro Module
RDO616	16 Channel Relay Output	8030ROM271
RDO732	32 Channel Relay Output	8030ROM871
RDOXXX	Relay Output	8030ROM871
ROM221	16-Function 120V AC Output	8030ROM221_431
ROM271	16-Function 120V AC Relay Output	8030ROM271
ROM421	35-140 VAC 16 Output Module	8030ROM421
ROM431	16-Function 240V AC Output	8030ROM221_431
ROM441	32-Function 24V DC Output	8030ROM441
ROM871	64-Function Relay Output	8030ROM871

I/O adapter

Legacy Module	Description	Substituted by Unity Pro Module
CRM931-D1	Digital 2 Slot RIO Adapter	8030CRM931DG1
CRM931-D2	Digital 4 Slot RIO Adapter	8030CRM931DG1
CRM931-D4	Digital 8 Slot RIO Adapter	8030CRM931DG1
CRM931-D8	Digital 16 Slot RIO Adapter	8030CRM931DG1
CRM931-RG	Register RIO Adapter Module	8030CRM931DG1

Rack

Legacy Module	Description	Substituted by Unity Pro Module
HRK200	Rack	8030-RRK-200
RRK100	Rack	8030-RRK-100
RRK200	Rack	8030-RRK-200
RRK300	Rack	8030-RRK-300

Simulator

Legacy Module	Description	Substituted by Unity Pro Module
SIM116	16 In Sim	830_SIM_116

Not Supported SY/MAX Modules

Power supply

Legacy Module	Description
PS25	-
PS35	-

NOTE:

Compact Modules

Overview

Compact modules can either be converted to Unity Pro Modicon M340 or Quantum modules depending on the conversion settings.

To define that Compact modules will be converted to Unity Pro Modicon M340 modules, open the **LL984 Options** dialog box and activate the option **M340** in this dialog box *(see page 209)*.

To convert Compact modules to Unity Pro Quantum modules, activate the option **Quantum**.

Please also refer to *Special Conversion for Compact I/O Modules* *(see page 275)*.

Supported Compact Modules

Analog input

Legacy Compact Module	Description	Substituted by Quantum Module	Substituted by Modicon M340 Module
ADU 204- ADU 254	4 Channel Register Input	GENANAIO	BMXART0414
ADU 205	4 Channel Register Input	140ACI03000	BMXAMI0410
ADU 206	4 Channel Register Input	140ACI03000	BMXAMI0410
ADU 210	4 Channel Voltage/Current Input	GENANAIO	BMXAMI0410
ADU 211	8 Channel Analog Input Module	140ACI03000	BMXART0814
ADU 212	8 Channel Analog Input Module	140ACI03000	BMXART0814
ADU 214	4 Channel Multi Range A/D Input	140ACI03000	BMXAMI0410, BMXART0414
ADU 216	8 Channel Thermocouple	available in Unity Pro	BMXART0814
ADU 256	4 Channel Register Input	140ACI03000	BMXAMI0410
ADU 257	8 Channel Thermocouple	140ACI03000	BMXART0814

Analog output

Legacy Compact Module	Description	Substituted by Quantum Module	Substituted by Modicon M340 Module
DAU 202	2 Channel Register Output	GENANAIO	BMXAMO0210
DAU 204	4 Channel Analog Output, Opto-Isolation	GENANAIO	BMXAMO0410
DAU 208	8 Channel Register Output	140ACO13000	BMXAMO0410, BMXAMO0410

Digital input

Legacy Compact Module	Description	Substituted by Quantum Module	Substituted by Modicon M340 Module
DEO 216	24 VDC 16 Point Input Module	140DDI35300	BMXDDI1602
DEP 208	230 VAC 8 Point Input Module	140DAI74000	BMXDAI0805
DEP 209	120 VAC 8 Point Input Module	140DAI55300	BMXDAI1604 See *Reduced Modules (see page 273)*.
DEP 210	115 VAC 8 Point Input Module	140DAI55300	BMXDAI1604 See *Reduced Modules (see page 273)*.
DEP 211	115 VAC 8 Point Input Module	140DAI55300	BMXDAI1604 See *Reduced Modules (see page 273)*.
DEP 214-254	12-60 VDC 16 Point Input Module	140DDI67300	BMXDDI1603
DEP 215	5 VDC TTL 16 Pt Input Module	140DDI35300	BMXXXXXXX See *Dummy Module (see page 229)*.
DEP 216	24 VDC 16 Point Input Module	140DDI35300	BMXDDI1602
DEP 217	24 VDC 16 Point Input Module	140DDI35300	BMXDDI1602
DEP 218	115 VAC 16 Point Input Module	140DAI54000	BMXDAI1604
DEP 220	Fast 24 VDC 16 Point Input Module	140DDM39000	BMXDDI1602
DEP 256	24 VDC 16 Point Input Module	140DDI35300	BMXDDI1602
DEP 257	110 VDC 16 In	140DAI54000	BMXDAI1604
DEP 296	60 VDC 16 In	140DDI67300	BMXXXXXXX See *Dummy Module (see page 229)*.
DEP 297	48 VDC 16 In	140DDI67300	BMXDDI1603

Digital output

Legacy Compact Module	Description	Substituted by Quantum Module	Substituted by Modicon M340 Module
DAO 216	24 VDC 16 Point Output Module	140DDO35300	BMXDDO1602
DAP 204	24 VDC 4 Point Relay	140DRC83000	BMXDRA0805
DAP 208	24 VDC 8 Point Relay	140DRC83000	BMXDRA0805
DAP 209	120 VAC 8 Point Output Module	140DDO84010	BMXDAO0805 See *Reduced Modules (see page 273)*.
DAP 210	24-230 VAC 8 Point Output Module	140DAO84220	BMXDAO0805 See *Reduced Modules (see page 273)*.
DAP 216	24 VDC 16 Point Output Module	140DDO35300	BMXDDO1602
DAP 217	5-24 VDC 16 Point Output Module	140DRC83000	BMXDDO1612
DAP 218	24-240 VAC 16 Point Output Module	140DAO84000	BMXDAO1605
DAP 258	24 VDC 8 Point Relay	140DRC83000	BMXDRA0805

Mixed

Legacy Compact Module	Description	Substituted by Quantum Module	Substituted by Modicon M340 Module
DAP 211	120 VAC Press and Stamp Module	140DDM39000	BMXXXXXXX See *Dummy Module (see page 229)*.
DAP 212	24 VDC 8 Point Input/4 Point Output	140DDM39000	BMXDDM16022
DAP 220-250	24 VDC 8 Point Input/Output Module	140DDM39000	BMXDDM16022
DAP 252	24 VDC 8In 4Out	140DDM39000	BMXDDM16022
DAP 253	110VDC 8In 4Out	140DDM39000	BMXDDI1604, BMXDRA0804 See *Reduced Modules (see page 273)*.
DAP 292	60 VDC 8In 4Out	140DDM39000	BMXXXXXXX See *Dummy Module (see page 229)*.
FRQ 204	Frequency Module	140EHC10500	BMXXXXXXX See *Dummy Module*.

Legacy Compact Module	Description	Substituted by Quantum Module	Substituted by Modicon M340 Module
FRQ 254	Frequency Module	140EHC10500	BMXXXXXXX See *Dummy Module*.
ZAE 201	High speed Counter/Positioner	140EHC10500	BMXXXXXXX See *Dummy Module*.
ZAE 204	4 Channel High speed Counter/Positioner	140EHC10500	BMXXXXXXX See *Dummy Module* *(see page 229)*.
VRC/CTR2xx	High Speed Counter	-	BMXXXXXXX See *Dummy Module*.
VIC 200	4 High Speed Pulse or 4 VRC Inputs	-	BMXXXXXXX See *Dummy Module*.
VIC 205	4 High Speed Pulse or 4 5V TTL Inputs	-	BMXXXXXXX See *Dummy Module*.
VIC 212	4 High Speed Pulse or 12 VDC Inputs	-	BMXXXXXXX See *Dummy Module*.
VIC 224	4 High Speed Pulse or 24 VDC Inputs	-	BMXXXXXXX See *Dummy Module* *(see page 229)*.

Controller

Legacy Compact Module	Description	Substituted by Quantum Module	Substituted by Modicon M340 Module
984-0120	Controller 16 Bit	140CPU53414A/U	BMXP342020
984-0130	Controller 16 Bit	140CPU53414A/U	BMXP342020
984-0145	Controller 16 Bit	140CPU53414A/U	BMXP342020
984-A120	Controller 16 Bit	140CPU53414A/U	BMXP342020
984-A13x	Controller 16 Bit	140CPU53414A/U	BMXP342020
984-A141	Controller 16 Bit	140CPU53414A/U	BMXP342020
984-A145	Controller 16 Bit	140CPU53414A/U	BMXP342020
984-E241	Controller 16 Bit	140CPU53414A/U	BMXP342020
984-E245	Controller 16 Bit	140CPU53414A/U	BMXP342020
984-E251	Controller 16 Bit	140CPU53414A/U	BMXP342020
984-E255	Controller 16 Bit	140CPU53414A/U	BMXP342020
984-E258	Controller 32 Bit	140CPU53414A/U	BMXP342020
984-E265	Controller 32 Bit	140CPU53414A/U	BMXP342020
984-E275	Controller 32 Bit	140CPU53414A/U	BMXP342020
984-E285	Controller 32 Bit	140CPU53414A/U	BMXP342020

Rack

Legacy Compact Module	Description	Substituted by Quantum Module	Substituted by Modicon M340 Module
HDTA-200	Rack	140XBP00600	BMXXBP0600
HDTA-201	Rack	140XBP00600	BMXXBP0600

Power supply

Legacy Compact Module	Description	Substituted by Quantum Module	Substituted by Modicon M340 Module
P120 000	Power Supply	140CPS11420	BMXXXXXXX See *Dummy Module* *(see page 229)*.
P120 125	Power Supply	140CPS11420	BMXXXXXXX See *Dummy Module*.

Mixed

Legacy Compact Module	Description	Substituted by Quantum Module	Substituted by Modicon M340 Module
CNOE-211	Ethernet Option module	140NOE77110	BMXNOE0100.2

Motion

Legacy Compact Module	Description	Substituted by Quantum Module	Substituted by Modicon M340 Module
MOT 201	Axis Motion Control Module Encoder	140MSC10100	BMXXXXXXX See *Dummy Module* *(see page 229)*.
MOT 202	Axis Motion Control Module Resolver & Encoder	140MSC10100	BMXXXXXXX See *Dummy Module*.

Reduced Modules

When importing Compact I/O modules with Modicon M340 as target PLC family, for compatibility reasons and to avoid address overlapping some modules are configured with a reduced number of I/Os in Unity Pro during import.

NOTE: These reduced modules can not be selected in the **New Device** dialog of the **PLC bus** editor in Unity Pro.

For the reduced modules you must physically insert the following modules into your hardware rack.

Reduced Module	Module to be Physically Inserted
BMXDAI0804	BMXDAI1604
BMXDAO0805	BMXDAO1605
BMXDDI0804	BMXDDI1604

Not Supported Compact Modules

I/O adapter

Legacy Compact Module	Description
BKF 202	INTERBUS S Slave
BKF201-16	16 Word INTERBUS S Master
BKF201-64	64 Word INTERBUS S Master
KOS260-24	24 Word Universal Comm
KOS260-64	64 Word Universal Comm

Chapter 26
Special Conversion for Compact I/O Modules

What Is in This Chapter?

This chapter contains the following sections:

Section	Topic	Page
26.1	Digital Input Modules Conversion	276
26.2	Digital Output Modules Conversion	277
26.3	Digital Mixed Modules Conversion	278
26.4	Analog Input Modules Conversion	279
26.5	Analog Output Modules Conversion	301

Section 26.1
Digital Input Modules Conversion

Digital Input Modules Conversion

Binary/BCD Conversion

For Quantum BCD configuration is possible for digital input modules mapped on %I or %IW.

For Modicon M340 BCD configuration only is possible for digital input modules mapped on %IW.

In case BCD is configured with discrete references you have to adapt your application accordingly.

Section 26.2
Digital Output Modules Conversion

Digital Output Modules Conversion

Binary/BCD Conversion

For Quantum BCD configuration is possible for digital output modules mapped on %M or %MW.

For Modicon M340 BCD configuration only is possible for digital output modules mapped on %MW.

In case BCD is configured with discrete references you have to adapt your application accordingly.

Timeout Behavior Conversion

Since Modicon M340 modules have a configurable PLC Stop behavior, the legacy Compact PLC Stop behavior can be converted to a Modicon M340 module specific PLC Stop behavior.

The legacy setting `Last Value selected` becomes the fallback mode `Maintain`, and in case a `User defined Value` is configured, this value will be converted to the channel specific fallback values, and the fallback mode is set to `Fallback`.

Section 26.3
Digital Mixed Modules Conversion

Digital Mixed Modules Conversion

Binary/BCD Conversion

For Quantum BCD configuration is possible for digital mixed modules mapped on %I or %IW (inputs) respectively on %M or %MW (outputs).

For Modicon M340 BCD configuration only is possible for digital mixed modules mapped on %IW (inputs) respectively on %MW (outputs).

In case BCD is configured with discrete references you have to adapt your application accordingly.

Timeout Behavior Conversion

Since Modicon M340 modules have a configurable PLC Stop behavior, the legacy Compact PLC Stop behavior can be converted to a Modicon M340 module specific PLC Stop behavior.

The legacy setting `Last Value selected` becomes the fallback mode `Maintain`, and in case a `User defined Value` is configured, this value will be converted to the channel specific fallback values, and the fallback mode is set to `Fallback`.

Section 26.4
Analog Input Modules Conversion

What Is in This Section?

This section contains the following topics:

Topic	Page
Analog Input Modules Conversion	280
ADU 206/256	281
ADU 205 Voltage and Current	286
ADU 210 Voltage and Current	290
ADU 204/ADU 254 Thermocouple	291
ADU 214 Mixed Voltage, Current, Thermocouple and Resistors	293
ADU 257 One and Two Channel Thermocouple with 32 bit	297
ADU 211/212, ADU 216 Hardware Switch Configured Modules	298
FRQ xxx, ZAE xxx, MOT_xxx, VIC_xxx, Counting and Positioning Modules	300

Analog Input Modules Conversion

Introduction

Unity Pro generalizes module configuration by using a similar dialog for all analog input modules, where the Concept and ProWORX Compact analog input module dialogs are very different in style and content.

Scale and Overflow Parameters

Scale and **Overflow** parameters are used by the converter for all analog input modules to implement the value ranges and limit checks, as they have been in the Compact system.

The **Range** column must always be filled, otherwise during **Analyze** appropriate messages are generated. The most similar sensor is generated in this case.

Additional parameters are required by the BMXARTxxxx modules. They are also determined by the converter.

Over/Under-Range Values

The Compact module specific pre-defined **Over/Under-Range** values are not supported by the Modicon M340 analog modules.

In case the legacy application uses any of the Compact module specific pre-defined **Over/Under-Range** values (for example for range error detection) this part of the application needs to be reworked manually.

In the following chapters the configuration of the specific Compact analog input modules is described.

Tables list all value ranges provided by the Compact analog module.

ADU 206/256

Modicon M340 Replacement

The legacy Compact ADU 256 module works just like the ADU 206 module, except that the ADU 256 module operates at extended temperature.

Therefore, both modules are substituted by Modicon M340 BMXAMI0410 modules.

Supported and Unsupported Modes

Please refer to the following tables to see how conversion is done and to find the modes for which automatic conversion cannot be provided.

Modes showing scaling values `Out of Range` cannot be converted automatically, because

- the required values exceed the maximum possible ones. In these cases additional emulation code for value conversion is necessary.
  For an example how to create emulation code please refer to the description of the ADU 214 module *(see page 295)*.
- the mode is not the import default mode (please refer to *Conversion Weaknesses* *(see page 283)*). In this case you have to parameterize the BMXAMI0410 channel manually.

The BMXAMI0410 module does not support a 16 bit conversion mode.

In case a legacy ADU 206 module is configured with a 16 bit resolution a message in the import log file is generated.

Also for each channel a message is generated in the import log file, and the BMXAMI0410 channel input/output blocks (IOBs) will contain the default values.

Additionally a `ConvError` block is inserted into a converter generated ST section. Please refer to *Module specific channel I/O blocks* *(see page 229)*.

Unipolar

ADU 206 0..20 mA	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
11 bit	0…2,000	0..20 mA	0 / 2,000
12 bit	0…4,00	0..20 mA	0 / 4,000
15 bit + sign	0…32,000	0..20 mA	0 / 32,000
16 bit	0…64,000	0..20 mA	Out of Range

ADU 206 0..1 V	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
11 bit	0…2,000	0..5 V	Out of Range
12 bit	0…4,000	0..5 V	Out of Range

ADU 206 0..1 V	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
15 bit + sign	0…32,000	0..5 V	Out of Range
16 bit	0…64,000	0..5 V	Out of Range

ADU 206 0..10 V	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
11 bit	0…2,000	0..10 V	0 / 2,000
12 bit	0…4,000	0..10 V	0 / 4,000
15 bit + sign	0…32,000	0..10 V	0 / 32,000
16 bit	0…64,000	0..10 V	Out of Range

Bipolar

ADU 206 +/-20 mA	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
11 bit	-2,000…2,000	+/-20 mA	-2,000…2,000
12 bit	48…4,048	+/-20 mA	48…4,048
15 bit + sign	-32,000…32,000	+/-20 mA	-32,000…32,000
16 bit	768…64,768	+/-20 mA	Out of Range

ADU 206 +/-1 V	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
11 bit	-2,000…2,000	+/-5 V	Out of Range
12 bit	48…4,048	+/-5 V	Out of Range
15 bit + sign	-32,000…32,000	+/-5 V	Out of Range
16 bit	768…64,768	+/-5 V	Out of Range

ADU 206 +/-10 V	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
11 bit	-2,000…2,000	+/-10 V	-2,000…2,000
12 bit	48…4,048	+/-10 V	48…4,048
15 bit + sign	-32,000…32,000	+/-10 V	-32,000…32,000
16 bit	768…64,768	+/-10 V	Out of Range

Broken Wire

ADU 206 4..20 mA	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
11 bit	0…2,000	4..20 mA	0 / 2,000
12 bit	0…4,000	4..20 mA	0 / 4,000
15 bit + sign	0…32,000	4..20 mA	0 / 32,000
16 bit	0…64,000	4..20 mA	Out of Range

ADU 206 0.2..1 V	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
11 bit	0…2,000	0..5 V	Out of Range
12 bit	0…4,000	0..5 V	Out of Range
15 bit + sign	0…32,000	0..5 V	Out of Range
16 bit	0…64,000	0..5 V	Out of Range

ADU 206 2..10 V	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
11 bit	0…2,000	0..10 V	-500 / 2,000
12 bit	0…4,000	0..10 V	-1000 / 4,000
15 bit + sign	0…32,000	0..10 V	-8000 / 32,000
16 bit	0…64,000	0..10 V	Out of Range

Conversion Weaknesses

The ADU206/ADU256 voltage/current input selection is made using jumpers. Therefore, the converter selects a default conversion mode and adapts the channel parameter to this default mode.

The conversion default ranges are:

- 0…20 mA
  In case the ADU206/ADU256 parameter shows +/-1 V,+/-20 mA.
- 0…20 mA
  In case the ADU206/ADU256 parameter shows 0..1 V, 0…20 mA.
- 4…20 mA
  In case the ADU206/ADU256 parameter shows 0,2..1 V, 4…20 mA.

Additional Actions for the User

- The ADU206/ADU256 voltage/current input selection is made using jumpers. Since the converter selects a default mode for the BMXAMI0410 channel mode (please refer to *Supported and Unsupported Modes (see page 281)*), the channel ranges and the channel scaling must be checked. In case a non default range should be used the channel range and the channel parameters must be adapted to the application needs.
- The 2..10 V mode is not directly supported by the BMXAMI0410 module. In case this mode is emulated with the 0..10 V BMXAMI0410 range, negative values may appear when the input voltage is lower than 2 V.
  For this mode no broken wire bit will be set in the emulated status word.
  Therefore, in this case the logic must be adapted to accept negative values, and the broken wire detection must be realized with additional logic.

Created Emulation Code

There is only automatic creation of emulation code for status information (see below).

Supported Status Information

The legacy Compact module specific input status bits, are not provided by Modicon M340 modules directly.

During import ST code is generated, which copies or combines existing BMXAMI0410 status information to the corresponding ADU 206 status area.

These bits are set and reset according to the BMXAMI0410 status information, which may differ from the ADU 206 status behavior.

The BMXAMI0410 module provides no information which is similar to the bit 5 and bit 7 status word meaning.

Therefore, these 2 bits have no equivalent in the Modicon M340 target status word.

The status bits are provided by the Modicon M340 module in the following way:

Bit 1..16	Meaning (Compact) 30xxx Register)	Equivalent at BMXAMI0410	Meaning (Modicon M340)
1	1 = channel 1 current error (overrange or open circuit)	%IWr.m.0.1.5 OR %IWr.m.0.1.6 *	measurement within lower or upper tolerance zone
2	1 = channel 2 current error (overrange or open circuit)	%IWr.m.1.1.5 OR %IWr.m.1.1.6 *	measurement within lower or upper tolerance zone
3	1 = channel 3 current error (overrange or open circuit)	%IWr.m.2.1.5 OR %IWr.m.2.1.6 *	measurement within lower or upper tolerance zone
4	1 = channel 4 current error (overrange or open circuit)	%IWr.m.3.1.5 OR %IWr.m.3.1.6 *	measurement within lower or upper tolerance zone
5	module measures current or voltage in 12 bit unipolar mode	not available with Modicon M340	not available with Modicon M340

Bit 1..16	Meaning (Compact) 30xxx Register)	Equivalent at BMXAMI0410	Meaning (Modicon M340)
6	module measures current or voltage with an offset over the restricted range 4-20 mA, 0.2-1 V, 2-10 V	one or more of bit 1-4 is set	measurement of one channel is within lower or upper tolerance zone
7	external supply voltage to the module is not connected	not available with Modicon M340	not available with Modicon M340
8	module is not ready yet, or one or more of bit 1-4 is set, or processor fault occurs	one or more of bit 1-4 is set or the module error bit is set	measurement of one channel is within lower or upper tolerance zone or a module error is shown
9-16	not used	not used	not used
* r = rack, m = module slot			

To ease the access to this status information, the converter creates emulation code, which transfers the bits to concerned input variable. The created code can be viewed inside the section `ATSTCopIn`.

Furthermore BMXAMI0410 does not provide a common error for overrun, the emulation code combines the channel overrun/underrun errors to simulate this status information. A logical `OR` is used for doing this.

ADU 205 Voltage and Current

Modicon M340 Replacement

The ADU 205 module is substituted by Modicon M340 BMXAMI0410 modules.

Supported and Unsupported Modes

Please refer to the following tables to see how conversion is done and to find the modes for which automatic conversion cannot be provided.

Modes showing scaling values `Out of Range` cannot be converted automatically, because the required values exceed the maximum possible ones. In these cases additional emulation code for value conversion is necessary.

For an example how to create emulation code please refer to the description of the ADU 214 module *(see page 295)*.

The BMXAMI0410 module does not support a 16 bit conversion mode.

In case a legacy ADU 205 module is configured with a 16 bit resolution a message in the import log file is generated.

Also for each channel with an unsupported mode selected a message is generated in the import log file, and the BMXAMI0410 channel input/output blocks (IOBs) will contain the default values.

Additionally a `ConvError` block is inserted into a converter generated ST section. Please refer to *Module specific channel I/O blocks* *(see page 229)*.

ADU 205 **Mode**	**AMI0410** **Sensor**
-20 to +20 V	**Not supported**
-40 to 40 mA	**Not supported**
-10 to +10 V	**+/-10 V**
+/- 20 mA	**+/- 5V +/- 20 mA**
0 to 20 mA	**0..5 V / 0..20 mA**
4 to 20 mA	**1..5 V / 4..20 mA**
0 to 10 V	**0..10 V**
0 to 20 V	**Not supported**

The ADU 205 ranges -20 to +20 V, -40 to 40 mA and 0 to 20 V are not supported by any Modicon M340 module.

Scaling Values

Modicon M340 scaling values for the ADU 205 modes supported for the BMXAMI0410module.

ADU 205 0..20 mA	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
12 bit	0…4,095	0..20 mA	0 / 4,095
12 bit + sign	0…4,095	0..20 mA	0 / 4,095
13 bit	0…8,191	0..20 mA	0 / 8,191
15 bit + sign	0…32,000	0..20 mA	0 / 32,000
16 bit	0…65,520	0..20 mA	Out of Range

ADU 205 0..10 V	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
12 bit	0…4,095	0..10 V	0 / 4,095
12 bit + sign	0…4,095	0..10 V	0 / 4,095
13 bit	0…8,191	0..10 V	0 / 8,191
15 bit + sign	0…32,000	0..10 V	0 / 32,000
16 bit	0…65,520	0..10 V	Out of Range

ADU 205 4..20 mA	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
12 bit	0…4,095	4..20 mA	0 / 4,095
12 bit + sign	0…4,095	4..20 mA	0 / 4,095
13 bit	0…8,191	4..20 mA	0 / 8,191
15 bit + sign	0…32,000	4..20 mA	0 / 32,000
16 bit	0…65,520	4..20 mA	Out of Range

ADU 205 +/- 20 mA	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
12 bit	0…4,095	+/- 20 mA	0 / 4,095
12 bit + sign	-4,095…4,095	+/- 20 mA	-4,095 / 4,095
13 bit	0…8,191	+/- 20 mA	0 / 8,191
15 bit + sign	-32,000…32,000	+/- 20 mA	-32,000 / 32,000
16 bit	0…65,520	+/- 20 mA	Out of Range

ADU 205 +/- 10 v	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
12 bit	0…4,095	+/- 10 V	0 / 4,095
12 bit + sign	-4,095…4,095	+/- 10 V	-4,095 / 4,095
13 bit	0…8,191	+/- 10 V	0 / 8,191
15 bit + sign	-32,000…32,000	+/- 10 V	-32,000 / 32,000
16 bit	0…65,520	+/- 10 V	Out of Range

Conversion Weaknesses

The ADU 205 voltage/current inputs support ranges which are not supported by any Modicon M340 analog module.

In case such an extended range like +/- 20 V, 0-20 V or +/-40 mA was selected an adaptation must be made manually and may be also an adaptation for the sensors must be made.

Additional Actions for the User

Manually created emulation code is required for applications where a 16 bit resolutions was used and for applications where a +/- 20 V, 0-20 V or +/-40 mA range was selected.

For an example how to create emulation code please refer to the description of the ADU 214 module *(see page 295)*.

Created Emulation Code

There is no automatic creation of emulation code for this module during import.

Supported Status Information

There is no automatic emulation of status information for the ADU 205 module during import.

The described Compact module specific input status bits below are not provided by Modicon M340 modules directly. In case the application references to ADU 205 status information, application needs to be adapted manually, and should use the available Modicon M340 module status Information.

Compact ADU 205 module specific I/O mapped status word

Bit	Meaning
0	Channel 1 - exceed/fall below measuring range
1	Channel 2 - exceed/fall below measuring range
2	Channel 3 - exceed/fall below measuring range
3	Channel 4 - exceed/fall below measuring range or open circuit
4	Channel 1 - open circuit
5	Channel 2 - open circuit
6	Channel 3 - open circuit
7	Channel 4 - open circuit
8-14	reserved
15	0 = module is healthy 1 = module is not healthy (multiple error: reports various errors which are viewable on the input status word of the module once identified.)

ADU 210 Voltage and Current

Modicon M340 Replacement

The legacy Compact ADU 210 module is substituted by a Modicon M340 BMXAMI0410 module.

Supported and Unsupported Modes

The conversion is performed by setting special scaling values inside the configuration of the BMXAMI0410 module.

NOTE: A Compact ADU 210 module initially set as inactive is converted to an active BMXAMI0410 module with all channels used and a range set to ± 10 V. To set the BMXAMI0410 module as inactive, manually clear all channels **Used** check boxes after import.

For ADU 210 conversion this mechanism is sufficient to support every mode of ADU 210 module.

ADU 210 Mode	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling Values 0% / 100%
Inactive	Inactive	± 10 V	Default
0…10 V	0…32,000	0…10 V	0 / 32,000
0…5 V	0…32,000	0…5 V	0 / 32,000
2…10 V	0…32,000	0…10 V	-8,000 / 32,000
1…5 V	0…32,000	1…5 V	0 / 32,000
± 5 V	-32,000…32,000	± 5 V	-32,000 / 32,000
± 10 V	-32,000…32,000	± 10 V	-32,000 / 32,000

Additional Actions for the User

Limit checks are different for the BMXAMI0410 module.

Therefore, additional user actions are required in case `limit <> 0` was activated for ADU 210 channel.

Created Emulation Code

There is no automatic creation of emulation code for the ADU 210 module during import.

Supported Status Information

There is no emulation of status information for the ADU 210 module during import.

ADU 204/ADU 254 Thermocouple

Modicon M340 Replacement

The legacy Compact ADU 204/ADU 254 is substituted by a Modicon M340 BMXART0414.

Supported and Unsupported Modes

With the exception of the 13 bit and 15 bit+sign resolutions in combination with RTD sensors, the ADU 204/ADU 254 modes are supported by automatic conversion.

ADU 204/ADU 254 RTD Sensor Mode	Temperature Range	Adequate BMXART0414 Mode	Temperature Range
Pt 100, -200 to 850 °C	-200...850 °C	Pt 100	-175…825 °C
Pt 200, -200 to 250 °C	-200…250 °C	Pt 100	-175…825 °C
Ni 100, -60 to 250 °C	-60...250 °C	Ni 100	-54…174 °C
Ni 200, -60 to 150 °C	-60...150 °C	Ni 100	-54…174 °C
APt 100, -200 to 600 °C	-200...600 °C	JPt 100	-87…437 °C
APt 200, -200 to 250 °C	-200...250 °C	JPt 100	-87…437 °C

If after conversion the resulting temperature ranges do not fit the process requirements, additional adaptations are necessary. Please refer to *Additional Actions for the User* *(see page 291)*.

In opposite to the RTD modes, whereupon the delivered values are in fixed units of 1/10 °C or 1/10 °F for the resistance measurement mode the converter inserts special scaling values in the configuration to adapt to the ADU 204/ADU 254 ranges.

ADU 204/ADU 254 Resistance Mode	Origin Value Range	Adequate BMXART0414 Mode	Scaling Values 0% / 100%
0...400 Ω, 13 bit	0…8,191	0...400 Ω	0 / 8,191
0...400 Ω, 15 bit+sign	0…32,000	0...400 Ω	0 / 32,000

Additional Actions for the User

For RTD sensor modes the resolutions 13 bit and 15bit+sign are not supported by automatic conversion. Manual created emulation code is required for these operation modes.

Furthermore it is possible that the temperature range of the resulting RTD does not comply with the process requirements. In this case the proposed mode and RTD sensor must be replaced by other.

The following table shows the RTD sensors supported by the BMXART0414 module and the provided temperature ranges.

RTD Type	Supported Temperature Range
Pt100 IEC 751-1995, JIS C1604-1997 (2/4 wires)	-175…825 °C (-283…1517 °F)
Pt1000 IEC 751-1995, JIS C1604-1997 (2/4 wires)	-175…825 °C (-283…1517 °F)
Ni100 DIN43760-1987 (2/4 wires)	-54…174 °C (-66…346 °F)
Ni1000 DIN43760-1987 (2/4 wires)	-54…174 °C (-66…346 °F)
Pt100 IEC 751-1995, JIS C1604-1997 (3 wires)	-175…825 °C (-283…1517 °F)
Pt1000 IEC 751-1995, JIS C1604-1997 (3 wires)	-175…825 °C (-283…1517 °F)
Ni100 DIN43760-1987 (3 wires)	-54…174 °C (-66…346 °F)
Ni1000 DIN43760-1987 (3 wires)	-54…174 °C (-66…346 °F)
JPt100 JIS C1604-1981, JIS C1606-1989 (2/4 wires)	-87…437 °C (-124…818 °F)
JPt1000 JIS C1604-1981, JIS C1606-1989 (2/4 wires)	-87…437 °C (-124…818 °F)
JPt100 JIS C1604-1981, JIS C1606-1989 (3 wires)	-87…437 °C (-124…818 °F)
JPt1000 JIS C1604-1981, JIS C1606-1989 (3 wires)	-87…437 °C (-124…818 °F)
Cu10 (2/4 wires)	-91…251 °C (-132…484 °F)
Cu10 (3 wires)	-91…251 °C (-132…484 °F)

Created Emulation Code

The converter creates emulation code for the temperature modes 1.0° Celsius and 1.0° Fahrenheit, which divides the 1/10° units of BMXART0414 by 10.

The created ST code for this purpose looks like this example:

```
%IW177 := WRITE_INPUT_INT(REAL_TO_INT(INT_TO_REAL(%IW177) / 10.0));
```

The automatically created emulation code is stored in the ST section `ATSTCopIn`.

Supported Status Information

There is no emulation of status information for the ADU 204/ADU 254 module during import.

ADU 214 Mixed Voltage, Current, Thermocouple and Resistors

Modicon M340 Replacement

Legacy Compact ADU 214 modules can be configured to Mixed Voltage, Current, Thermocouple and Resistor inputs on a per channel basis. There is no equivalent module in the Modicon M340 range and therefore more than one module is needed to substitute a ADU 214 module.

Inside the configuration the converter places BMXAMI0410 and BMXART0414 module(s) for ADU 214 substitution, by applying the following rules:

First Found Sensor is	Replacement
ADU 214 Unipolar Voltage	one BMXAMI0410
ADU 214 Bipolar Voltage	one BMXAMI0410
ADU 214 Thermocouple	one BMXART0414
ADU 214 Resistor	one BMXART0414

A second module is placed if more than 4 channels are needed, either one BMXAMI0410 or one BMXART0414.

Supported and Unsupported Modes

Because conversion of the different ADU 214 modes is done by using the scaling feature of BMXAMI0410 and BMXART0414, only a few conversions can be provided automatically. Please refer to the following tables to get an overview.

Modes showing scaling values `Out of Range` cannot be converted automatically. In these cases additional emulation code for value conversion is necessary.

Please refer to Additional Actions for the User *(see page 295)*.

ADU 214 Unipolar Voltage	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
0...0.5 V	0…32,000	0...5 V	Out of Range
0…1 V	0…32,000	0…5 V	Out of Range
0…5 V	0…32,000	0…5 V	0 / 32,000
0…10 V	0…32,000	0…10 V	0 / 32,000
0.1…0.5 V	0…32,000	0…5 V	Out of Range
0.2…1 V	0…32,000	0…5 V	Out of Range
1…5 V	0…32,000	1…5 V	0 / 32,000
2...10 V	0…32,000	0…10 V	-8,000 / 32,000

ADU 214 Bipolar Voltage	Origin Value Range	Adequate BMXAMI0410 Mode	Scaling values 0% / 100%
± 0.5 V	-32,000…32,000	± 5 V	Out of Range
± 1 V	-32,000…32,000	± 5 V	Out of Range
± 5 V	-32,000…32,000	± 5 V	-32,000 / 32,000
± 10 V	-32,000…32,000	± 10 V	-32,000 / 32,000

ADU 214 Thermocouple	Origin Value Range	Adequate BMXART0414 Mode	Scaling values 0% / 100%
100 Ω -60…+160 °C Ni	0…32,000	100 Ω -54…+174 °C Ni	Out of Range
200 Ω -60…+160 °C Ni	0…32,000	none	Out of Range
500 Ω -60…+160 °C Ni	0…32,000	none	Out of Range
1 kΩ -60…+160 °C Ni	0…32,000	1 kΩ -54…+174 °C Ni	Out of Range
100 Ω -160…+160 °C Pt	0…32,000	100 Ω -175…+825 °C Pt	Out of Range
200 Ω -160…+160 °C Pt	0…32,000	none	Out of Range
500 Ω -160…+160 °C Pt	0…32,000	none	Out of Range
1 kΩ -160…+160 °C Pt	0…32,000	1 kΩ -175…+825 °C Pt	Out of Range
100 Ω -200…+320 °C Pt	0…32,000	100 Ω -175…+825 °C Pt	Out of Range
200 Ω -200…+320 °C Pt	0…32,000	none	Out of Range
500 Ω -200…+320 °C Pt	0…32,000	none	Out of Range
1 kΩ -200…+320 °C Pt	0…32,000	1 kΩ -175…+825 °C Pt	Out of Range
100 Ω -200…+640 °C Pt	0…32,000	100 Ω -175…+825 °C Pt	Out of Range
200 Ω -200…+640 °C Pt	0…32,000	none	Out of Range

ADU 214 Thermocouple	Origin Value Range	Adequate BMXART0414 Mode	Scaling values 0% / 100%
500 Ω -200…+640 °C Pt	0…32,000	none	Out of Range
1 kΩ -200…+640 °C Pt	0…32,000	1 kΩ -175…+825 °C Pt	Out of Range

ADU 214 Resistor	Origin Value Range	Adequate BMXART0414 Mode	Scaling values 0% / 100%
0...100 Ω	0…32,000	0...400 Ω	Out of Range
0...200 Ω	0…32,000	0...400 Ω	Out of Range
0...500 Ω	0…32,000	0...4 kΩ	Out of Range
0...1 kΩ	0…32,000	0...4 kΩ	Out of Range
0...2 kΩ	0…32,000	0...4 kΩ	Out of Range

Additional Actions for the User

Because of the huge value range of the ADU 214 module most of the resulting scaling values are exceeding the maximum possible range. Therefore additional emulation code is needed to convert and emulate the origin range of values.

The ST Section `ATSTCopIn` has been already created to admit additional emulation code. Please also refer to *Module Emulation* *(see page 231)*.

Of course also other types of sections (like FBD, LD etc.) can be used for this purpose.

Example conversion for Thermocouple -60…+160 Range:

Within this temperature range the ADU 214 module delivers values between 0…32,000 as it is noted in the table above. In the opposite the BMXART0414 module delivers always in units of 1/10 °C and therefore results in values between -600…1600 for the same temperature range.

Compact Module Low	Compact Module High	Modicon M340 Module Low	Modicon M340 Module High
0	32,000	-600	1600

Now you have to perform a linear conversion of these values from one range into the other. One possibility to do that is the usage of the common equation for a line:

Y = m * X + b

Whereas **m** is the gradient and **b** is the intersection with the ordinate of that searched line.

The gradient **m** can be calculated by:

m = (Compact high - Compact low) / (M340 high - M340 low)

In the example the gradient **m** results into **14.545**.

Now you can calculate the intersection point **b** using the formula:

b = Compact high - m * M340 high

And this results into **8,727.27** for **b**.

With both values it is now possible to build your ST code for doing the conversion. Remember this is based on the equation for a line **Y = m * X + b** from the beginning, which looks adapted to your purpose like this:

<Compact value> = m * <M340 value> + b

and expressed in structured text (ST) like this:

%IW999 := WRITE_INPUT_INT(REAL_TO_INT(**m** * INT_TO_REAL(%IW999) + **b**));

and finally with inserted values like this:

%IW999 := WRITE_INPUT_INT(REAL_TO_INT(**14.545** * INT_TO_REAL(%IW999) + **8727.27**));

After this ST code has been added to the application than the received values will be the same as they were delivered by the ADU 214 module within the temperature range:

-60…160 °C.

Creation of emulation code for other modes can be done in the same way.

Created Emulation Code

There is no automatic creation of emulation code for the ADU 214 module during import.

Supported Status Information

There is no emulation of status information for the ADU 214 module during import.

ADU 257 One and Two Channel Thermocouple with 32 bit

Modicon M340 Replacement

The legacy Compact ADU 257 module is substituted by a Modicon M340 BMXART0814 module.

The BMXART0814 module is equipped with a 16 bit converter for each input and therefore every ADU 257 resolution can be reached at least.

Supported and Unsupported Modes

For this module the way of conversion differs from others, because no scaling value mechanism has been used, rather the converter creates emulation code to convert the BMXART0814 values into ADU 257values.

Due to this principle almost every mode of the ADU 257 module is supported. Only the IEEE754 floating point resolutions must be adapted manually.

Additional Actions for the User

Manually created emulation code is required for channels with IEEE754 floating point resolution.

Created Emulation Code

The converter creates emulation code for almost every ADU 257 mode and every resolution. Only IEEE754 floating point resolution is not supported.

Automatically created emulation code is stored in the ST section `ATSTCopIn`.

The basic principle of conversion is the equation for a line `(Y = m * X + b)`. For an example how to create emulation code please refer to the description of the ADU 214 *(see page 295)* module.

Example for created ST code:

```
%IW130 := WRITE_INPUT_INT(REAL_TO_INT(0.216667 * INT_TO_REAL(%IW130) +
710.666626));
```

Supported Status Information

There is no emulation of status information for the ADU 257 module during import.

ADU 211/212, ADU 216 Hardware Switch Configured Modules

Modicon M340 Replacement

The legacy Compact modules ADU 211/212 and ADU 216 are substituted in the following way:

Legacy Compact Module	Description	Substituted by Modicon M340 Module
ADU 211	8 Channel Analog Input Module	BMXART0814
ADU 212	8 Channel Analog Input Module	BMXART0814
ADU 216	8 Channel Thermocouple	BMXART0814

They are converted with Unity Pro default configuration - as if just entered by Unity Pro configuration.

The import log file gives a hint to correct the default module choice.

Supported and Unsupported Modes

Since this legacy analog module is configured during runtime by the logic, it is not possible to adjust the BMXART0814 channel settings during import.

Conversion Weaknesses

For the converter there is no information available, which could be used to adjust the BMXART0814 parameters.

Additional Actions for the User

- The BMXART0814 configuration must be adapted to the sensor and application needs.
- Since the application contains logic to configure the Compact module during runtime, this configuration logic must be removed.
- With the legacy ADU211/212/216 modules it was necessary to demultiplex the input values with help of logic. This logic must be removed.
- In case the legacy Compact application used a data format or data type which is not supported by the BMXART0814 module, the logic must be adapted.
- In case the legacy Compact application used status information of these modules, you have to adapt the logic, and status information, accessible in the topological area of the BMXART0814 module, has to be used.
- In case on the ADU211/212 dip switch settings `Voltage Conversion` is selected, and the input `Voltage Range` is greater than +/-1.28 V the BMXART0814 must be replaced by two BMXAMI0410 modules.
  In case only for one group (4 channels) `Voltage Conversion` is selected, and the input `Voltage Range` is greater than +/-1.28 V, the BMXART0814 module must be replaced by one BMXART0414 and one BMXAMI0410 module.

Created Emulation Code

There is no automatic creation of emulation code for these modules during import.

Supported Status Information

There is no emulation of status information or I/O mapped status word information for these modules during import.

FRQ xxx, ZAE xxx, MOT_xxx, VIC_xxx, Counting and Positioning Modules

Modicon M340 Replacement

The legacy counting and positioning modules are substituted by dummy modules (BMXXXXXXX).

There is no configuration conversion nor parameter conversion.

You have to modify the configuration and logic of your application regarding the counting and positioning modules.

Section 26.5
Analog Output Modules Conversion

What Is in This Section?

This section contains the following topics:

Topic	Page
Analog Output Modules Conversion	302
DAU 204	303
DAU 208	306
DAU 2x2	307

Analog Output Modules Conversion

Introduction

Analog output modules provide **Fallback Values**. The **Stop** behavior of Compact modules can be taken over in an identical manner.

User Defined Values

If **User Defined** values are set for Compact modules, these values are moved to Modicon M340 modules and used for **Fallback Values**. In such cases you have to activate the assigned **Fallback** checkbox in the Modicon M340 target module.

Affected modules are DAU 204, DAU 208 and DAU 2x2.

DAU 204

Modicon M340 Replacement

The legacy Compact DAU 204 module is substituted by a Modicon M340 BMXAMO0410 module.

Supported and Unsupported Modes

Please refer to the following tables to see how conversion is done and to find the modes for which automatic conversion cannot be provided.

Modes showing scaling values `Out of Range` cannot be converted automatically, because the required values exceed the maximum possible ones. In these cases additional emulation code for value conversion is necessary.

For an example how to create emulation code please refer to the description of the ADU 214 module *(see page 295)*.

DAU 204 0..20 mA, 4..20 mA	Origin Value Range	Adequate BMXAMO0410 Mode	Scaling Values 0% / 100%
11 bit	0…2,047	0..20 mA, 4..20m A	0 / 2,048
12 bit	0…4,095	0..20 mA, 4..20m A	0 / 4,096
15 bit + sign	0…32,000	0..20 mA, 4..20m A	0 / 32,000
16 bit	0…65,520	0..20 mA, 4..20m A	Out of Range

DAU 204 ±1 V	Origin Value Range	Adequate BMXAMO0410 Mode	Scaling Values 0% / 100%
11 bit	0…2,047	±10 V	−9,216 / 11,264
12 bit	0…4,095	±10 V	−18,432 / 22,528
15 bit + sign	-32,000…32,000	±10 V	Out of Range
16 bit	0…65,520	±10 V	Out of Range

DAU 204 ±5 V	Origin Value Range	Adequate BMXAMO0410 Mode	Scaling Values 0% / 100%
11 bit	0…2,047	±10 V	-1,024 / 3,072
12 bit	0…4,095	±10 V	-2,048 / 6,144
15 bit + sign	-32,000…32,000	±10 V	Out of Range
16 bit	0…65,520	±10 V	Out of Range

DAU 204 ±10 V	Origin Value Range	Adequate BMXAMO0410 Mode	Scaling Values 0% / 100%
11 bit	0…2,047	±10 V	0 / 2,048
12 bit	0…4,095	±10 V	0 / 4,096
15 bit + sign	-32,000…32,000	±10 V	-32,000…32,000
16 bit	0…65,520	±10 V	Out of Range

DAU 204 0...1 V	Origin Value Range	Adequate BMXAMO0410 Mode	Scaling Values 0% / 100%
11 bit	0…2,047	±10 V	-20,480 / 20,480
12 bit	0…4,095	±10 V	Out of Range
15 bit + sign	0…32,000	±10 V	Out of Range
16 bit	0…65,520	±10 V	Out of Range

DAU 204 0...5 V	Origin Value Range	Adequate BMXAMO0410 Mode	Scaling Values 0% / 100%
11 bit	0…2,047	±10 V	-4,096 / 4,096
12 bit	0…4,095	±10 V	-8,192 / 8,192
15 bit + sign	0…32,000	±10 V	Out of Range
16 bit	0…65,520	±10 V	Out of Range

DAU 204 0...10 V	Origin Value Range	Adequate BMXAMO0410 Mode	Scaling Values 0% / 100%
11 bit	0…2,047	±10 V	-2,048 / 2,048
12 bit	0…4,095	±10 V	-4,096 / 4,096
15 bit + sign	0…32,000	±10 V	-32,000 / 32,000
16 bit	0…65,520	±10 V	Out of Range

Additional Actions for the User

Manually created emulation code is required for every mode in which the scaling values are `Out of Range`.

For an example how to create emulation code please refer to the description of the ADU 214 module *(see page 295)*.

Created Emulation Code

There is only automatic creation of emulation code for status information (see below).

Supported Status Information

The status bits are provided by the Modicon M340 module in the following way:

Bit (1..16)	Meaning (Compact) Register 30xxx	Equivalent at BMXAMO0410	Meaning (Modicon M340)
1	1 = current error	OR of Ch.1 … Ch.4	-
2	1 = Ch.1 current error	[T_ANA_OUT_BMX].ACT_WIRE_FLT	actuator wire open or short
3	1 = Ch.2 current error	[T_ANA_OUT_BMX].ACT_WIRE_FLT	actuator wire open or short
4	1 = Ch.3 current error	[T_ANA_OUT_BMX].ACT_WIRE_FLT	actuator wire open or short
5	1 = Ch.4 current error	[T_ANA_OUT_BMX].ACT_WIRE_FLT	actuator wire open or short
6..14	not used	-	-
15	1 = module memory error	not known/available	-
16	1 = module error	[T_GEN_MOD].MOD_ERROR	module detected error bit

To ease the access to this status information, the converter creates emulation code, which transfers the bits to concerned input variable. The created code can be viewed inside the section `ATSTCopIn`.

Furthermore BMXAMO0410 does not provide a common error for current, the emulation code combines the by-channel errors to simulate this status information; a logical `OR` is used for doing this.

DAU 208

Modicon M340 Replacement

The legacy Compact DAU 208 module has 8 output channels and is substituted by 2 Modicon M340 BMXAMO0410 modules.

Supported and Unsupported Modes

The conversion is performed by setting special scaling values inside the configuration of the BMXAMO0410 modules.

This conversion mechanism is sufficient to support every mode at every resolution of DAU 208 modules.

Also the values for Overflow and Underflow and the configured Timeout values are taken over to the BMXAMO0410 configuration to get an identical behavior in cases of overflow and timeout.

DAU 208 Mode	Origin Value Range	Adequate BMXAMO0410 Mode	Scaling Values 0% / 100%	Overflow Values Below /Above
±10 V, 12 bit	48…4,048	±10 V	48 / 4,048	47 / 4,049
±10 V, 15 bit+sign	-32,000…32,000	±10 V	-32,000 / 32,000	-32,016 / 32,016

Additional Actions for the User

No additional actions are necessary.

Created Emulation Code

There is no automatic creation of emulation code for this module during import.

Supported Status Information

There is no emulation of status information for the DAU 208 module during import.

DAU 2x2

Modicon M340 Replacement

The legacy Compact DAU 208/DAU 252 module is substituted by a Modicon M340 BMXAMO0210 module.

Supported and Unsupported Modes

The conversion is performed by setting special scaling values inside the configuration of the BMXAMO0210 module.

This conversion mechanism is sufficient to support every mode at every resolution of DAU 2x2 modules.

Also the values for Overflow and Underflow and the configured Timeout values are taken over to the BMXAMO0210 configuration to get an identical behavior in cases of overflow and timeout.

DAU 2x8 Mode	Origin Value Range	Adequate BMXAMO0210 Mode	Scaling Values 0% / 100%	Overflow Values Below /Above
0…20 mA, 12 bit	48…4,048	0…20 mA	48 / 4,048	47 / 4,049
0…20 mA, 15-bit+sign	-32,000…32,000	0…20 mA	-32,000 / 32,000	-32,016 / 32,016
±10 V, 12bit	48…4,048	±10 V	48 / 4,048	47 / 4,049
±10 V, 15 bit+sign	-32,000…32,000	±10 V	-32,000 / 32,000	-32,016 / 32,016

Additional Actions for the User

No additional actions are necessary.

Created Emulation Code

There is no automatic creation of emulation code for this module during import.

Supported Status Information

There is no emulation of status information for the DAU 2x2 module during import.

Appendices

Overview

This section contains the appendicies.

What Is in This Appendix?

The appendix contains the following chapters:

Chapter	Chapter Name	Page
A	FAQ Build Errors	311
B	FAQ Conversion Errors	337

Appendix A
FAQ Build Errors

Overview

This chapter includes information on build errors.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
General	312
Object Link Creation Error	313
Object Must be Connected to a Successor	314
Link Together with Variable isn't Allowed	315
Data Type 'xxxx' Expected	316
Empty DFB to Replace Obsolete EFB	322
Undefined Symbol 'xxxx'	323
Call of Non-Function Block	325
Parameter 'xxxx' Has to Be Assigned	328
' xxxx' Is Not a Parameter of 'yyyy'	329
DDT Component Is Missing	330
EHC Parameters Out of Range	331
Not a Valid Address	332
140 NOG 111 00 Configuration Not Converted	333
E1163 Use of Unconfigured Direct Address	334
The Instance Is Located on an Address That Is Not Configured	335

General

Overview

After converting a Concept application, the **Rebuild All** Menu should be invoked.

If the application is not built with this command, all error messages in the build output window should be examined by double-clicking on them. This opens the section with the origin of the problem.

The whole section should be compared to the original in Concept and functional differences should be manually corrected in the converted application.

Example

Examples for messages:

- `{SCADA_Info : [MAST]} : (r: 172, c: 4) E1218 Object must be connected to a successor, at least the Right-Power-Rail`
- `{FC124_Visual_call_up_part_3 : [MAST]} : (r: 31, c: 5) E1189 converter error: 'Object Link creation error (Link pin can not be located in original object) : Link to pin (linkSource: row=30, col=4, Object=, Pin=OUT1.) can not be created. Object has not been created during import.'`

Potential Messages

Short forms of potential messages are given in the following list, which are linked to explanation details:

- *Object Link Creation Error*, *page 313*
- *Object Must be Connected to a Successor*, *page 314*
- *Link Together with Variable isn't Allowed*, *page 315*
- *Data Type 'xxxx' Expected*, *page 316*
- *Empty DFB to Replace Obsolete EFB*, *page 322*
- *Undefined Symbol 'xxxx'*, *page 323*
- *Call of Non-Function Block*, *page 325*
- *Parameter 'xxxx' Has to Be Assigned*, *page 328*
- *' xxxx' Is Not a Parameter of 'yyyy'*, *page 329*
- *DDT Component Is Missing*, *page 330*
- *EHC Parameters Out of Range*, *page 331*
- *Not a Valid Address*, *page 332*

Object Link Creation Error

Cause

One reason of this message, which occurs during import already and when analyzing, can be that the converter does not have implemented the substituion of the extensible diagnostic blocks with dual FBs.

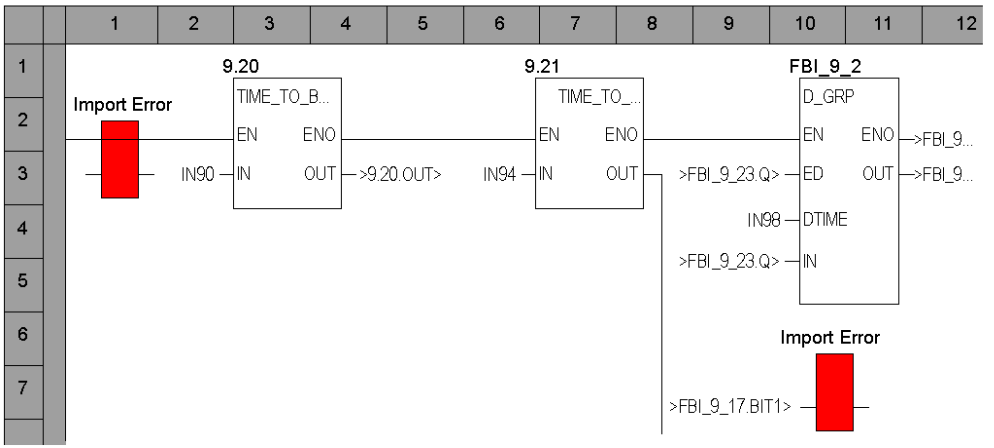
Explanation

D_GRP and D_PRE both need an AND block attached to their IN input. This additional AND has to be implemented so, thatit gets all inputs of the former extensible area. Add the missing block by hand.

Example

```
{_9_TIME : [MAST]} : (r: 2, c: 1) E1189 converter error: 'Object Link
creation error (Link pin can not be located in original object) : Link
to pin (linkSource: row=1, col=0, Object=FBI_9_2_DRAW, Pin=OUT.) can not
be created. Object has not been created during import.'
{_9_TIME : [MAST]} : (r: 2, c: 1) E1002 syntax error
{_9_TIME : [MAST]} : (r: 6, c: 13) E1189 converter error: 'Object Link
creation error (Link pin can not be located in original object) : Link
to pin (linkDestination: row=5, col=12, Object=FBI_9_2, Pin=.) can not
be created. Object has not been created during import.'
{_9_TIME : [MAST]} : (r: 6, c: 13) E1002 syntax error
```

Figure



Object Must be Connected to a Successor

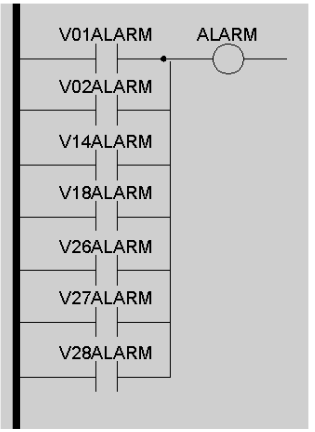
Cause

A message as follows, can have its reason in a Concept 2.1 LD bug:

```
{TANKVLVS <DFB> : [TVALVE]} : (r: 93, c: 3) E1218 Object must be connected
to a successor, at least the Right-Power-Rail
```

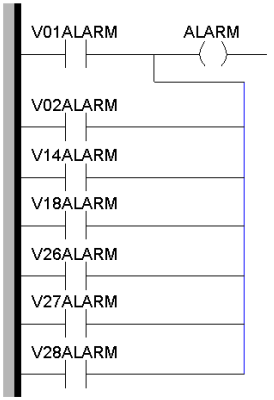
When connecting contacts to an OR (a vertical short), it sometimes happens that the intended first output contact is connected to the input of the OR.

Concept even shows this in its graphics with a small dot at the input of the OR:



In this case, the ALARM coil is connected ONLY to the ontact V01ALARM. The OR output is connected to NOTHING.

Consequently, the Unity V1.1 converter translates this to:

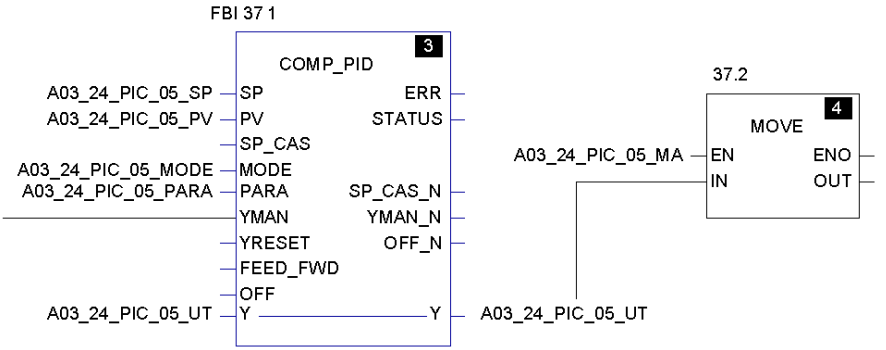


Link Together with Variable isn't Allowed

Overview

This error is reported in connection with **INOUT** pins.

Example

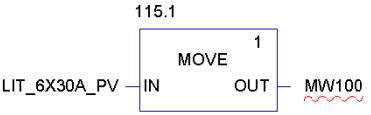


Solution

Delete the link and insert the variable to the destination parameter of the link.

Data Type 'xxxx' Expected

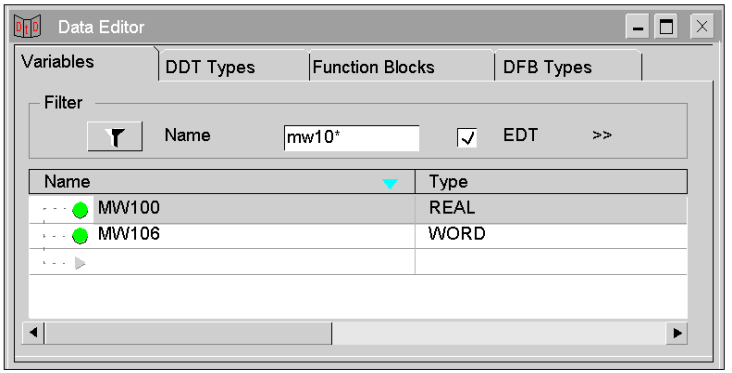
Example

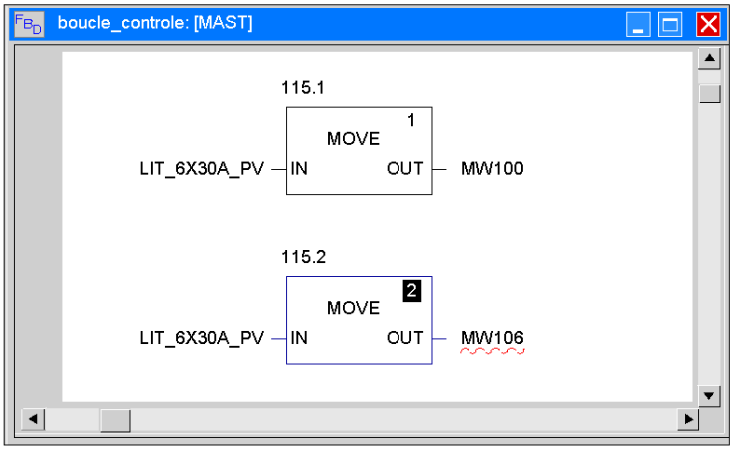


Solution

Replace used data types according to the required type.

The following picture shows the error correction for the 115.1 function block, where the type of output (MW100) has been modified to the type used for the input (REAL).





The Concept converter of V1.0 estimates the type from the address and does not take the actual type into account. This is subject to a later version.

Retyping EFB Parameter

Another reason of this message can be e.g., that the EFB parameter has been retyped to ANY_ARRAY_WORD. See also Parameter type changed.

Combinations of Variables, Variable Instances and Parameters from Concept

Coming from Concept are the following combinations of variables, variable instances and parameters (pins):

Element	Description

Variable Declarations	• The variable declaration has a type of its own and can have a register
	• Several variable declarations can have individual types and the same register
	`CP_GV1 "Symbol" 4:100 DPM_Time INIT: FALSE 0 EXP: FALSE RET: FALSE READONLY: FALSE MAS: FALSE TEXT:`
	`CP_GV1 "SymbolElem" 4:100 IEC_INT_ID INIT: FALSE 0 EXP: FALSE RET: FALSE READONLY: FALSE MAS: FALSE TEXT:`

Varaiable Instances	• Variable INSTANCES coming with a SYMBOL have no own type and no register and use the type of the necessary variable declaration.
	`CP_GVI NAMED_VAR: "SymbolElem" 10 9 FP_IO_OUTPUT`
	• The variable instance can come with a register, in this case it has a type of its own in the instance declaration and no symbol.
	`CP_GVI REG_VAR: 4:100 27 16 FP_IO_INPUT DPM_Time EXP: FALSE RET: TRUE MAS: FALSE`
	• It is not necessary to have a variable declaration for register variable instances:
	❍ Textual anonymous declarations (AT %MWxx:DDT;) are equivalent to variable instance declarations with register and declare the type as well.
	❍ The type is forced to be the same as an existing variable declaration. If they are conflicting, the declaration is refused in Concept.
	`CP__ST AT %QW102: REAL;`

Parameters	The pin a variable instance is attached to has a type of its own, which is not necessarily the same as that of the variable instance.
	It cannot be changed and can be generic.
	`VS_FRM "IN1" HIDE POSL 2 FP_IO_INPUT FP_INP_NORMAL FP_LOC_OUTSIDE INT TEXT:`
	`VS_FRM "IN" HIDE POSL 2 FP_IO_INPUT FP_INP_NORMAL FP_LOC_OUTSIDE ANY TEXT:`

NOTE: So there are three + n different types possible to be declared for a register variable in Concept (1(2=>n),4,6) .

Type Declaration in Unity

Unity accepts one type declared with a symbol associated to a register. If the register is used directly, only its default type is assumed.

To generate code, the type and size of a variable attached to a pin must be determined to one type. Different pins might have different types.

Register Variable Instance

If there is a register variable instance with its type and additionally a variable declaration with a different type and the same register, Concept generates code according to the type supplied with the register or with the symbol individual for each pin.

Default Type

Unity knows only a default type for registers. If this type is to be changed, a variable with a symbol must be declared to carry the type, but two symbols with different types for one register are not accepted.

Unity does not import the second variable, if this application is imported.

Behaviour of Variables, Variable Instances and Parameters in Unity

If...	And There Is...	Then...
symbols are used with a variable instance	-	• the type declared in the declaration with the symbol is to be used • the type of a possibly present register variable instance is not
a register variable with a type different from the default is to be used	already a variable declaration with the same register, but a different type	an error message for this impossibility is issued.
a register variable with a type different from the default is to be used	a variable declaration with the same register with the same type	its symbol is to be used instead of the direct address.
a register variable with a type different from the default is to be used	no variable declaration with the same register	an artificial symbol is to be declared and used instead of the direct address.
a pin in the Unity template has the type `ANY_ARRAY_WORD`	-	an attached register variable could get the type `ARRAY[0..0] OF WORD`, if it previously had the type `WORD`.
the register is used also at pins with the type `WORD`	-	the register gets the index [0] attached.

Other Type Mismatch Cases

Other type mismatch cases are reported with a build(=analyze) message and left to be resolved by the user.

Word Arrays in Communication Blocks

Communication blocks have Word arrays as parameters, whic are defined in Concept with a reference to the first Word only.

The size of the array is often given by the content of a variable, which is defined during run-time. So the size cannot be determined by the converter.

The user must determine the maximum size and declare the array accordingly by himself.

Example from Concept

Step	Action
1	MicroScaleConfig: [MAST] (*Micro Scale Clear-Tare (402310 - 402319) *) IF MicroScCTActive = 0 THEN MicroScCTMSTR1 := 1; MicroScCTMSTR2 := 0; MicroScCTMSTR3 := 3; MicroScCTMSTR4 := 47; MicroScCTMSTR5 := 1028; MicroScCTMSTR6 := 1; MicroScCTMSTR7 := 0; MicroScCTMSTR8 := 0; MicroScCTMSTR9 := 0; MicroScCTMSTR10 := 62; END_IF; All array members appear as single variables. In Unity, they must be combined to an array.
2	This is prepared by the converter by declaring a variable with the range of [0..0].

Step	Action
3	This leads to a set of analyze messages to make the user aware of the correction need. In this case, the correction of the user should look like:

4	The source code related to this is in this case:
	`{RESET_CONV <DFB> : [RC_HSK_1]} : (r: 2, c: 2) E1063 call of non-function block`
	`{RESET_CONV <DFB> : [RC_HSK_1]} : (r: 4, c: 29) E1067 'Q' is not a member of 'SECT_CTRL'`

MicroScaleConfig: [MAST]

```
(*Micro Scale Clear-Tare (402310 - 402319) *)
IF MicroScCTActive = 0 THEN
    MicroScCTMSTR [1] := 1;
    MicroScCTMSTR [2] := 0;
    MicroScCTMSTR [3] := 3;
    MicroScCTMSTR [4] := 47;
    MicroScCTMSTR [5] := 1028;
    MicroScCTMSTR [6] := 1;
    MicroScCTMSTR [7] := 0;
    MicroScCTMSTR [8] := 0;
    MicroScCTMSTR [9] := 0;
    MicroScCTMSTR10 [1] := 62;
END_IF;
```

Data Editor

Variables | DDT Types | Function Blocks | DFB Types

Filter — Name: * — EDT — DDT — IODDT

Name	Type	Address	Value
MicroScCTMSTR10	ARRAY[1..1] OF WORD	%MW2319	
MicroScCTMSTR9	WORD	%MW2318	
MicroScCTMSTR8	WORD	%MW2317	
MicroScCTMSTR7	WORD	%MW2316	
MicroScCTMSTR6	WORD	%MW2315	1
MicroScCTMSTR5	WORD	%MW2314	1028
MicroScCTMSTR4	WORD	%MW2313	47
MicroScCTMSTR3	WORD	%MW2312	3
MicroScCTMSTR2	WORD	%MW2311	
MicroScCTMSTR	ARRAY[1..9] OF WORD	%MW2310	

Empty DFB to Replace Obsolete EFB

Cause

A few standard EFB have not been ported from Concept to Unity.

If the Converter finds one of them, it inserts an Empty DFB with the same parameters as the original to allow building of the application and to give the possibility to the user to substitute the original with code of his own.

Solution

Insert the code into the body of the empty DFB, which contains the command to generate a message like the following in it:

```
{S1 : [REAL_W2]} : (r: 1, c: 2) E1189 converter error: 'Empty DFB to
replace obsolete EFB - fill by user'
```

The command for the message has to be deleted, if valid code has been filled into the DFB body to allow building of the application.

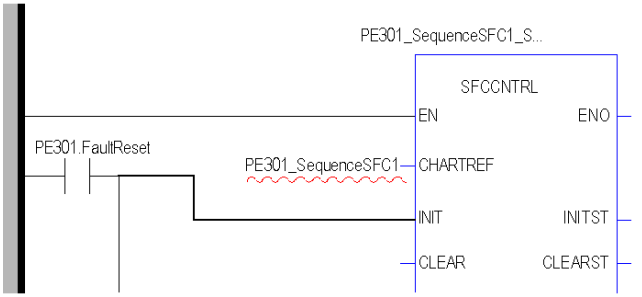
NOTE: Because the ANY type is not allowed on DFBs in Unity, an additional problem occurs if this type has been used in the substituted EFB (e.g. XXMIT EFB).

Customer Defined EFBs

Customer defined EFBs are not converted. If you want to convert an application using Customer defined EFBs, get in contact with Schneider support.

Undefined Symbol 'xxxx'

Wrong SFC Section Name

Explanation	This message is given sometimes in conjunction with the block SFCCNTRL. The message means, that the SFC section name, which has to be attached to the CHARTREF input is not the name of an existing SFC section inside of the current application.
Solution	Create the according section and the message disappears. 

Wrong Instance Name

Explanation	Another reason of such a message can be, that a Concept function block now in Unity is a function or a procedure.
	The converter does conversion work in the textual languages ST and IL half-automatic in the case of this incompatibility. The instance name of Concept is removed and substituted by the type name, also for addressing outputs, which is then an illegal syntax:
	`LOOKUP_TABLE1(X := ODT,`
	`XiYi1 := -30.0, XiYi2 := PARA.p1,`
	`XiYi3 := -20.0, XiYi4 := PARA.p2,`
	`XiYi5 := -10.0, XiYi6 := PARA.p3,`
	`XiYi7 := 0.0, XiYi8 := PARA.p4,`
	`XiYi9 := 10.0, XiYi10:= PARA.p5,`
	`XiYi11:= 20.0, XiYi12:= PARA.p6);`
	`OUT := LOOKUP_TABLE1.Y;`

Solution	The last line must be corrected manually. Using the output assign operator, this statement must be changed and moved inside the call parentheses:
	`LOOKUP_TABLE1(X := ODT,`
	`XiYi1 := -30.0, XiYi2 := PARA.p1,`
	`XiYi3 := -20.0, XiYi4 := PARA.p2,`
	`XiYi5 := -10.0, XiYi6 := PARA.p3,`
	`XiYi7 := 0.0, XiYi8 := PARA.p4,`
	`XiYi9 := 10.0, XiYi10:= PARA.p5,`
	`XiYi11:= 20.0, XiYi12:= PARA.p6,`
	`Y=>OUT);`

Call of Non-Function Block

Cause

This message can appear, when a Concept function block now in Unity is a function or a procedure.

The converter removes the instance name of the Concept-block and substitutes it with the type name and moves assignments of outputs inside the invocation parentheses.

For the blocks GET_BIT and SET_BIT, the treatment does not completely apply. During analysis, messages occur:

```
{INPUTS : [MAST]} : (r: 7, c: 4) E1063 call of non-function block
```

The function names remain marked as erroneous after conversion, because the functions are converted with the procedure syntax in ST, not with the needed function syntax, as the corrected version shows. Also, the converter has dropped the indices for the result variable of GET_BIT.

Example

Original Concept Codes	After Conversion	Corrected Version
VAR INPUT_WORD : GET_BIT; END_VAR; FOR I_BASE := 1 TO 20 DO FOR I_POINT := 1 TO 16 DO INPUT_WORD (IN:=IO_SCAN_IN_WORD[I_BA SE], NO:=I_POINT); INPUT[I_BASE,I_POINT] := INPUT_WORD.RES; END_FOR; END_FOR;	FOR I_BASE := 1 TO 20 DO FOR I_POINT := 1 TO 16 DO GET_BIT(IN:=IO_SCAN_IN_WORD [I_BASE], NO:=I_POINT, RES => INPUT); ; END_FOR;	FOR I_BASE := 1 TO 20 DO FOR I_POINT := 1 TO 16 DO INPUT[I_BASE,I_POINT]:= GET_BIT(IN:=IO_SCAN_IN_WORD [I_BASE], NO:=I_POINT); END_FOR; END_FOR;
VAR OUTPUT_WORD : SET_BITX; END_VAR; FOR O_BASE := 1 TO 20 DO FOR O_POINT := 1 TO 16 DO OUTPUT_WORD (RES := IO_SCAN_OUT_WORD[O_BASE], IN := OUTPUT[O_BASE,O_POINT], NO := O_POINT); END_FOR; END_FOR;	FOR O_BASE := 1 TO 20 DO FOR O_POINT := 1 TO 16 DO SET_BIT(RES := IO_SCAN_OUT_WORD[O_BASE], IN := OUTPUT[O_BASE,O_POINT], NO := O_POINT); END_FOR; END_FOR;	FOR O_BASE := 1 TO 20 DO FOR O_POINT := 1 TO 16 DO IO_SCAN_OUT_WORD[O_BASE]:= SET_BIT(IN := OUTPUT[O_BASE,O_POINT], NO := O_POINT); END_FOR; END_FOR;

Conjunction With Other Messages

This message can appear in conjunction with other messages:

- `{RESET_CONV <DFB> : [RC_HSK_1]} : (r: 2, c: 2) E1063 call of non-function block`
- `{RESET_CONV <DFB> : [RC_HSK_1]} : (r: 4, c: 29) E1067 'Q' is not a member of 'SECT_CTRL'`

Related Source Code

The source code related to this is in this case:

- `RESET_CONV147(IN := (CTRL.TB.RC_INI AND V_SYNCHRO), PT := t#500ms);`
- `T_CONVRESET := RESET_CONV147.Q;`

Double Use of the Instance Name

Unity Pro associates the instance name to the derived data type SECT_CTRL, even though it is intended to address a timer. This usually happens, if the Concept application used the instance name twice. To find this out, proceed as follows:

Step	Action
1	Open the Concept *.asc* export file, and search the instance name without the figures at the end with a search command of the text editor. **Result:** In this case here we find: `STR_RCI:` (* RC Eingänge = SPS Ausgänge *) `STRUCT` `AUTO : BOOL ;` (* Betriebsart Automatik / Hand *) `AXIS_EN : BOOL ;` (* Achsen angewählt *) `Z_UP : BOOL ;` (* Z-Achse auf *) `RESET_PROG : BOOL ;` (* Programm abbrechen *) `RESET_CONV : BOOL ;` (* Förderer synchronisieren *)

2	The line introduction...**STR** has been omitted.
	`CP_GVS "RESET_CONV" SECT_CTRL INIT: FALSE 0 EXP: FALSE RET: TRUE READONLY: FALSE MAS: FALSE TEXT:`
	`CP_SEC "RESET_CONV" SECTK_F_SECTION LANG_ST SVB: FALSE ID: 27 EXEC: 26 TEXT:`
	`CP__ST`
	`CP__ST VAR`
	`CP__ST RESET_CONV : TP; (* Impuls Reset Conveyor *)`
	The same name has been used as a structure component name, a section name with its control variable, and for a "TP" timer instance.

3	Change the type of the instance in the Data editor to "TP".

Substitute Procedures in ST/IL

Some EFBs from Concept are implemented as procedures in Unity Pro without instance names.

Open the **Conversion Settings** tab via **Tools → Options** in Unity Pro to enable/disable the **Substitute Procedures in ST/IL** check box before converting.

- When this checkbox is enabled, the instance name of the Concept call will be replaced with the type name.
- When this checkbox is not enabled, a DFB will be created, which will then access the procedure.

Parameter 'xxxx' Has to Be Assigned

Cause

For inputs, left open pins at blocks get an automatically generated variable with the approriate type. For outputs, this is not done yet.

In the case of generic data types, it cannot be done easily.

Solution

In these cases the user still must declare appropriate variables and attach them to the left-open pins.

' xxxx' Is Not a Parameter of 'yyyy'

Cause

The diagnostic EFBs, which have been extensible in Concept, do not get the right calling syntax in IL.

```
{_9_TIME : [MAST]} : (r: 43, c: 17) E1031 'IN1' is not a parameter of
function block 'GRP_DIA_9'
```

```
{_9_TIME : [MAST]} : (r: 44, c: 17) E1031 'IN2' is not a parameter of
function block 'GRP_DIA_9'
```

Solution

In the case of the in Concept extensible diagnostic EFBs, the extension can be done with a logical AND function the output of which is tied to the single input of the diagnostic function. This is done with the first three lines in the correction.

The used output must be processed by `BOOL_TO_TIME`, which is bypassed in the automatic conversion and which is corrected in the last three lines.

Example

Original Concept Code	After Conversion	Corrected Version
CAL GRP_DIA_9 (ED :=DUMMY_1_91, DTIME :=IN92, IN1 :=DUMMY_1_94, IN2 :=DUMMY_1_96) LD GRP_DIA_9.ERR BOOL_TO_TIME ST OUT90	CAL GRP_DIA_9 (ED :=DUMMY_1_91, DTIME :=IN92, IN1 :=DUMMY_1_94, IN2 :=DUMMY_1_96, ERR => OUT90) BOOL_TO_TIME	LD DUMMY_1_94 AND DUMMY_1_96 ST GRP_DIA_9.IN CAL GRP_DIA_9 (ED :=DUMMY_1_91, DTIME :=IN92) LD GRP_DIA_9.ERR BOOL_TO_TIME ST OUT90

DDT Component Is Missing

Cause

Keywords may not be used as symbols of DDT components or as variable names. Such a case is the name slot.

Solution

If DDT components are missing or import conflicts are imported, proceed as follows:

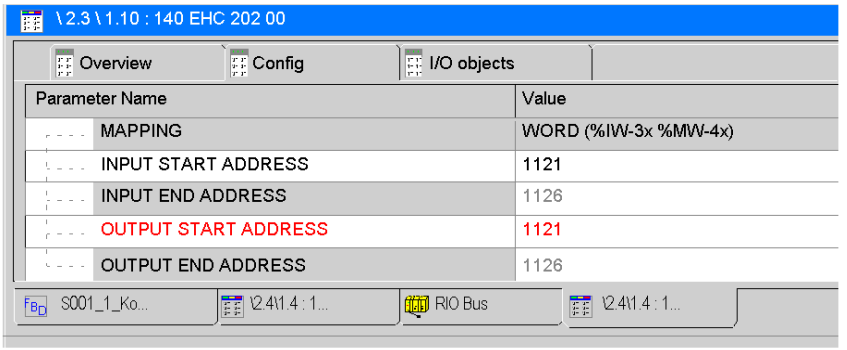
Step	Action
1	Search for the occurrence of the name in the .asc file with another meaning.
2	Change the name for the conflicting meaning.

EHC Parameters Out of Range

Cause

For the High speed counter module, parameter limits are not treated right.

Example




```
Parameter <OUTPUT START ADDRESS> out of range (Error with param 17)
```

Solution

Such Parameters must be manually corrected.

Not a Valid Address

Cause

A message like the following one is generated at analyze time, if a Hot_Stand-By system is incompletely defined.

```
Analyzing...
{Cpu (1.2 ) 140 CPU 671 60} : %MW0 is not a valid address in Quantum
```

Solution

Step	Action
1	Open the local rack of the configuration and the CPU configuration itself and select the **Hot Stand-By** tab of the CPU configuration.
2	In its lower part appear **State Ram** and **Non Transfer area**. Usually both the **Start** and **Length** fiels both contain a Zero, directly passed through from the Concept application.
3	To remove the error message, enter 1 into the start field.

140 NOG 111 00 Configuration Not Converted

Concept

The 140 NOG 111 00 is used as a NOM in Concept.

Configuration Not Converted

The conversion creates also a NOM in Unity Pro but the I/O configuration gets lost.

E1163 Use of Unconfigured Direct Address

Description

Address values configured in the legacy application are higher than the maximum allowed **State RAM**.

NOTE: Quantum CPUs maximum value for **%M 0x State RAM**: 65280.

The Instance Is Located on an Address That Is Not Configured

Solution

Reserve more memory words in the CPU **State RAM**.

Appendix B
FAQ Conversion Errors

FAQ Conversion Errors

Overview

This section gives the list of keywords that must not be used to define object names (variables and types of variables).

If a Concept application contains such variable or variable type names, the application cannot be converted to Unity Pro.

The following message to be acknowledged by the user appear: **Error in definition of located variable: Keywords may not be used as variable names.**

Keywords

The following lists gives the keywords in alphabetical order:

A
- Address
- AI_CONSTANT
- AI_SECTION
- AI_VARIABLEINST
- AI_VARIABLEINST_REG
- ANY
- AsciiMsg
- ATTRIBUTE

B
- BCD16
- BCD32
- BCD64
- BCD8
- BEGIN
- binData
- BOOL
- byte

C
- CL_DFB
- CL_FRM
- CL_PLC
- COLUMN_WIDTH

- CONCEPT_
- CONCEPT_VERSION
- configImage
- configTable
- CP__IL
- CP__ST
- CP_ABR
- CP_ACT
- CP_AJN
- CP_APP
- CP_COM
- CP_CON
- CP_DBI
- CP_DFB
- CP_FBI
- CP_GEN
- CP_GV1
- CP_GV2
- CP_GV4
- CP_GVI
- CP_GVS
- CP_GVT
- CP_IFP
- CP_INV
- CP_JMP
- CP_LNK
- CP_OPT
- CP_PBR
- CP_PJN
- CP_PRG
- CP_PRI
- CP_PRO
- CP_PRP
- CP_SEC
- CP_STP
- CP_STR
- CP_TRN
- CP_VRS
- CR_END

D

- DATE
- dint
- DisplayFormat

- dpMasterData
- dpSlaveData
- DRAW
- DropHead
- dropNumber
- DT
- DWORD

E

- END
- END_ENTRY
- END_RDE_TEMPLATE
- ENTRY
- EVN
- extType

F

- FALSE
- FBIPH_INPUT
- FBIPH_OUTPUT
- FP_IO_INOUTPUT
- FP_IO_INOUTPUT_COMP
- FP_IO_INPUT
- FP_IO_INPUT_COMP
- FP_IO_OUTPUT
- FP_IO_OUTPUT_COMP

G

- global

H

- headIndex
- HEIGHT
- HIDE
- HX_n

I

- ID_{Digit}+
- IEC_BCD16_ID
- IEC_BCD32_ID
- IEC_BCD64_ID
- IEC_BCD8_ID
- IEC_BOOL_ID
- IEC_BYTE_ID
- IEC_DATE_ID
- IEC_DINT_ID
- IEC_DT_ID

- IEC_DWORD_ID
- IEC_INT_ID
- IEC_LINT_ID
- IEC_LREAL_ID
- IEC_LWORD_ID
- IEC_REAL_ID
- IEC_SINT_ID
- IEC_STRING_ID
- IEC_TIME_ID
- IEC_TOD_ID
- IEC_UDINT_ID
- IEC_UINT_ID
- IEC_ULINT_ID
- IEC_UNKNOWN_ID
- IEC_USINT_ID
- IEC_WORD_ID
- INIT
- inputBytes
- inputReference
- int
- INV
- IODrop
- IOModule

L

- LANG_FBD
- LANG_IL
- LANG_LD
- LANG_LL
- LANG_SFC
- LANG_ST
- LINT
- LL_INS
- LL_NET
- LL_NOD
- LL_REG
- LL_SON
- LL_SRD
- LL_SRM
- LL_VAR
- local
- locInc
- LREAL
- LWORD

M
- macAddr
- MAS
- maxConstant
- modData

N
- NAMED_VAR
- nodeParam

O
- outputBytes
- outputReference

P
- pcHealthTimeout
- pcHoldLastValue
- PlcCnfDb
- PLCConfig
- plcName
- POSR

R
- rack
- RDE_TEMPLATE
- RDE_TEMPLATE_VERSION
- READONLY
- Real
- REG_VAR
- RET
- rs232Params

S
- scratchPad
- SECTK_F_SECTION
- SetValue
- SFC_STEP_INIT
- SFC_STEP_NORMAL
- SINT
- SLOT
- STRING
- svcFile

T
- Tagname
- TEXT
- time

- TIMN
- TOD

U

- udint
- uint
- ULINT
- UNKNOWN
- USINT

V

- VAL
- Value
- VS_FFB
- VS_FRM

W

- WIDTH
- WINDOW_LOCATION
- WITHOUT_ATT
- WORD

Index

0-9

140 NOG 111 00 configuration
  not converted, *333*

A

address
  not configured, *335*
analyzing
  projects, *53*, *105*
application behavior
  changes, *91*

B

build errors, *311*
BYTE_TO_BIT_DFB, *117*

C

Compact I/O modules
  special conversion, *275*
Concept
  conversion wizard, *19*
Concept converter
  Edge Managing, *71*
Concept Converter - instructions
  BYTE_TO_BIT_DFB, *117*
  CREADREG, *121*
  CWRITREG, *129*
  DINT_AS_WORD_DFB, *135*
  DIOSTAT, *137*
  GET_TOD, *139*
  LIMIT_IND_DFB, *143*
  LOOKUP_TABLE1_DFB, *147*
  PLCSTAT, *153*, *159*
  READREG, *175*
  RIOSTAT, *183*
  SET_TOD, *187*
  WORD_AS_BYTE_DFB, *191*
  WORD_TO_BIT_DFB, *193*
  WRITEREG, *197*
configuration
  differences, *24*
conversion errors, *337*
conversion wizard for Concept), *19*
converter, *15*
convertion
  procedure, *107*
  process, *105*
CREADREG, *121*
CWRITREG, *129*

D

DINT_AS_WORD_DFB, *135*
DIOSTAT, *137*
direct address
  E1163, *334*

E

E1163
  direct address, *334*
EFBs
  differences, *33*

EN
not connected, *95*
error messages, *53*, *91*, *105*, *110*
export/import LL984 segments and networks
LL984 editor, *245*
exporting
DFBs, *108*
macros, *108*
projects, *17*, *108*
sections, *108*

F

Function Block Diagram
differences, *52*

G

GET_TOD, *139*

H

hardware
correspondences, *23*
hardware platforms
supported, *23*

I

I/O modules supported/not supported
LL984 editor, *249*
import restrictions
LL984 editor, *207*
importing
DDTs, *110*
macros, *53*, *112*
projects, *17*, *109*
importing an LL984 application
LL984 editor, *207*
importing parts of an LL984 application
LL984 editor, *207*
initialization values
array, *113*
cluster, *113*
LL_SRAMxxx, *113*
Instruction List
differences, *49*
instructions
differences, *33*

L

Ladder Diagram
differences, *37*
Ladder Logic
differences, *51*
language objects, *53*
differences, *25*
LIMIT_IND_DFB, *143*
LL_SRAMxxx
array, *113*
initialization values, *113*
LL984 editor
export/import LL984 segments and networks, *245*
I/O modules supported/not supported, *249*
import restrictions, *207*
importing an LL984 application, *207*
importing parts of an LL984 application, *207*
LOOKUP_TABLE1_DFB, *147*

M

M580
PLCSTAT, *153*

N

not configured
address, *335*

O

object types
differences, *53*

P

PLCSTAT, *153*
    M580, *153*
    Quantum, *159*
preconditions, *21*
program execution
    differences, *25*

Q

Quantum
    PLCSTAT, *159*

R

READREG, *175*
requirements, *21*
RIOSTAT, *183*

S

Sequential Function Chart
    differences, *36*
SET_TOD, *187*
special conversion for Compact I/O modules, *275*
Structured Text
    differences, *49*
system objects
    differences, *25*

W

WORD_AS_BYTE_DFB, *191*
WORD_TO_BIT_DFB, *193*
WRITEREG, *197*